



CuentaVoz

«Tu voz cuenta»

Manual de construcción del agente conversacional

Herramientas e instalación paso a paso · vistas de usuario · arquitectura y conexiones · código completo del prototipo (cerebro: Gemini / Google AI Studio) · diseño adaptable móvil/tableta/escritorio · ciberseguridad · dashboard en Power BI · plan de construcción por fases

Documento complementario a la Propuesta de Proyecto

Reto de Hotelería · Hackathon Colsubsidio x 30X · 22 al 26 de julio de 2026

Equipo: Luis Guillermo Nieto Patiño · Valentina Burbano Salazar · Stephanie · Diana

Tabla de contenido

1. El ciclo del agente: paso a paso de cada turno	5
2. Casos de uso: todas las funcionalidades en un mapa	5
3. Paso a paso de una captura: las decisiones del agente	6
4. Herramientas tecnológicas e instalación desde cero: todo gratis, paso a paso	7
4.1 Las siete herramientas y para qué sirve cada una	8
4.2 Instalar las herramientas (Windows, unos 25 minutos)	8
4.3 Conseguir la llave de Gemini en Google AI Studio (gratis, 3 minutos)	9
4.4 Crear la carpeta del proyecto	9
4.5 Llenar los archivos y configurar las llaves	11
4.6 Levantar el proyecto por primera vez	11
4.7 Probar que el agente responde	12
4.8 Conectar Power BI y publicar en GitHub.....	12
4.9 Problemas frecuentes y cómo resolverlos	12
5. Vistas de la aplicación: cómo la ve el usuario	13
5.0a Cómo se conecta una vista con la siguiente	13
5.00 Identidad visual: marca Colsubsidio aplicada	15
5.0 Las vistas organizadas por el menú	16
5.1 Ingreso por perfil.....	17
5.2 Inicio: el punto de partida del día	18
5.3 Pedido por voz: del plato a los insumos	20
5.4 El pedido calculado: solo lo que de verdad falta	21
5.5 La receta que sustenta el cálculo	22
5.6 Conteo por voz (pantalla principal del auxiliar)	23
5.7 Desambiguación: elegir entre opciones.....	25
5.8 Alerta de anomalía: preguntar antes de guardar.....	26
5.9 Crear producto: pendiente de aprobación.....	27
5.10 Modo formulario: contingencia sin voz ni internet.....	28
5.11 Legalización del servicio: lo pedido contra lo usado	29
5.12 Cierre de la legalización y actualización del histórico.....	30
5.13 Tablero de bodegas en vivo.....	31
5.14 Detalle de una bodega	32
5.15 Consulta de artículo: dónde está y cuánto hay.....	33
5.16 Auditoría: comparativo y doble firma.....	34
5.17 Cierre con doble firma digital	36

5.18 Aprobaciones pendientes: cerrar el ciclo de lo creado.....	37
5.19 Bandeja de alertas.....	38
5.20 Reportes y consolidado.....	39
5.21 Análisis de consumo: lo pedido contra lo usado.....	40
5.22 Panel: resumen ejecutivo.....	42
5.23 Panel: bodegas y alertas.....	43
5.24 Ajustes: la aplicación se adapta a la operación.....	44
5.25 Gestión de usuarios y permisos	45
5.26 Registro de trazabilidad	46
5.27 Centro de ayuda y estado del sistema	47
5.28 Mi perfil: datos personales y foto	48
5.29 Cerrar sesión con trabajo sin guardar.....	49
5b. El código del diseño adaptable.....	49
5b.0 Diseño adaptable: la misma aplicación en cualquier dispositivo.....	49
5b.1 Variables de diseño y tipografía.....	50
5b.2 Cuadrícula principal y puntos de quiebre	51
5b.3 Botones responsivos.....	51
5b.4 El micrófono: diferente en cada formato.....	52
5b.5 Hook de detección de dispositivo	53
5b.6 Cómo lo usa la pantalla de conteo	53
6. Cómo piensa el agente: intención y salida estructurada	54
6.1 Instrucciones del sistema (prompt del agente)	54
6.2 Esquema de salida (el JSON que el backend procesa).....	55
7. Las manos del agente: herramientas del backend	55
8. Arquitectura: cómo queda construido y cómo se conecta.....	56
8.1 Las cinco capas.....	56
8.2 Componentes y conexiones: quién habla con quién.....	56
8.3 Secuencia de un turno completo	57
9. Diagrama de clases y modelo de datos	58
10. Estados de la bodega y trabajo en paralelo	60
11. Conversaciones de ejemplo (guiones implementables).....	60
11.1 Conteo con conciliación de nombre y anomalía	60
11.2 «No entiendo esta tabla»: el agente explica el reporte	61
11.3 Consulta de datos con salida impresa	61
12. Código completo del prototipo	61

12.1 Dependencias y contenedores.....	62
12.2 Variables de entorno	62
12.3 Orquestación con Docker	62
12.4 Conexión a la base de datos	63
12.5 Las tablas (el diagrama de clases hecho código).....	63
12.6 Carga del extracto real de My Inventory.....	64
12.7 Servicio de conciliación (la sugerencia oficial de Colsubsidio)	65
12.8 Servicio de validación (incluye el mini reto de los negativos)	66
12.9 El cerebro: conexión con Gemini (Google AI Studio).....	67
12.10 El orquestador: donde el pensar se vuelve acción	67
12.11 Reportes: el consolidado para My Inventory.....	69
12.12 La API completa: endpoints y tablero en vivo	70
12.13 Frontend: el puente con la API.....	71
12.14 Frontend: la pantalla de conteo con voz.....	72
12.15 Frontend: el código de cada vista.....	73
12.16 Frontend: Pedidos, receta y legalización (vistas 5.3 a 5.5 y 5.11 a 5.12)	81
12.17 Frontend: las vistas de administración (5.14, 5.15, 5.19, 5.25 a 5.27)	85
12.18 Frontend: Mi perfil (vista 5.28)	91
12.19 Frontend: Cerrar sesión (vista 5.29)	93
12.20 Frontend: el enrutador que une las veintinueve vistas	95
13. Dashboard en Power BI: conexión, modelo, medidas y diseño	98
13.1 Conexión paso a paso	99
13.2 El componente que inserta el informe	99
13.3 Medidas DAX del tablero.....	100
13.4 Diseño del tablero	101
14. Ciberseguridad: el blindaje de CuentaVoz.....	101
14.1 Defensa en profundidad.....	101
14.2 Identidad: contraseñas cifradas, token firmado y perfiles.....	102
14.3 Blindaje de la API: límites, CORS cerrado y cabeceras.....	103
14.4 Configuración segura: secretos, contenedores, HTTPS y respaldos.....	104
14.5 Mapa de amenazas: cada ataque con su escudo	104
14.6 Datos personales y pruebas de seguridad.....	105
15. Plan de construcción paso a paso (para que todo funcione)	106
16. Matriz de cumplimiento: ninguna funcionalidad se queda en el papel	107

1. El ciclo del agente: paso a paso de cada turno

Cada vez que la persona habla ocurre un «turno», y todos los turnos —contar, corregir, preguntar, pedir ayuda— recorren el mismo ciclo de cinco pasos:

EL CICLO DEL AGENTE: QUÉ PASA EN CADA TURNO (objetivo: menos de 2 segundos)

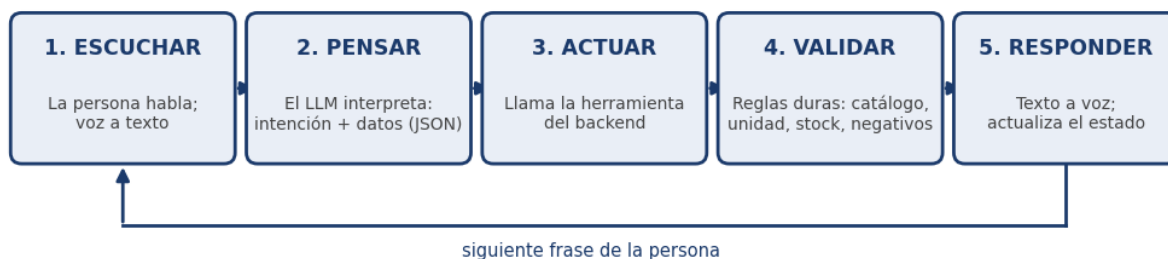


Figura 1. El ciclo del agente en cada turno de conversación.

1. **Escuchar.** La tableta captura el audio y el motor de voz a texto (Web Speech API en español de Colombia) lo convierte en una frase escrita. Este paso ocurre en el navegador de la tableta: no consume servidor ni requiere instalar nada.
2. **Pensar.** La frase viaja al backend y de allí al modelo de lenguaje —Gemini, con la llave gratuita de Google AI Studio— junto con el contexto de la sesión (bodega activa, último artículo, si hay algo pendiente de confirmar) y las reglas del sistema. El modelo devuelve la intención y los datos en un JSON fijo (sección 6).
3. **Actuar.** El orquestador ejecuta la herramienta que corresponde a esa intención: buscar el artículo, validar el conteo, consultar el stock, generar un reporte (sección 7). El modelo nunca toca los datos directamente.
4. **Validar.** La herramienta aplica las reglas duras en código determinista: ¿el artículo existe?, ¿la unidad coincide con el catálogo?, ¿la cantidad es posible (no negativa)?, ¿se desvía del stock esperado? Devuelve ok, o una alerta con el motivo exacto.
5. **Responder.** El agente arma la respuesta (confirmación, pregunta, explicación o alerta), la tableta la reproduce en voz y la muestra en pantalla, y el estado de la sesión queda actualizado para la siguiente frase.

La regla que sostiene todo el diseño: el agente propone, el backend dispone. El modelo de lenguaje interpreta y conversa, pero solo las herramientas del backend tocan los datos, y todas las validaciones críticas son código determinista que se comporta igual siempre. Por eso el sistema es inteligente y a la vez confiable — y por eso este manual puede darle a cada integrante una pieza clara que construir.

2. Casos de uso: todas las funcionalidades en un mapa

Antes de construir hay que ver completo lo que se promete. Este diagrama reúne, en una sola vista, todas las funcionalidades de la propuesta separadas por actor: el auxiliar de inventarios hace el conteo inicial;

el administrador de bodega audita, aprueba y cierra; y el extracto de My Inventory es la fuente de datos que alimenta las validaciones y recibe el archivo de carga:

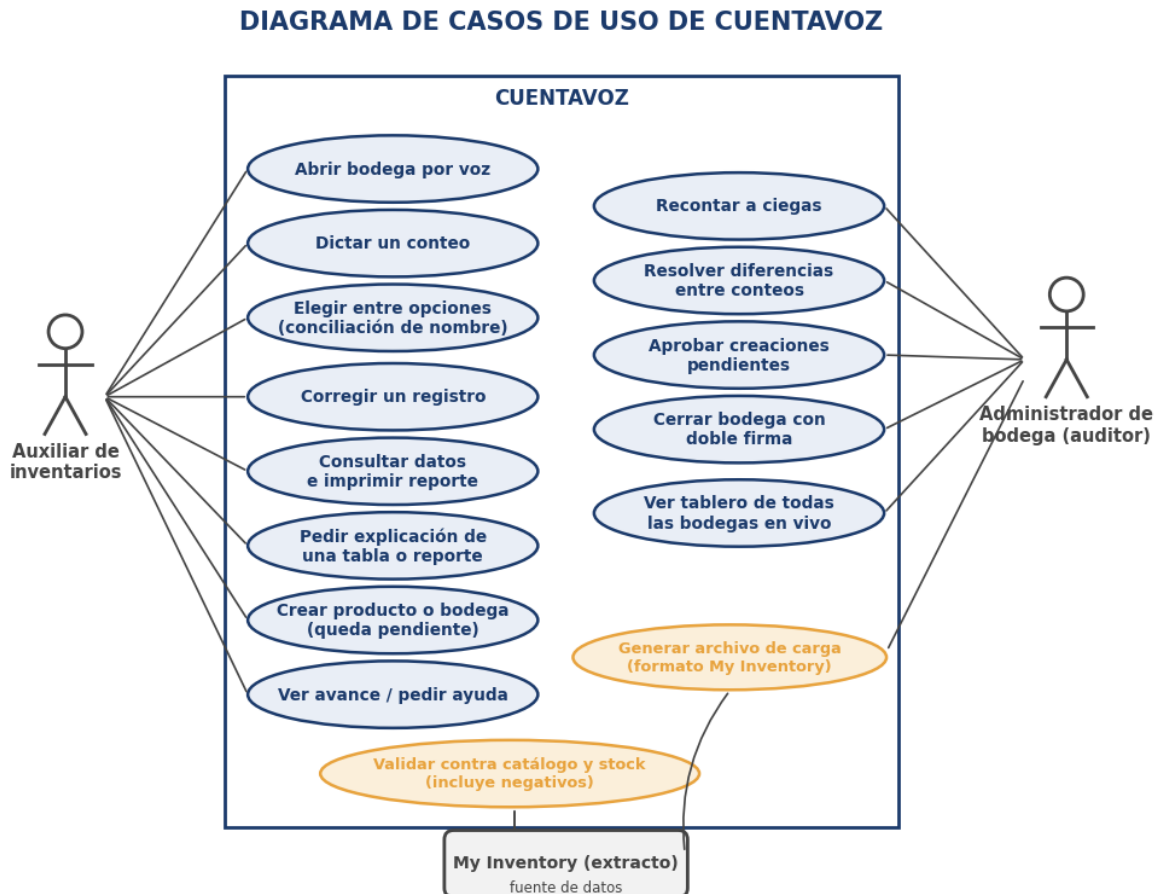


Figura 2. Casos de uso de CuentaVoz por actor; los óvalos amarillos son compartidos con la fuente de datos.

Cada óvalo reaparece más adelante tres veces: como pantalla en la sección 5, como código en la sección 12 y como prueba de cumplimiento en la matriz de la sección 16. Así ninguna funcionalidad se queda en el papel.

3. Paso a paso de una captura: las decisiones del agente

El momento más delicado es capturar un conteo. El diagrama muestra las decisiones del agente desde la frase dictada hasta el registro guardado, con los tres desvíos posibles: confianza media (elegir entre opciones), confianza baja (repetir o crear) y anomalía (preguntar antes de guardar):

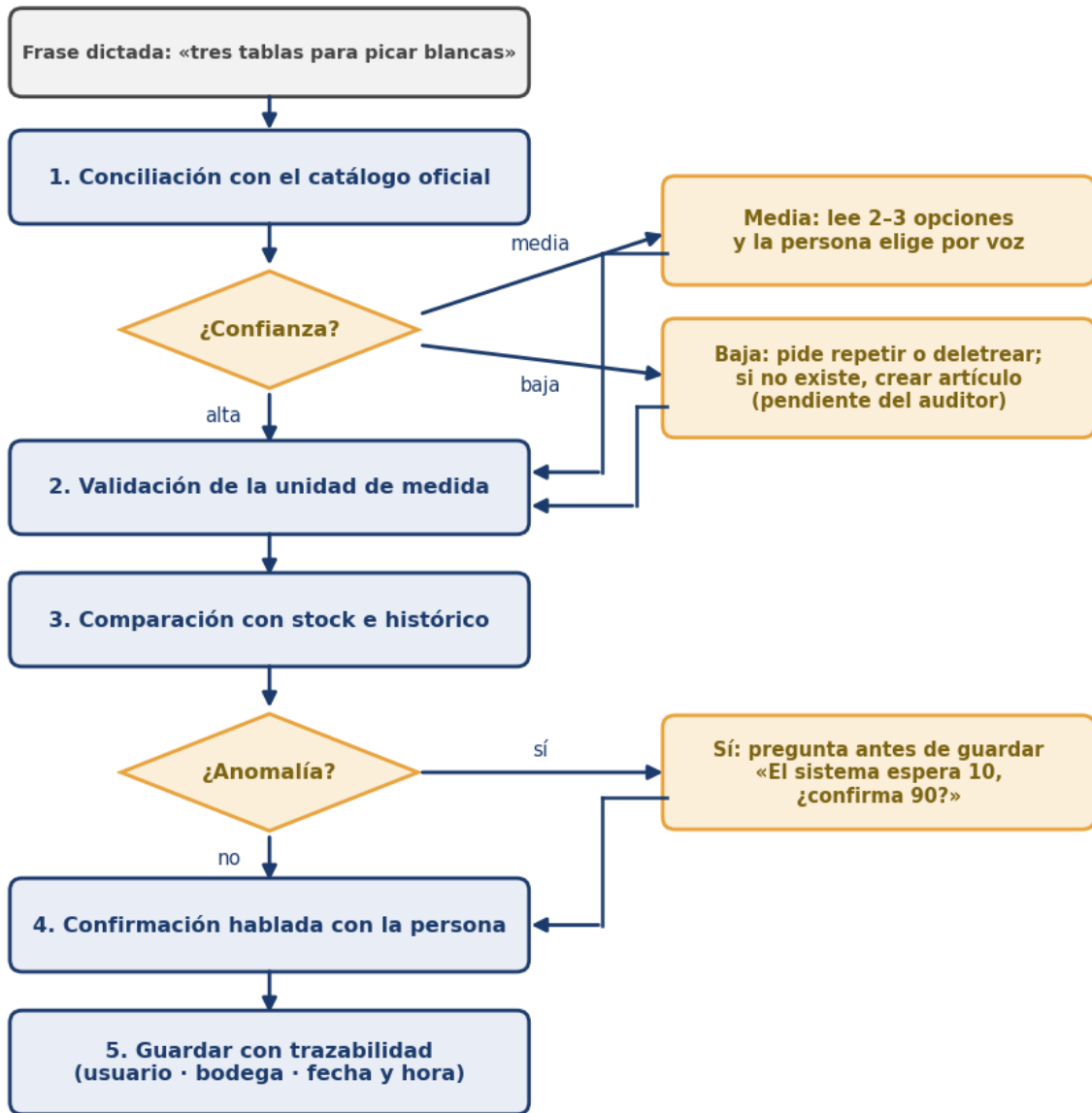
PASO A PASO DE UNA CAPTURA: LAS DECISIONES DEL AGENTE

Figura 3. Decisiones del agente en una captura; toda rama termina en una salida hablada y nada se guarda sin validación.

4. Herramientas tecnológicas e instalación desde cero: todo gratis, paso a paso

Esta sección está escrita para alguien que nunca ha instalado un entorno de desarrollo. Cada paso dice qué descargar, dónde, cuánto pesa, qué escribir y cómo verificar que quedó bien antes de seguir. Todas las herramientas son gratuitas: el costo del prototipo es cero.

4.1 Las siete herramientas y para qué sirve cada una

Herramienta	Para qué sirve en CuentaVoz	Costo	Dónde se descarga
Visual Studio Code	El editor donde se escribe y se organiza todo el código	Gratis	code.visualstudio.com
Node.js (versión LTS)	Motor que ejecuta el frontend en React durante el desarrollo	Gratis	nodejs.org
Python 3.12	Lenguaje del backend, el agente y los servicios de validación	Gratis	python.org/downloads
Docker Desktop	Levanta la base de datos y el backend con un solo comando, sin instalar PostgreSQL a mano	Gratis (uso personal y empresas pequeñas)	docker.com/products/docker-desktop
Git	Guarda el historial del código y lo sube a GitHub (entregable del reto)	Gratis	git-scm.com
Google AI Studio	Entrega la llave de Gemini, el cerebro del agente	Gratis (cuota diaria)	aistudio.google.com
Power BI Desktop	Arma el informe gerencial que luego se inserta en la aplicación	Gratis	powerbi.microsoft.com/desktop

Nota sobre PostgreSQL: no aparece en la lista porque no se instala. Docker lo descarga y lo ejecuta solo cuando levantamos el proyecto, y así nadie tiene que configurar una base de datos a mano.

4.2 Instalar las herramientas (Windows, unos 25 minutos)

Paso 1 — Visual Studio Code. Entre a code.visualstudio.com y pulse el botón azul de descarga para Windows. Ejecute el archivo descargado y acepte todo, pero marque la casilla «Agregar al PATH» cuando aparezca: eso permite abrir proyectos escribiendo code en la terminal.

Paso 2 — Node.js. Entre a nodejs.org y descargue la versión que diga LTS (soporte a largo plazo), nunca la «Current». Ejecute el instalador y acepte las opciones por defecto.

Paso 3 — Python. Entre a python.org/downloads y descargue Python 3.12. Al abrir el instalador, antes de pulsar «Install Now», marque abajo la casilla «Add python.exe to PATH». Si no la marca, la terminal no reconocerá el comando python y tendrá que reinstalar.

Paso 4 — Git. Entre a git-scm.com y descargue la versión para Windows. Acepte todas las opciones por defecto del instalador.

Paso 5 — Docker Desktop. Entre a docker.com/products/docker-desktop y descargue la versión para Windows. Al instalar pedirá activar WSL 2 (el subsistema de Linux de Windows): acepte. Reinicie el

computador cuando lo pida y abra Docker Desktop; la primera vez tarda un par de minutos en arrancar. Debe quedar diciendo «Engine running» en la esquina inferior izquierda.

Paso 6 — Power BI Desktop. Descárguelo desde la Microsoft Store buscando «Power BI Desktop», o desde powerbi.microsoft.com/desktop. Solo se necesita al final, en la fase del dashboard.

Verificación obligatoria. Abra PowerShell (tecla Windows, escriba «powershell», Enter) y escriba estos cuatro comandos, uno por uno. Cada uno debe responder con un número de versión; si alguno dice «no se reconoce», esa herramienta no quedó instalada o falta el PATH:

```
node -v          # debe responder algo como v20.11.0
python --version # debe responder Python 3.12.x
git --version    # debe responder git version 2.4x
docker --version # debe responder Docker version 27.x
```

4.3 Conseguir la llave de Gemini en Google AI Studio (gratis, 3 minutos)

1. Entre a aistudio.google.com e inicie sesión con la cuenta de Google del equipo (sirve cualquier cuenta de Gmail).
2. Acepte los términos de uso que aparecen la primera vez.
3. En el menú de la izquierda, pulse «Get API key» (Obtener clave de API).
4. Pulse «Create API key» y elija «Create API key in new project».
5. Copie la clave que aparece — es una cadena larga que empieza por Alza. Guárdela en un bloc de notas: la va a pegar en el archivo .env del paso 14.5.
6. No comparta esa clave ni la suba a GitHub. Por eso el archivo .env está en el .gitignore (sección 14.4).

¿Cuánto cuesta? El plan gratuito de AI Studio permite una cantidad generosa de consultas por día, más que suficiente para desarrollar y demostrar el prototipo durante el hackathon. No pide tarjeta de crédito.

4.4 Crear la carpeta del proyecto

Abra PowerShell en la carpeta donde quiera guardar el proyecto (por ejemplo, Documentos) y pegue este bloque completo. Crea todas las carpetas y los archivos vacíos de una sola vez:

```
mkdir cuentavoz; cd cuentavoz
mkdir backend\agente, backend\servicios, frontend\src\vistas, data, reportes
ni backend\main.py, backend\bd.py, backend\modelos.py -ItemType File
ni backend\seguridad.py, backend\reportes.py -ItemType File
ni backend\requirements.txt, backend\Dockerfile -ItemType File
ni backend\agente\cerebro.py, backend\agente\orquestador.py -ItemType File
ni backend\servicios\conciliacion.py -ItemType File
ni backend\servicios\validacion.py -ItemType File
ni frontend\src\App.jsx, frontend\src\api.js -ItemType File
ni frontend\src\index.css, frontend\Dockerfile -ItemType File
ni data\cargar_excel.py -ItemType File
ni .env, .gitignore, docker-compose.yml -ItemType File
code . # abre el proyecto en Visual Studio Code
```

Copie también el archivo BODEGAS_Y_STOCK.xlsx que entregó Colsubsidio dentro de la carpeta data. Así queda la estructura completa del proyecto — cada archivo con su función y, en verde, el componente de cada vista:

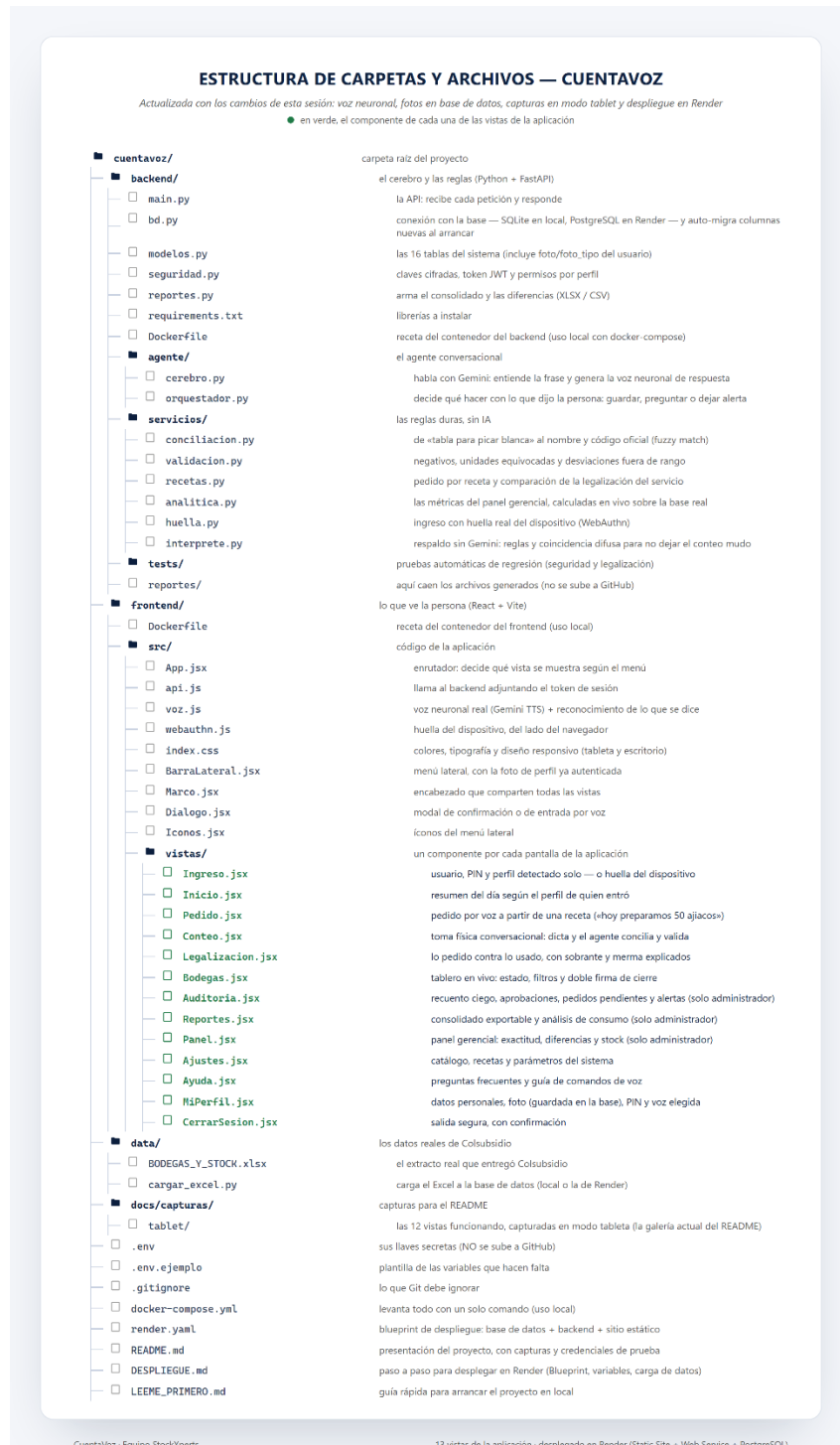


Figura 4. Estructura de carpetas y archivos del proyecto: en verde, el componente de cada una de las veintinueve vistas del capítulo 5.

4.5 Llenar los archivos y configurar las llaves

En Visual Studio Code, abra cada archivo de la lista anterior y pegue el código de la sección 12, en ese mismo orden. Son quince archivos; el manual los trae completos y listos para copiar. El único que hay que personalizar es `.env`, donde va la llave de Gemini que obtuvo en 14.3:

```
GOOGLE_API_KEY=Aiza...pegue_aqui_su_llave
MODELO=gemini-3.5-flash
DB_URL=postgresql://cuentavoz:cambieme@db:5432/cuentavoz
IDIOMA_VOZ=es-CO
UMBRAL_ANOMALIA=0.5
SECRETO_JWT=escriba_aqui_una_frase_larga_y_dificil
MINUTOS_TOKEN=480
ORIGEN_PERMITIDO=http://localhost:5173
```

Y en `.gitignore`, para que las llaves nunca salgan de su computador:

```
.env
.venv/
__pycache__/
node_modules/
reportes/
```

Extensiones recomendadas en VS Code (ícono de bloques en la barra izquierda, o Ctrl+Shift+X): Python y Pylance de Microsoft, Docker, ESLint y Prettier. Opcional pero muy útil: Thunder Client, para probar la API sin salir del editor.

4.6 Levantar el proyecto por primera vez

Con Docker Desktop abierto, en la terminal de VS Code (menú Terminal → Nueva terminal) escriba:

```
docker compose up -build
```

La primera vez tarda entre 3 y 5 minutos porque descarga PostgreSQL y las librerías. Cuando termine, verifique estas dos direcciones en el navegador:

- <http://localhost:8000/docs> — debe aparecer la documentación automática de la API con todos los endpoints. Si la ve, el backend y la base de datos están funcionando.
- <http://localhost:5173> — debe aparecer la pantalla de ingreso de CuentaVoz (vista 5.1).

Ahora cargue los datos reales de Colsubsidio. Abra una segunda terminal (el símbolo + en el panel de terminal, para no detener la primera) y escriba:

```
cd data
python cargar_excel.py
```

Debe imprimir el conteo de lo cargado: 48 bodegas, 936 artículos y 1.421 registros de stock. Si ese número aparece, el prototipo ya tiene el inventario real adentro.

4.7 Probar que el agente responde

Abra <http://localhost:5173> en Chrome o Edge (Firefox no soporta bien el reconocimiento de voz), ingrese con cualquier usuario y pruebe este guion:

Diga esto	Debe pasar esto	Si falla, revise
«Iniciar conteo en almacén de suministros»	El agente abre la bodega y dice cuántas referencias tiene	Que cargar_excel.py haya corrido bien
«Tres tablas para picar blancas»	Ofrece las dos opciones con su código	La llave de Gemini en el .env
«La FB»	Repite el nombre oficial y pregunta si confirma	El servicio de conciliación (12.7)
«Confirmo»	Guarda y anuncia el avance	La conexión con PostgreSQL
«Noventa cazuelas de dieciséis onzas»	Dispara la alerta de cantidad fuera de lo esperado	El umbral en el .env (0.5)

4.8 Conectar Power BI y publicar en GitHub

Power BI: abra Power BI Desktop, siga los seis pasos de conexión de la sección 13.1 usando servidor localhost:5432, base cuentavoz, usuario cuentavoz y la contraseña del docker-compose. Cree las medidas DAX de 13.3 y arme el lienzo de 13.4.

GitHub (es entregable del reto): en VS Code, pulse el ícono de Control de código fuente en la barra izquierda (el tercero), luego «Inicializar repositorio», escriba un mensaje como «estructura inicial», pulse «Confirmar» y después «Publicar en Branch». VS Code pedirá iniciar sesión en GitHub y creará el repositorio solo. Verifique en github.com que el archivo .env NO aparezca en la lista: si aparece, bórralo del repositorio y revise el .gitignore.

4.9 Problemas frecuentes y cómo resolverlos

Lo que ve en pantalla	Qué significa	Cómo se resuelve
«docker: command not found»	Docker Desktop no está instalado o no está abierto	Abra Docker Desktop y espere a que diga «Engine running»
«port is already allocated»	Otro programa está usando el puerto 8000 o el 5432	Cambie el puerto en docker-compose.yml (por ejemplo 8001:8000) o cierre el otro programa
«python no se reconoce»	Python quedó instalado sin agregarse al PATH	Reinstale Python marcando la casilla «Add python.exe to PATH»

Lo que ve en pantalla	Qué significa	Cómo se resuelve
El agente responde «no le entendí» siempre	La llave de Gemini está mal copiada o vencida	Genere una nueva en aistudio.google.com y péguela completa en el .env
El micrófono no hace nada	El navegador no tiene permiso o no es Chrome/Edge	Use Chrome o Edge y acepte el permiso de micrófono que pide el navegador
«relation does not exist»	Las tablas no se crearon antes de cargar el Excel	Levante primero con docker compose up y luego corra cargar_excel.py
La página queda en blanco	El frontend no encuentra la API	Revise que VITE_API_URL apunte a http://localhost:8000

Resumen del costo: cero pesos. Visual Studio Code, Node.js, Python, Git, Docker Desktop, PostgreSQL, Power BI Desktop y la llave de Gemini en Google AI Studio son gratuitos para este uso. Lo único que se necesita es un computador con Windows 10 u 11, 8 GB de memoria y conexión a internet.

5. Vistas de la aplicación: cómo la ve el usuario

Estas son las diez pantallas del prototipo, tal como las verá el personal de Colsubsidio. Cada vista se describe con sus elementos y con la conexión que usa por debajo, para que quien programe el frontend sepa exactamente qué construir y contra qué endpoint. El diseño busca tres cosas: botones grandes (se usa de pie, a veces con guantes), una sola acción principal por pantalla, y la voz siempre como camino más rápido — nunca como obligación.

Antes de entrar al menú: la puerta de la aplicación.

5.0a Cómo se conecta una vista con la siguiente

Una plataforma no es un catálogo de pantallas sueltas: es un recorrido. Este mapa muestra las veintinueve vistas agrupadas por la entrada de menú donde viven, y las flechas que las unen. Las azules son navegación por el menú; las doradas, transiciones que dispara el propio agente sin que nadie toque nada.

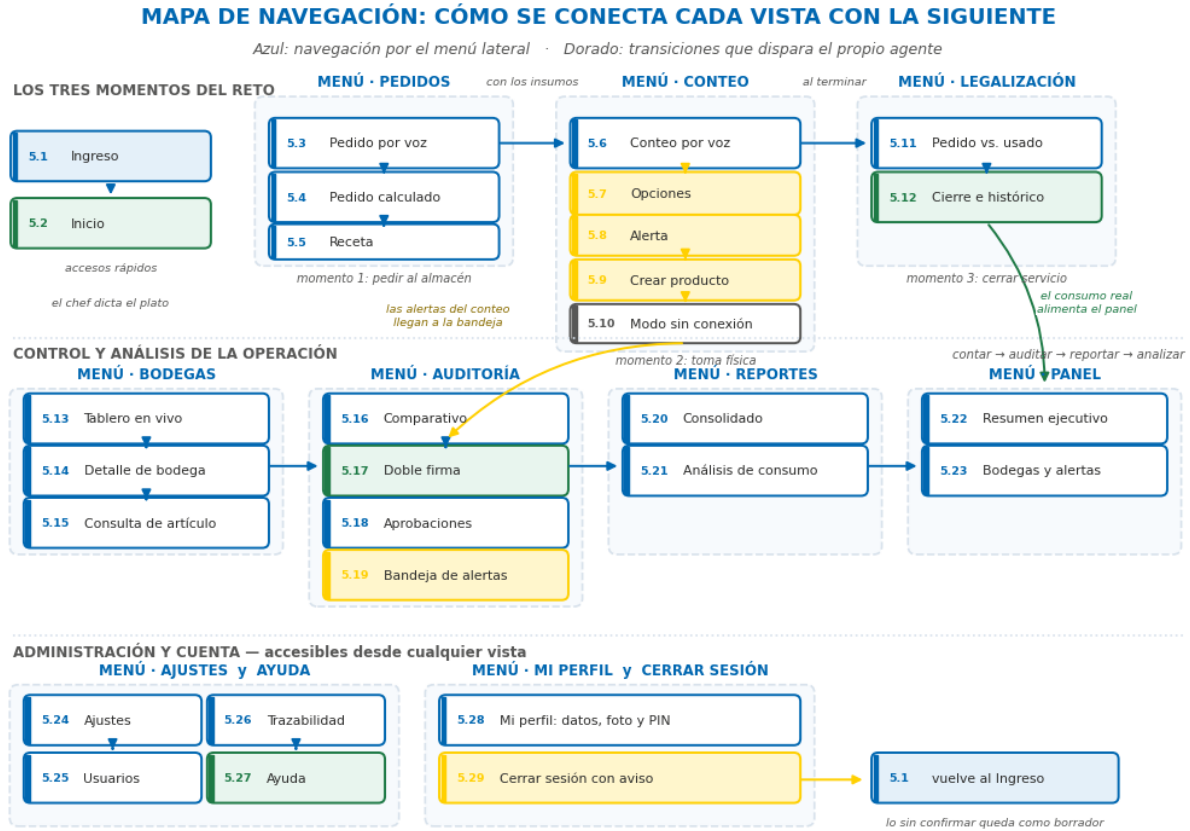


Figura 5. Mapa de navegación: los tres momentos del reto arriba, el control y el análisis en el medio, y la administración abajo.

Las transiciones que no nacen del menú son las que hacen que la aplicación se sienta fluida. Estas son todas:

Desde	Qué la dispara	Hacia
5.2 Inicio	Los cuatro accesos rápidos del día	5.3 Pedido · 5.13 Tablero · 5.16 Auditoría · 5.20 Reportes
5.3 Pedido por voz	El agente reconoce el plato y las porciones	5.4 Pedido calculado
5.4 Pedido calculado	La persona pregunta de dónde salió la cantidad	5.5 Receta consultada
5.6 Conteo por voz	El agente encuentra dos candidatos parecidos	5.7 Elegir entre opciones
5.6 Conteo por voz	La cantidad se desvía de lo esperado	5.8 Alerta de anomalía
5.6 Conteo por voz	El artículo no existe en el catálogo	5.9 Crear producto
5.6 Conteo por voz	Se pierde la conexión o el agente no responde	5.10 Modo sin conexión

Desde	Qué la dispara	Hacia
5.9 Crear producto	El registro queda pendiente de aprobación	5.18 Aprobaciones (del administrador)
5.8 Alerta de anomalía	La alerta se confirma y queda abierta	5.19 Bandeja de alertas
5.11 Legalización	Se confirma el ajuste del servicio	5.12 Cierre e histórico
5.12 Cierre e histórico	El consumo real alimenta el modelo	5.21 Análisis · 5.22 Panel
5.13 Tablero de bodegas	Se toca la tarjeta de una bodega	5.14 Detalle de esa bodega
5.17 Doble firma	La bodega queda cerrada	5.13 Tablero (se actualiza en vivo)
Cualquier vista	El botón «Pedir ayuda» o la entrada del menú	5.27 Centro de ayuda
5.29 Cerrar sesión	Se confirma la salida	5.1 Ingreso (lo pendiente queda como borrador)

Una regla de coherencia que se cumple en las veintinueve vistas: la entrada del menú que aparece resaltada es siempre aquella donde vive la pantalla. Si la persona está creando un producto, el menú resalta Conteo, porque crear nace del conteo; si está cerrando la sesión, resalta Cerrar sesión. Nunca hay duda de dónde se está.

5.00 Identidad visual: marca Colsubsidio aplicada

Las pantallas cumplen el manual de marca de Colsubsidio. La aplicación usa la paleta institucional y lleva el logotipo en dos lugares: en color, arriba a la derecha de la barra superior, y en blanco al pie de la barra lateral, sobre el fondo azul profundo. CuentaVoz conserva su propio logotipo a la izquierda, en la disposición de co-marca habitual: la solución se identifica, la institución la respalda.

Color oficial	HEX	Pantone	Dónde se usa en la aplicación
Azul Colsubsidio	#0067B1	Pantone 2196 C	Color principal: barra superior, títulos, botones de acción y series de datos
Amarillo Colsubsidio	#FFD000	Pantone 109 C	Acento: línea de marca bajo la barra, alertas de atención y la serie destacada de las gráficas
Grafito	#575756	Cool Gray 11 C	Textos secundarios, etiquetas y notas al pie
Blanco	#FFFFFF	—	Fondo de tarjetas y texto sobre azul

Nota de aplicación: el amarillo institucional se reserva para llamar la atención, nunca para el fondo de bloques grandes de texto, y las alertas usan ese amarillo en lugar de rojo: el sistema pregunta, no regaña. La tipografía es Calibri en toda la aplicación y en este manual.

5.0 Las vistas organizadas por el menú

El reto plantea tres momentos manuales: pedir insumos al almacén, contar el inventario físico y legalizar el servicio. Este capítulo recorre los tres, y las vistas de administración que una plataforma empresarial necesita alrededor de ellos, en el mismo orden del menú lateral.

Entrada del menú	Momento del reto	Vistas de este capítulo
(antes de entrar)	—	5.1 Ingreso con usuario, PIN y perfil
Inicio	—	5.2 Resumen del día, accesos rápidos y actividad reciente
Pedidos	1. Pedido al almacén	5.3 Pedido por voz · 5.4 Pedido calculado por receta · 5.5 Consulta de la receta
Conteo	2. Toma física	5.6 Conteo por voz · 5.7 Elegir entre opciones · 5.8 Alerta de anomalía · 5.9 Crear producto · 5.10 Modo sin conexión
Legalización	3. Legalización	5.11 Lo pedido contra lo usado · 5.12 Cierre y actualización del histórico
Bodegas	2. Toma física	5.13 Tablero en vivo · 5.14 Detalle de una bodega · 5.15 Consulta de artículo
Auditoría	2. Toma física	5.16 Comparativo de diferencias · 5.17 Cierre con doble firma · 5.18 Aprobaciones pendientes · 5.19 Bandeja de alertas
Reportes	los tres	5.20 Consolidado para My Inventory · 5.21 Análisis de consumo (lo pedido vs. lo usado)
Panel	los tres	5.22 Resumen ejecutivo · 5.23 Bodegas y alertas (informe de Power BI insertado)
Ajustes	—	5.24 Parámetros del sistema · 5.25 Gestión de usuarios · 5.26 Registro de trazabilidad
Ayuda	—	5.27 Preguntas frecuentes, soporte, estado del sistema y comandos de voz
Mi perfil	—	5.28 Datos personales, foto, PIN y preferencias de voz
Cerrar sesión	—	5.29 Confirmación que avisa del trabajo sin guardar

5.1 Ingreso por perfil

Figura 6. Vista de ingreso: dos perfiles, un toque.

- **Qué tiene:** los dos perfiles del reto como botones grandes (auxiliar de inventarios y administrador de bodega), el campo de usuario y el ingreso con el PIN o la huella que la tableta ya trae.
- **Qué conecta:** crea la identidad que viaja en cada turno; el perfil decide qué pantallas y qué herramientas están disponibles (el auxiliar no puede cerrar bodegas; el auditor no ve el conteo 1 durante su recuento).

Lo primero que ve cualquier persona al entrar.

MENÚ · INICIO *lo primero que ve la persona al entrar*

5.2 Inicio: el punto de partida del día

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Inicio · Lunes, 27 de julio de 2026

TOMA EN CURSO

Colsubsidio

Buenos días, Diana. Esto es lo que hay para hoy.

Bodegas asignadas a usted

0

según su perfil

Referencias contadas hoy

98

en sus sesiones de conteo

Alertas por revisar

18

en toda la operación

Su exactitud del mes

89.9 %

promedio de bodegas cerradas

¿Qué desea hacer?

Iniciar un conteo

dicte y CuentaVoz registra

Ver el tablero

estado de las bodegas

Continuar auditoría

recuento ciego y aprobaciones

Generar reporte

consolidado del día

Actividad reciente

● 22:45	Luis firmó el conteo de Autoservicios Cascada	firma
● 22:45	Luis abrió Autoservicios Cascada	apertura
● 22:45	Luis firmó el conteo de Autoservicio Mirador	firma
● 22:45	Luis abrió Autoservicio Mirador	apertura
● 22:45	Luis abrió Almacen General	apertura
● 22:45	Luis abrió Aire Libre Suministros Piscilago	apertura
● 22:45	Luis abrió Administracion Suministros Piscilago	apertura
● 22:17	Almacen Suministros quedó cerrada con doble firma	cierre

Figura 7. Vista de inicio: el resumen del día y los cuatro accesos rápidos a lo que la persona hace de verdad.

- **Qué tiene:** el saludo con la fecha, cuatro indicadores personales (bodegas asignadas, referencias contadas hoy, alertas por revisar y la exactitud del mes), cuatro accesos directos con su ícono y la actividad reciente del equipo con hora y responsable.
- **Por qué existe:** el menú lateral tenía la sección Inicio pero ninguna pantalla detrás. Esta vista responde la pregunta con la que llega el auxiliar cada mañana: ¿qué me toca hoy y cómo voy?

Las cinco pantallas del auxiliar mientras cuenta en la bodega.

MENÚ · PEDIDOS *el primero de los tres momentos manuales del reto: pedir insumos al almacén*

5.3 Pedido por voz: del plato a los insumos

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Pedidos · Cocina Piscilago

SERVICIO ALMUERZO

Colsubsidio

Lunes · almuerzo

Cocina Piscilago

Pedido sin enviar

Se descuenta de: ALMACEN AYB

Lo que dijo el chef

Aún no ha dictado nada: pulse el micrófono y diga, por ejemplo, «hoy preparamos cincuenta ajiacos».

Lo que entendió CuentaVoz

Todavía no hay ningún plato dictado en esta conversación.



Mantenga presionado y hable

También sirve: «pedir insumos para 30 sancochos»

Receta + stock = pedido

 Platos con receta registrada

Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.

AJIACO SANTA FERREÑO

ARROZ CON POLLO

FRIJOL CON GARRA

LENTEJAS

SANCOCHO DE GALLINA

CuentaVoz responde

Diga el plato y las porciones: «hoy preparamos cincuenta ajiacos».

Calcular el pedido

Corregir cantidad

Ver la receta

Figura 8. Vista de pedido: el chef dice el plato y las porciones, y el agente traduce eso a insumos.

- **El caso que plantea el reto:** «hoy preparamos cincuenta ajiacos». La persona no dicta ingredientes ni cantidades: dicta el plato. El agente reconoce la preparación, las porciones y la receta que aplica, y lo confirma en voz antes de calcular nada.
- **Qué tiene:** lo que dijo el chef tal cual, lo que entendió CuentaVoz en cuatro campos (preparación, porciones, receta aplicada y momento), el micrófono con ejemplos de otras frases válidas, y la respuesta hablada que anuncia el siguiente paso.
- **Qué conecta:** la intención pedir del JSON de la sección 6.2 activa la herramienta calcular_pedido; el agente no inventa cantidades: las saca de la receta del catálogo de Colsubsidio.

5.4 El pedido calculado: solo lo que de verdad falta

CuentaVoz
Tu voz cuenta

Pedidos · Insumos calculados para 50 ajiacos 3 POR PEDIR Colsubsidio

Inicio
Pedidos
Conteo
Legalización
Bodegas
Auditoría
Reportes
Panel
Ajustes
Ayuda
Mi perfil
Cerrar sesión

Luis
Auxiliar de inventarios
Colsubsidio
Solución para Colsubsidio · v1.0.0

Insumos de la receta
8
Ingredientes estándar

Ya disponibles en bodega
5
no se piden

Hay que pedir al almacén
3
faltante calculado

La cantidad necesaria sale de la receta por porción; el faltante es lo necesario menos lo que ya hay en la bodega.

Insumo (nombre oficial)	Unidad	Necesario	Hay en bodega	Falta: pedir
POLLO PECHUGA SIN HUESO	Kilogram	12,5	4,0	8,5
MAZORCA TIERNA	Unidad	50	20	30
GUASCAS DESHIDRATADAS	Kilogram	0,5	0,1	0,4
PAPA CRIOLLA	Kilogram	10,0	10,0	—
CREMA DE LECHE	Liter	5,0	6,0	—
ALCAPARRAS	Kilogram	1,0	2,4	—

Las tres últimas filas no se piden: la bodega ya tiene suficiente.

Enviar pedido al almacén **Ajustar por voz** **Imprimir el pedido**

Figura 9. Vista del pedido calculado: necesario según receta, lo que ya hay en bodega y el faltante real.

- **La cuenta que hace el agente:** multiplica la cantidad por porción de la receta por las porciones pedidas, consulta el stock de esa bodega y resta. Lo que da cero o negativo no se pide: por eso en el ejemplo solo salen tres de los ocho ingredientes.
- **Por qué importa:** es exactamente el resultado que el reto pone como ejemplo de buena solución. Sin este cálculo, el pedido se hace «a ojo» y de ahí nacen los sobrantes que después nadie sabe explicar.
- **Qué conecta:** POST /api/pedidos/calculador devuelve la tabla; POST /api/pedidos/Enviar la deja registrada para el almacén, con los nombres y códigos oficiales del catálogo.

5.5 La receta que sustenta el cálculo

CuentaVoz
Tu voz cuenta

Pedidos · Receta estándar SOLO CONSULTA Colsubsidio

AJIACO SANTA FERREÑO

Rendimiento: 1 porción 8 ingredientes Catálogo de Colsubsidio

Ingrediente	Unidad	Por porción	Para 50 porciones
POLLO PECHUGA SIN HUESO	Kílogram	0,250	12,5
PAPA CRIOLLA	Kílogram	0,200	10,0
PAPA SABANERA	Kílogram	0,180	9,0
MAZORCA TIERNA	Unidad	1	50
GUASCAS DESHIDRATADAS	Kílogram	0,010	0,5
CREMA DE LECHE	Liter	0,100	5,0
ALCAPARRAS	Kílogram	0,020	1,0
ARROZ BLANCO	Kílogram	0,080	4,0

El prototipo consulta las recetas, no las gestiona

Crear o modificar recetas y menús está fuera del alcance del reto: eso lo resuelve el sistema de Colsubsidio.
CuentaVoz las lee para calcular pedidos y, en la legalización, para comparar lo previsto contra lo consumido.

[Usar para un pedido](#)

Luis
Auxiliar de inventarios
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 10. Vista de receta: el desglose por porción del que sale todo el cálculo del pedido.

- **Qué tiene:** la receta estándar con su rendimiento, la cantidad por porción de cada ingrediente y la proyección a las porciones pedidas. Es la pantalla que responde «¿de dónde salió ese 12,5?».
- **Límite de alcance, declarado en la propia vista:** CuentaVoz consulta las recetas, no las gestiona. Crear o modificar recetas y menús está fuera del alcance del reto y lo resuelve el sistema de Colsubsidio.

MENÚ · CONTEO *el trabajo diario del auxiliar de inventarios*

5.6 Conteo por voz (pantalla principal del auxiliar)

CuentaVoz
Tu voz cuenta

[Inicio](#)
[Pedidos](#)
[Conteo](#)
[Legalización](#)
[Bodegas](#)
[Auditoría](#)
[Reportes](#)
[Panel](#)
[Ajustes](#)
[Ayuda](#)
[Mi perfil](#)
[Cerrar sesión](#)

Luis
Auxiliar de inventarios

Conteo por voz · Almacén de Suministros

EN CONTEO

Avance: 2 de 312
Alertas: 0
Sesión bloqueada

CuentaVoz responde

«Tabla acrílica picar blanco 50x38 FB: tres unidades. ¿Confirma?»

Últimos registros confirmados

✓ TABLA ACRILICA PICAR BLANCO 50X38CM FB	3 Unidad
✓ CAZUELA 16 ONZAS	9 Unidad

Mantenga presionado y hable
Reconocimiento en español (Colombia)
Tiempo de respuesta: 1,2 s

Confirmar
Corregir
Teclado
Pedir ayuda

La voz es el camino rápido; el teclado siempre queda disponible como respaldo.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Conteo por voz

SIN BODEGA

Colsubsidio

¿Qué bodega va a contar?

escriba para buscar...

Abrir por nombre exacto

ADMINISTRACION SUMINISTROS PISCILAGO EN CONTEO 0 referencias	AIRE LIBRE SUMINISTROS PISCILAGO EN CONTEO 0 referencias	ALMACEN AYB CERRADA 270 referencias
ALMACEN GENERAL EN CONTEO 0 referencias	ALMACEN SUMINISTROS CERRADA 296 referencias	AUTOSERVICIO MIRADOR EN AUDITORÍA 0 referencias
AUTOSERVICIOS CASCADA EN AUDITORÍA 0 referencias	AUTOSERVICIOS LAS FUENTES PENDIENTE 0 referencias	BODEGA DE RECREACION SUMINISTROS PENDIENTE 0 referencias
CAF. VELAS SUMINISTROS PENDIENTE 0 referencias	CAF.VELAS PENDIENTE 0 referencias	CAFETERIA ACUARIO SUMINISTROS PENDIENTE 0 referencias
COCINA MIRADOR SUMINISTROS PENDIENTE 0 referencias	COCINA PRINCIPAL SUMINISTROS PENDIENTE 0 referencias	ENFERMERIA SUMINISTROS PENDIENTE 0 referencias
KIOSCO 1 PISCILAGO PENDIENTE 0 referencias	KIOSCO 2 PISCILAGO PENDIENTE 0 referencias	KIOSCO 2 SUMINISTROS PENDIENTE 0 referencias
KIOSCO 3 PISCILAGO PENDIENTE 0 referencias	KIOSCO 3 SUMINISTROS PENDIENTE 0 referencias	KIOSCO BOSQUE SUMINISTROS PENDIENTE 0 referencias
KIOSCO BOSQUES PENDIENTE 0 referencias	KIOSCO CASCADA SUMINISTROS PENDIENTE 0 referencias	KIOSCO PAQUEADERO SUMINISTROS PISCILAGO PENDIENTE 0 referencias

Mostrando 24 de 53 — escriba para filtrar.

Figura 11. Vista de conteo: la conversación a la izquierda, el micrófono a la derecha, el avance siempre visible.

- **Qué tiene:** la bodega activa y el avance arriba (2 de 312), la burbuja con lo que CuentaVoz entendió y pregunta, la lista de últimos registros con su chulo de confirmación, el botón grande de micrófono («mantenga presionado y hable») y los botones de respaldo: Confirmar, Corregir, Teclado y Ayuda.
- **Qué conecta:** cada frase va por POST /api/agente/turno (sección 12.13); la respuesta vuelve como texto que la tableta lee en voz alta; el chip de avance se refresca con la respuesta de cada turno.

5.7 Desambiguación: elegir entre opciones



Figura 12. Vista de opciones: cuando hay dos candidatas, se eligen por voz o con un toque.

- **Qué tiene:** las candidatas como tarjetas grandes con el nombre oficial del catálogo, el código y lo que el sistema registra en esa bodega; y la salida «ninguna de las dos» que lleva al flujo de crear producto (pendiente del auditor).
- **Qué conecta:** las tarjetas vienen del campo opciones del JSON del turno (los candidatos de buscar_articulo, sección 12.7); tocar una tarjeta equivale a decir su nombre — ambas rutas llaman al mismo endpoint.

5.8 Alerta de anomalía: preguntar antes de guardar

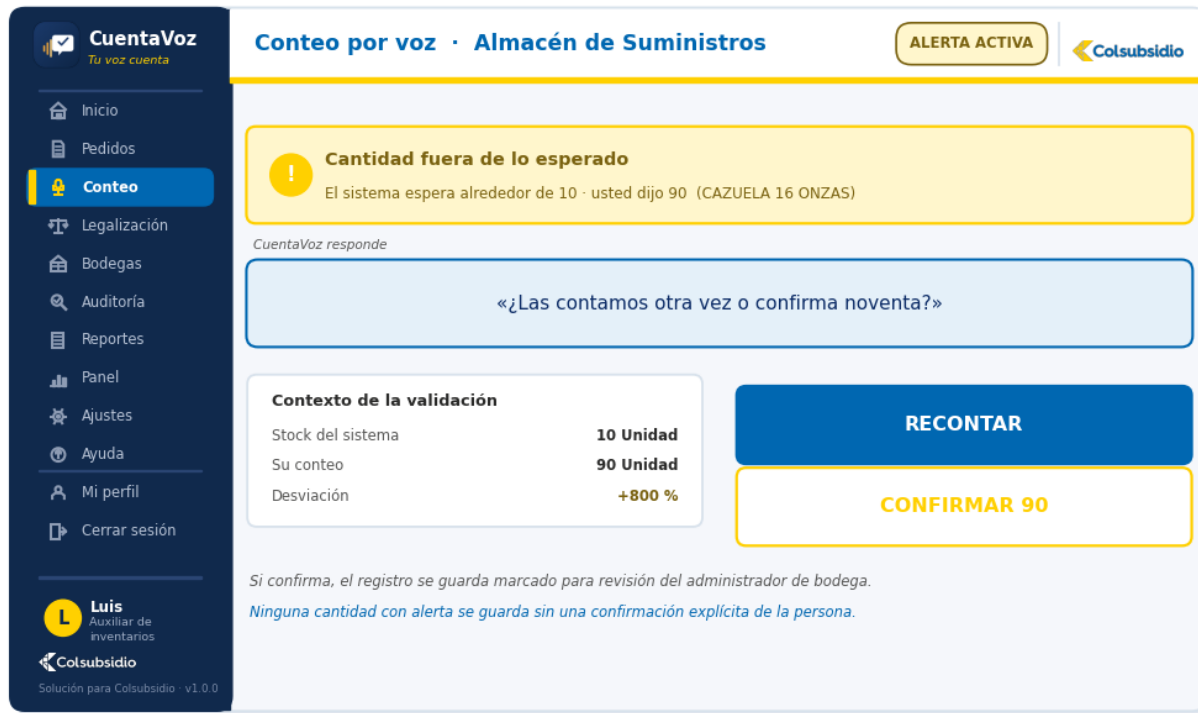


Figura 13. Vista de alerta: el famoso «9 vs. 90» detenido en el origen.

- **Qué tiene:** el banner amarillo con la razón exacta («el sistema espera alrededor de 10 · usted dijo 90»), la pregunta hablada y dos salidas claras: RECONTAR o CONFIRMAR con marca para el auditor. Nada se guarda solo.
- **Qué conecta:** la alerta nace en validar_conteo (sección 12.8) comparando contra StockSistema — el extracto real de My Inventory —; si la persona confirma, el registro se guarda con su Alerta asociada para que la auditoría la revise.

5.9 Crear producto: pendiente de aprobación

CuentaVoz
Tu voz cuenta

Crear producto · Almacén de Suministros **PENDIENTE DE APROBACIÓN** **Colsubsidio**

CuentaVoz responde

«No encontré “ají casero piscilago” en el catálogo.
Lo creamos y queda pendiente del administrador.»

Nombre tal como lo dictó: **AJÍ CASERO PISCILAGO**

Estado del registro: **PENDIENTE DE APROBACIÓN**

Unidad de medida: **Unidad** **Kilogram** **Liter** **Portion**

Cantidad inicial contada: **36** en Almacén de Suministros

No entra al catálogo oficial hasta la firma del administrador de bodega.

Crear pendiente **Cancelar**

El conteo continúa sin interrupción: la aprobación ocurre en paralelo.

Luis
Auxiliar de inventarios
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 14. Vista de creación: el formulario llega precargado con lo dictado y nace en estado pendiente.

- **Qué tiene:** el nombre tal como se dictó, la unidad por chips grandes, la cantidad inicial y el sello visible «PENDIENTE DE APROBACIÓN»: nadie mete un artículo al catálogo oficial por su cuenta.
- **Qué conecta:** crear_articulo (12.10) guarda el registro con estado pendiente_aprobacion; aparece en la bandeja del administrador, y solo su firma lo vuelve oficial.

5.10 Modo formulario: contingencia sin voz ni internet

CuentaVoz
Tu voz cuenta

Conteo · Modo formulario (sin conexión) **SIN CONEXIÓN** **Colsubsidio**

El asistente no responde en este momento
Modo formulario activo: lo que registre se guarda en el equipo y se sincroniza al volver la señal.

Artículo
tabla acril...
TABLA ACRILICA PICAR BLANCO 50X38CM FB 97503004
El catálogo local sigue sugiriendo nombres oficiales sin internet.

Cantidad
- 3 + Unidad

Guardados en este equipo

CAZUELA 16 ONZAS	9 Unidad
ARROZ	120 Kilogram
ACEITE VEGETAL	24 Liter

3 registros por sincronizar

Guardar **Reintentar la voz**

Al volver la señal, la cola se envía sola a la API y las validaciones se aplican igual que en línea.

Luis
Auxiliar de inventarios
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 15. Vista de contingencia: si el asistente no responde, el conteo no se detiene.

- **Qué tiene:** el aviso claro de «sin conexión», el buscador que sigue sugiriendo nombres oficiales del catálogo local, la cantidad con botones grandes – / + y la cola de registros «por sincronizar».
- **Qué conecta:** los mismos endpoints de siempre: los registros esperan en el equipo y se sincronizan al volver la señal — es la prueba 5 de la sección 14.6 hecha pantalla, y el respaldo si Gemini llegara a caerse en plena demo.

El estado de toda la operación, en vivo.

MENÚ · LEGALIZACIÓN *el tercer momento del reto: cerrar el servicio comparando lo pedido con lo usado*

5.11 Legalización del servicio: lo pedido contra lo usado

Inicio
Pedidos
Conteo
Legalización
Bodegas
Auditoría
Reportes
Panel

Legalización · ajiaco santafereño
SERVICIO CERRADO
Colsubsidio

80 porciones servidas
1 sobrantes
2 mermas por revisar

Al cerrar el servicio se compara lo pedido con lo realmente usado: cada diferencia queda explicada como sobrante o como merma.

Insumo	Pedido	Usado	Diferencia	Qué significa
POLLO ENTERO	20	18	-2	Sobrante: vuelve a bodega
PAPA CRIOLLA	16	17	+1	Merma: revisar con el chef
ARROZ	6.4	6.4	0	Cuadro exacto
SAL	0.4	0.5	+0.1	Merma: revisar con el chef

CuentaVoz responde

Sobró 2 de pollo entero: lo devuelvo a la bodega. Hubo merma de 1 en papa criolla, 0,1 en sal.
¿Confirmo la legalización?

Confirmar legalización

Explicar la merma

Ajustar por voz

Figura 16. Vista de legalización: cada diferencia entre lo pedido y lo usado queda explicada como sobrante o como merma.

- **Qué resuelve:** al cerrar el servicio, la cocina devuelve lo que no usó y reporta lo que se perdió. Hoy eso se anota en papel y se digita después — el mismo problema del conteo, en otro momento del día.
- **Qué tiene:** la tabla de comparación insumo por insumo con la diferencia resaltada y su lectura en palabras: «sobrante que vuelve a bodega», «merma para revisar con el chef» o «cuadró exacto». Y la pregunta hablada que resume todo antes de confirmar.
- **La diferencia entre sobrante y merma:** el sobrante vuelve a la bodega y suma al stock; la merma no vuelve y queda registrada con responsable y hora. Confundirlas es lo que descuadra el inventario del mes siguiente.

5.12 Cierre de la legalización y actualización del histórico



Figura 17. Vista de cierre: qué pasa con lo devuelto, con la merma y con el consumo real.

- **Las tres salidas del cierre:** lo devuelto vuelve a bodega, la merma queda registrada con su nota, y el consumo real alimenta el histórico.
- **El histórico es la pieza que cierra la meta del reto:** si el ajiaco gasta siempre más papa criolla de la que dice la receta, el sistema lo detecta comparando servicio tras servicio. Ahí es donde la solución deja de ser un capturador de datos y se vuelve el sistema integral que valida con recetas e histórico.
- **Qué conecta:** POST /api/legalizacion/confirmar registra el ajuste, devuelve el sobrante al stock de la bodega y deja el archivo listo para cargarse a My Inventory con los nombres oficiales.

MENÚ · BODEGAS la vista de control del administrador

5.13 Tablero de bodegas en vivo

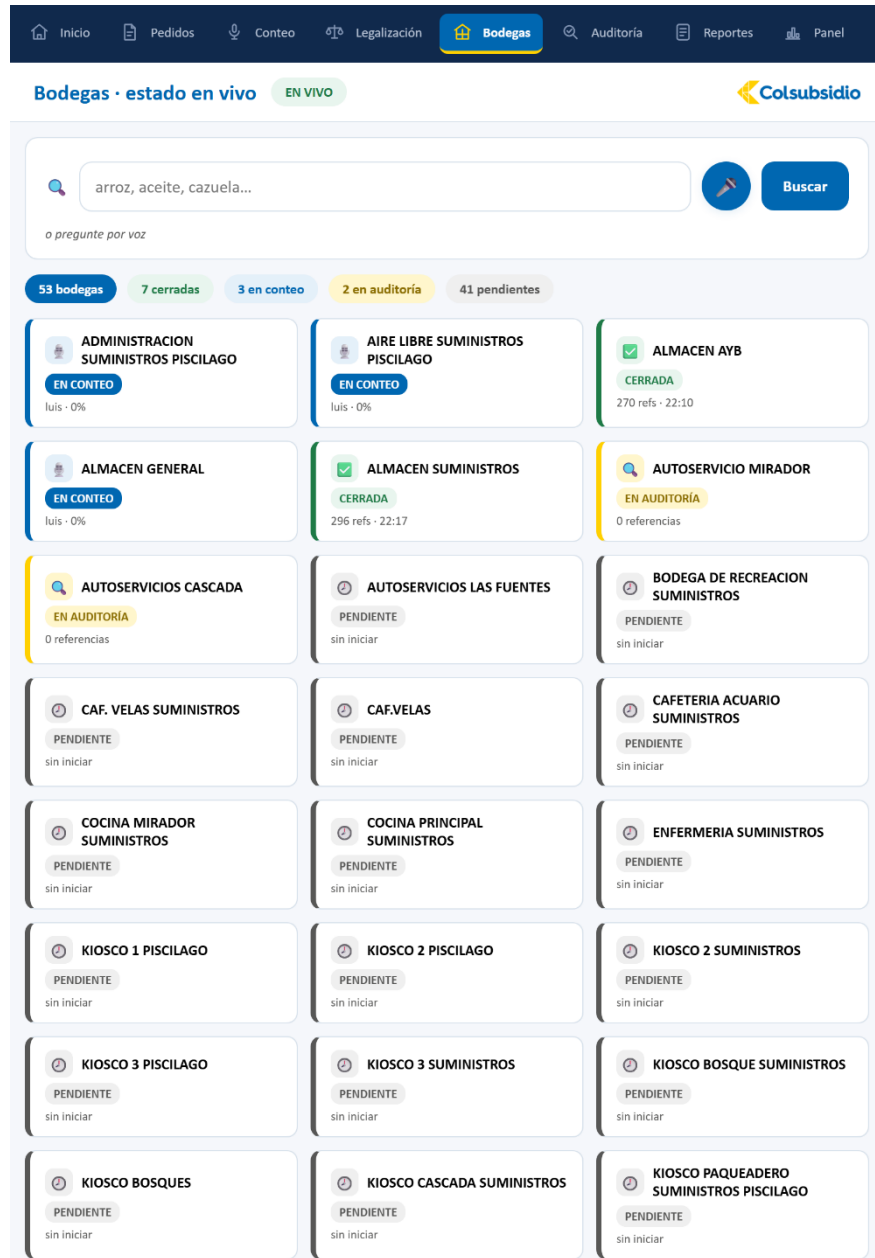


Figura 18. Vista de tablero: las 48 bodegas de un vistazo, cada una con su estado y su responsable.

- **Qué tiene:** una tarjeta por bodega con su estado por color (verde cerrada, azul en conteo, amarillo en auditoría, gris pendiente) y quién está en ella; los totales arriba; tocar una tarjeta abre el detalle.
- **Qué conecta:** el WebSocket /api/bodegas/estado (sección 12.12): cuando alguien abre o cierra una bodega, todas las tabletas se actualizan al instante. Es la vista que demuestra los conteos simultáneos sin choques.

Lo que hace el administrador para dar por buena una bodega.

5.14 Detalle de una bodega

CuentaVoz
Tu voz cuenta

Bodegas · Almacén de Suministros CERRADA Colsubsidio

312 referencias Cerrada 10:12 Luis y Stephanie 2 alertas resueltas

Exactitud de esta bodega: **99,4 %**
2 diferencias de 312

Duración del conteo: **38 min**
frente a 95 min en papel

Valor del inventario: **\$ 48,2 M**
al corte de hoy

Última toma anterior: **30 jun**
exactitud 96,1 %

Referencias con diferencia

Artículo	Contado	Sistema	Dif.
TABLA ACRILICA FB	3	5	-2
CAZUELA 16 ONZAS	9	10	-1

Las otras 310 referencias cuadraron exactas.
Las dos diferencias las aceptó la administradora.

Línea de tiempo de la toma

- 07:42 Luis abre la bodega
- 08:20 Alerta: 90 cazuelas → corregido a 9
- 09:35 Luis cierra el conteo 1
- 09:50 Stephanie inicia el recuento ciego
- 10:12 Cierre con doble firma

Reabrir la bodega Descargar detalle Ver en el panel gerencial

Reabrir una bodega cerrada exige justificación escrita y queda en el registro de trazabilidad (vista 5.26).

Diana
Administradora
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 19. Vista de detalle: todo lo que pasó en una bodega, desde que se abrió hasta la doble firma.

- **Cierra una promesa del tablero:** la vista 5.13 invita a tocar una tarjeta para ver el detalle. Esta es esa pantalla, y es la que abre el administrador cuando algo no cuadra.
- **Qué tiene:** la exactitud de esa bodega, la duración del conteo frente al método en papel, el valor del inventario y la comparación con la toma anterior; la tabla de las referencias con diferencia; y la línea de tiempo minuto a minuto, desde la apertura hasta la firma.
- **La línea de tiempo es la pieza de auditoría:** muestra que la alerta del 9 contra 90 se disparó a las 08:20 y se corrigió en el momento, no al final del mes. Reabrir una bodega cerrada exige justificación y queda en el registro de trazabilidad (vista 5.26).

5.15 Consulta de artículo: dónde está y cuánto hay

CuentaVoz
Tu voz cuenta

Bodegas · Consulta de artículo BÚSQUEDA GLOBAL Colsubsidio

🔍 arroz o pregunte por voz

CuentaVoz responde

«Arroz hay en cuatro bodegas: ciento ochenta y cinco kilos en total.»

Total en el sistema

185 kg

en 4 bodegas

Consumo del mes

142 kg

8 servicios

Cobertura estimada

9 días

al ritmo actual

Bodega	Cantidad	Última toma	Estado
ALMACEN AYB	120 kg	hoy 09:12	Cerrada
RESTAURANTE FUENTES AYB	45 kg	hoy 10:12	Cerrada
KIOSCO TAQUILLA AYB	12 kg	en conteo	En conteo
KIOSCO PISCIGIROS	8 kg	30 jun	Pendiente

[Generar reporte](#) [Ver movimientos](#) [Comparar con la receta](#)

Diana
Administradora

Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 20. Vista de consulta: la pregunta «¿cuánto arroz hay y en qué bodegas?» resuelta en una pantalla.

- **Qué resuelve:** la conversación 11.3 del manual documenta esta pregunta hablada, pero faltaba la pantalla que la muestra. Aquí está: se busca escribiendo o preguntando en voz alta, y el resultado sale en texto y en voz.
- **Qué tiene:** el total del artículo en el sistema, el consumo del mes, la cobertura estimada en días al ritmo actual, y la tabla de en qué bodegas está con el estado de cada una.
- **Por qué la cobertura importa:** no basta saber cuánto hay; el chef necesita saber para cuántos días alcanza. Ese dato sale de cruzar el stock con el consumo real que dejan las legalizaciones.

MENÚ · AUDITORÍA *el segundo conteo, las diferencias y las aprobaciones*

5.16 Auditoría: comparativo y doble firma

Inicio
Pedidos
Conteo
Legalización
Bodegas
Auditoría
Reportes
Panel

Auditoría · recuento ciego, aprobaciones y cierre
ADMINISTRADORA
Colsubsidio

Recuento ciego y cierre
Aprobaciones
Pedidos pendientes
Bandeja de alertas

Bodegas listas para recuento ciego o cierre

CERRADA	ALMACEN AYB	270 refs	Abrir
CERRADA	ALMACEN SUMINISTROS	296 refs	Abrir
EN AUDITORÍA	AUTOSERVICIO MIRADOR	0 refs	Abrir
EN AUDITORÍA	AUTOSERVICIOS CASCADA	0 refs	Abrir
CERRADA	KIOSCO PISCIGIROS AYB	56 refs	Abrir
CERRADA	KIOSCO TAQUILLA AYB	58 refs	Abrir
CERRADA	RESTAURANTE FUENTES AYB	344 refs	Abrir
CERRADA	RESTAURANTE FUENTES SUMIN	133 refs	Abrir
CERRADA	ZOOLOGICO SUMINISTROS	248 refs	Abrir

El recuento del administrador es a ciegas; no ve el conteo del auxiliar hasta comparar.

CuentaVoz
Tu voz cuenta

Auditoría · Restaurante Fuentes AyB RECONTEO CIEGO Colsubsidio

178 referencias 3 con diferencia 175 coinciden

Solo se muestran las referencias con diferencia. Su recuento fue a ciegas: el conteo 1 se revela al comparar.

Artículo	Conteo 1	Su conteo	Sistema	Diferencia	Acción
ARROZ (Kilogram)	118	120	120	-2	Aceptar
AJÍ CASERO PISCILAGO (Kg)	36	36	-482	+518	Revisar
ACEITE VEGETAL (Liter)	24	25	26	-1	Aceptar

Diagnóstico automático

El -482 del ají no viene de su conteo: el sistema ya traía saldo negativo (salida registrada sin la entrada).
Su conteo físico es el que vale. El reporte lo marca como negativo del sistema para corrección en My Inventory.

Resolver por voz **Aceptar todas** **Cerrar con doble firma**

Diana
Administradora

Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 21. Vista del auditor: solo las diferencias, con el conteo 1 revelado apenas termina el recuento ciego.

- **Qué tiene:** la tabla de solo diferencias (3 de 178, no las 178), la columna Dif. resaltada, la explicación de los negativos del sistema, y las dos acciones: resolver por voz o cerrar la bodega con doble firma digital.
- **Qué conecta:** POST /api/bodegas/cerrar arma el comparativo conteo 1 vs. conteo 2 vs. sistema; al firmar, la bodega pasa a «cerrada», el tablero se actualiza por WebSocket y sus datos entran al consolidado (sección 12.11).

5.17 Cierre con doble firma digital

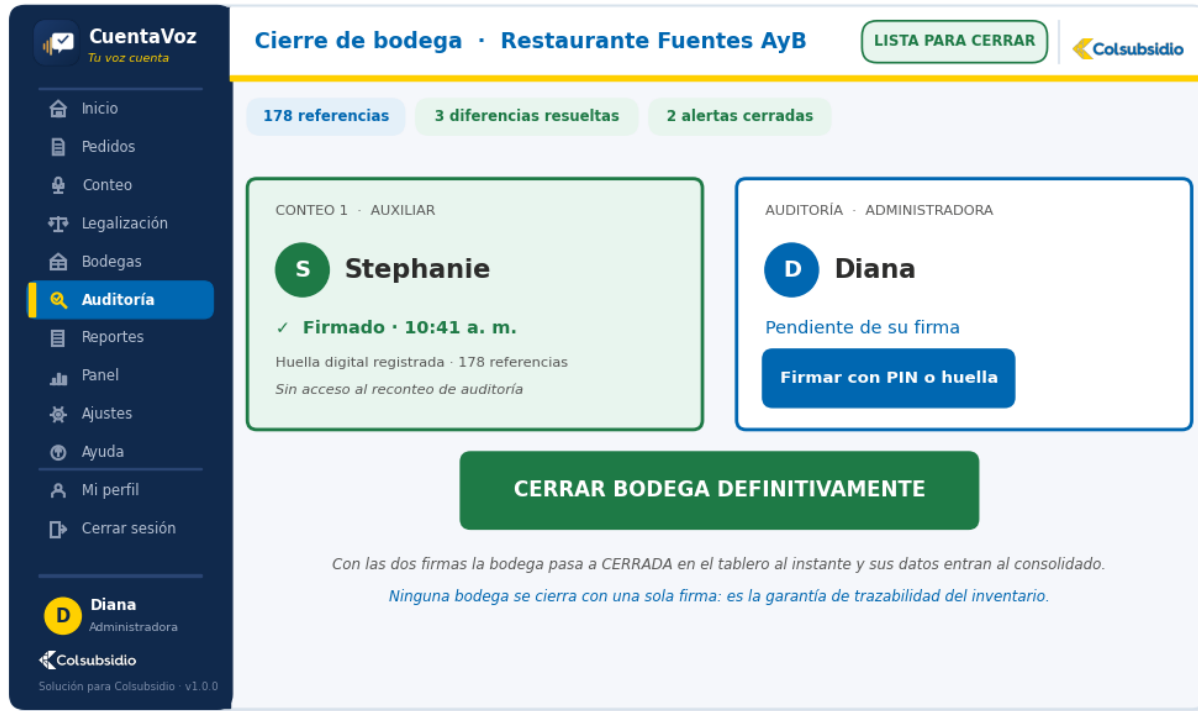


Figura 22. Vista de cierre: las dos firmas —auxiliar y administradora— antes de que la bodega quede cerrada.

- **Qué tiene:** el resumen de la bodega (178 referencias, diferencias y alertas resueltas), la firma del conteo 1 ya estampada con hora, y la tarjeta de la administradora con «Firmar ahora» por PIN o huella.
- **Qué conecta:** POST /api/bodegas/cerrar exige el perfil de administrador (requiere_perfil, 14.2); con las dos firmas la bodega pasa a «cerrada», el tablero 5.9 se entera por WebSocket y sus datos entran al consolidado 12.11.

5.18 Aprobaciones pendientes: cerrar el ciclo de lo creado

CuentaVoz
Tu voz cuenta

Aprobaciones pendientes 3 PENDIENTES Colsubsidio

Productos y bodegas creados durante la toma que esperan su firma para entrar al catálogo oficial.

- AJI CASERO PISCILAGO**
 Producto nuevo · Kilogram
 36 Kg contados en Almacén Suministros
 Luis · hoy 09:48
 [Rechazar] [Aprobar]
- SALSA DE PIÑA CASERA**
 Producto nuevo · Liter
 8 L contados en Rest. Fuentes AyB
 Stephanie · hoy 10:05
 [Rechazar] [Aprobar]
- BODEGA MÓVIL EVENTOS**
 Bodega nueva
 sin referencias asignadas todavía
 Valentina · hoy 10:22
 [Rechazar] [Aprobar]

Al aprobar, el producto entra al catálogo con su código definitivo y el conteo asociado deja de estar marcado.
 Mientras tanto el conteo nunca se detiene: la aprobación ocurre en paralelo al trabajo en bodega.

Diana
Administradora
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 23. Vista de aprobaciones: lo que se creó durante la toma esperando la firma del administrador.

- **Qué tiene:** una tarjeta por producto o bodega creada, con quién lo creó, a qué hora, qué cantidad se contó y los botones Aprobar o Rechazar.
- **Por qué importa:** la vista 5.16 deja los productos nuevos en estado pendiente, pero faltaba la pantalla donde ese estado se resuelve. Sin ella, el ciclo quedaba abierto: el auxiliar creaba y nadie aprobaba.

Los archivos oficiales que salen de la toma.

5.19 Bandeja de alertas

CuentaVoz
Tu voz cuenta

Auditoría · Bandeja de alertas 5 ABIERTAS Colsubsidio

5 abiertas 32 resueltas hoy Tiempo medio: 3 min

Desviación de cantidad CAZUELA 16 ONZAS · Almacén Suministros Sistema espera 10 · se dictó 90	Luis · 08:20	Ver	Resolver
Saldo negativo del sistema COLA Y POLA 330 · Restaurante Fuentes El sistema traía -7.434 antes del conteo	automática · 09:02	Ver	Resolver
Unidad de medida ARROZ · Kiosco Taquilla Se dictó en unidades y el catálogo usa kilos	Valentina · 09:48	Ver	Resolver
Artículo fuera del catálogo Ají CASERO PISCILAGO · Almacén Creado durante la toma, pendiente de aprobación	Luis · 09:55	Ver	Resolver

Cada alerta guarda quién la generó, con qué dato y cómo se resolvió: es la trazabilidad que exige una auditoría de inventario.

Diana
Administradora
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 24. Vista de alertas: la cola de trabajo del administrador, con el contexto de cada caso.

- **Qué resuelve:** el panel gerencial informa «37 alertas gestionadas», pero faltaba el lugar donde se gestionan. Esta es la cola de trabajo del administrador durante la toma.
- **Qué tiene:** una tarjeta por alerta con su tipo, el artículo y la bodega, el detalle de por qué se disparó, quién la generó y a qué hora, y las dos acciones: ver el caso o resolverlo.
- **Los cuatro tipos que produce el sistema:** desviación de cantidad, saldo negativo del sistema, unidad de medida equivocada y artículo fuera del catálogo — los mismos que devuelve validar_conteo en la sección 12.8.

MENÚ · REPORTES los archivos oficiales que salen de la toma

5.20 Reportes y consolidado

Inicio
Pedidos
Conteo
Legalización
Bodegas
Auditoría
Reportes
Panel

Reportes · consolidado de la toma
ARCHIVOS
Colsubsidio

Consolidado de la toma
Análisis de consumo

Archivos generados con los códigos oficiales de la base

Diferencias por bodega
XLSX · hoy 00:01 · 12 filas
6 bodegas con descuadre

Diferencias por bodega
XLSX · hoy 23:58 · 12 filas
6 bodegas con descuadre

Consolidado para My Inventory
XLSX · hoy 23:58 · 196 filas
Listo para carga

Consolidado para My Inventory
XLSX · hoy 20:27 · 1 filas
Listo para carga

Generar consolidado
Generar diferencias

Imprimir

Por voz: «genérame el consolidado» produce el mismo archivo sin tocar la pantalla. Dele clic a cualquier tarjeta para ver su contenido aquí al lado.

Vista previa · Diferencias por bodega

codigo	articulo	unidad	bodega	contado	sistema	diferencia
97503004	TABLA ACRILICA PICAR BLANCO 50X38CM FB	Unidad	ALMACEN SUMINISTROS	3	5	-2
1483116	ACEITE DE OLIVA 10ML /BOLSA SOBRE X50 UN	Unidad	RESTAURANTE FUENTES AYB	22	10	+12
1483116	ACEITE DE OLIVA 10ML /BOLSA SOBRE X50 UN	Unidad	RESTAURANTE FUENTES AYB	22	10	+12
97503004	TABLA ACRILICA PICAR BLANCO 50X38CM FB	Unidad	ALMACEN SUMINISTROS	3	5	-2
29054	MR TEA 500	Unidad	KIOSCO PISCIGIROS AYB	163.3	71	+92.3
29054	MR TEA 500	Unidad	KIOSCO PISCIGIROS AYB	163.3	71	+92.3
SIN-0218	SALPICON PISCILAGO (PA)	Unidad	KIOSCO TAQUILLA AYB	23	10	+13
SIN-0218	SALPICON PISCILAGO (PA)	Unidad	KIOSCO TAQUILLA AYB	23	10	+13

Mostrando 8 de 12 filas. Los códigos SIN-#### corresponden a registros del extracto que llegaron sin número de artículo.

Figura 25. Vista de reportes: los archivos oficiales listos para descargar, imprimir o enviar — también por voz.

- **Qué tiene:** la lista de archivos generados con su estado (el consolidado para My Inventory «listo para carga»), la vista previa con códigos y nombres oficiales —incluido el negativo del sistema ya diagnosticado—, y las tres salidas: descargar, imprimir o enviar al administrador.
- **Qué conecta:** POST /api/reportes ejecuta el consolidado de la sección 12.11 (SQL sobre las tablas reales) y deja el XLSX/CSV en la carpeta de reportes; por voz, «genérame el consolidado» llega aquí a través del agente.

El informe de Power BI insertado en la aplicación.

5.21 Análisis de consumo: lo pedido contra lo usado

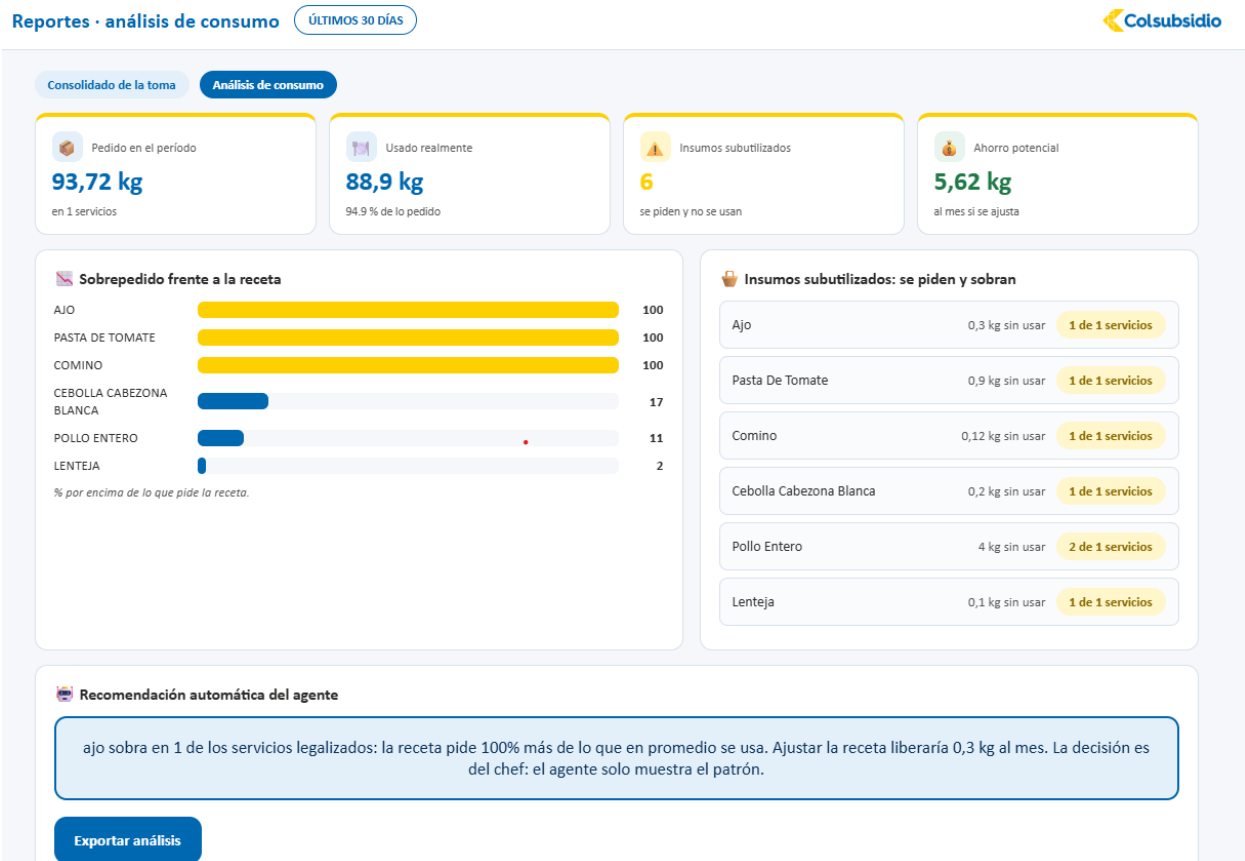


Figura 26. Vista de análisis: dónde se está sobrepidiendo y qué insumos se piden y no se usan.

- **Responde al «suma puntos» del reto:** el reto pide reportes útiles sobre qué se pidió contra qué se usó, dónde hay insumos subutilizados y dónde se está sobrepidiendo. Esta vista es exactamente eso, construida sobre los datos que dejan las legalizaciones.
- **Qué tiene:** lo pedido y lo usado del período con su porcentaje de aprovechamiento, el conteo de insumos subutilizados y el ahorro potencial mensual; las barras de sobrepedido frente a la receta; y la tabla de insumos que se piden y sobran, con en cuántos servicios ocurre.

- **La recomendación y su límite:** el agente calcula el patrón («la receta pide 1,0 kg de alcaparras y el consumo real promedio es 0,4 kg») y sugiere el ajuste, pero la decisión sigue siendo del chef. El sistema muestra evidencia, no cambia recetas: eso está fuera del alcance del reto.

MENÚ · PANEL *el informe de Power BI insertado en la aplicación*

5.22 Panel: resumen ejecutivo

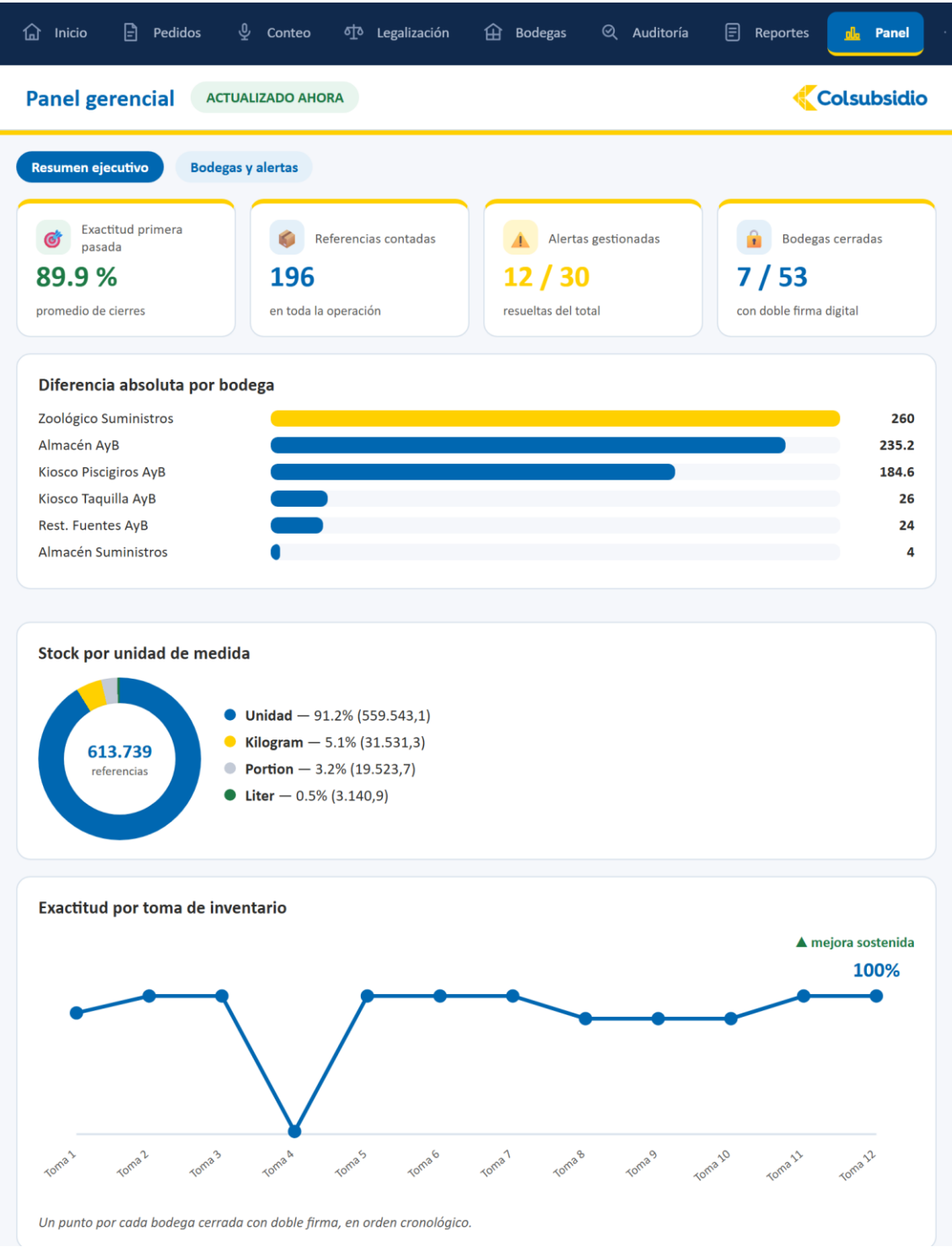


Figura 27. Panel — página 1 (Resumen ejecutivo): los cuatro indicadores del cierre, la diferencia por bodega, la composición del stock y la tendencia entre tomas.

- **Qué tiene:** cuatro tarjetas de indicador con su variación (exactitud 98,2 % con su flecha, referencias contadas, alertas gestionadas y bodegas cerradas), las diferencias por bodega con la crítica resaltada en dorado, la dona de stock por unidad de medida y la tendencia de exactitud entre tomas.
- **Qué conecta:** no es una gráfica dibujada aparte: es el informe de Power BI insertado en la aplicación, alimentado por el modelo semántico sobre PostgreSQL. Los pasos para construirlo están en la sección 13.

5.23 Panel: bodegas y alertas



Figura 28. Panel — página 2 (Bodegas y alertas): los negativos corregidos, el tiempo por bodega, el mapa de las 48 bodegas y los descuadres recurrentes.

- **Qué tiene:** los 79 saldos negativos llevados a cero, el tiempo promedio por bodega con su mejora frente al conteo manual, los alias aprendidos por el agente, el mapa de puntos con el estado de las 48 bodegas, las alertas por tipo y la tabla de descuadres recurrentes con su acción sugerida.
- **Para qué sirve:** esta página responde la pregunta de fondo del reto: no solo cuántos descuadres hubo, sino dónde está la causa raíz para que la próxima toma tenga menos.

MENÚ · AJUSTES los parámetros que el administrador mueve sin tocar código

5.24 Ajustes: la aplicación se adapta a la operación

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mi perfil

Cerrar sesión

Ajustes · configuración del sistema

ADMINISTRADORA

Colsubsidio

Configuración

Gestión de usuarios

Recetas

Registro de trazabilidad

Validación de datos

Umbral de anomalía

50

%

Bloquear cantidades negativas

Exigir confirmación en alertas

Permitir crear productos pendientes

Las reglas fijas del reto (bloquear negativos, exigir confirmación) no se pueden desactivar: son la garantía de integridad de la toma.

Conexión y sincronización

Modo sin conexión

Refresco del tablero

tiempo real

Refresco de Power BI

15 minutos

Administración

Usuarios activos

4

Aprobaciones pendientes

1

Acerca de CuentaVoz

Versión

1.0.0

Modelo del agente

gemini-flash-latest

Base de datos

SQLite

Idioma del reconocimiento

es-CO

Guardar configuración

Restaurar valores

Figura 29. Vista de ajustes: los parámetros que el administrador puede mover sin tocar el código.

- **Validación de datos:** el umbral de anomalía (50 % por defecto), el bloqueo de cantidades negativas, la confirmación obligatoria ante alertas y el permiso para crear productos. Son los mismos valores del archivo .env de la sección 12.2, pero editables desde la pantalla.
- **Conexión y sincronización:** modo sin conexión, refresco del tablero, refresco de Power BI y reintentos automáticos.
- **Administración:** gestión de usuarios, aprobaciones pendientes, respaldo automático diario y registro de auditoría — visible solo para el perfil de administrador.

- **Acerca de:** versión de la aplicación, modelo del agente en uso, versión de la base y fecha de la última copia de seguridad. Esta tarjeta es la primera que pide soporte cuando algo falla.

Los datos, la foto y las preferencias de cada persona.

5.25 Gestión de usuarios y permisos

Ajustes · Gestión de usuarios 12 ACTIVOS Colsubsidio

9 auxiliares 3 administradores 2 inactivos + Nuevo usuario

Persona	Perfil	Bodegas asignadas	Último acceso	Estado
Luis G. Nieto	Auxiliar	3 bodegas	hoy 07:42	Activo
Stephanie	Auxiliar	2 bodegas	hoy 07:55	Activo
Valentina Burbano	Auxiliar	4 bodegas	hoy 08:10	Activo
Diana	Administradora	todas	hoy 07:30	Activo
Carlos M.	Auxiliar	1 bodega	12 jul	Inactivo

Qué puede hacer cada perfil

- Contar por voz y corregir sus propios registros **Auxiliar y administrador**
- Crear productos y bodegas (quedan pendientes) **Auxiliar y administrador**
- Aprobar creaciones, auditar y cerrar bodegas **Solo administrador**
- Gestionar usuarios y cambiar los ajustes del sistema **Solo administrador**

Diana Administradora
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 30. Vista de usuarios: quién puede entrar, con qué perfil y a qué bodegas.

- **Qué resuelve:** la vista de ajustes anuncia «gestión de usuarios · 12 activos» pero no había pantalla detrás. Sin ella, dar de alta a un auxiliar nuevo obligaría a tocar la base de datos.
- **Qué tiene:** la lista de personas con su perfil, las bodegas que tiene asignadas, su último acceso y su estado; el botón para crear una persona nueva; y la tabla que deja explícito qué puede hacer cada perfil.
- **La regla de permisos, escrita en la propia pantalla:** el auxiliar cuenta y crea (pendiente de aprobación); solo el administrador aprueba, audita, cierra bodegas y gestiona usuarios. Es la traducción visual de `requiere_perfil` de la sección 14.2.

5.26 Registro de trazabilidad

CuentaVoz
Tu voz cuenta

Ajustes · Registro de trazabilidad INMUTABLE Colsubsidio

Todo lo que ocurre en CuentaVoz queda registrado con persona, fecha y dato. El registro no se puede editar ni borrar.

Hoy ▼ Todas las personas ▼ Todas las acciones ▼ Exportar

Hora	Persona	Acción	Detalle
10:12:04	Diana	CIERRE	Restaurante Fuentes AyB cerrada con doble firma
10:11:38	Stephanie	FIRMA	Conteo 1 firmado · 178 referencias
09:55:12	Luis	CREACIÓN	Ají CASERO PISCILAGO creado, pendiente de aprobación
09:48:07	Valentina	ALERTA	Unidad corregida: ARROZ de unidades a kilos
08:20:45	Luis	CORRECCIÓN	CAZUELA 16 ONZAS: 90 → 9 (valor anterior conservado)
08:19:52	Luis	ALERTA	Desviación +800 % detectada antes de guardar
07:42:10	Luis	APERTURA	Almacén de Suministros abierto · sesión bloqueada

Cumple la Ley 1581 de 2012: se registra la acción, no datos personales sensibles. El registro sustenta cualquier auditoría de inventario.

Por qué el registro es inmutable

Una corrección no borra el valor anterior: crea un registro nuevo que apunta al original (campo corrige_a de la tabla Conteo). Así, si alguien pregunta «¿quién cambió este dato y por qué?», la respuesta está en el sistema y no en la memoria de nadie.

Diana
Administradora
Colsubsidio
Solución para Colsubsidio · v1.0.0

Figura 31. Vista de trazabilidad: quién hizo qué, cuándo y con qué dato — un registro que no se puede editar.

- **Qué resuelve:** los ajustes muestran «registro de auditoría: activado» y la sección 14 promete trazabilidad total, pero no existía la pantalla donde se consulta. Para una empresa que audita inventarios, esta vista no es opcional.
- **Qué tiene:** la lista cronológica de acciones con hora exacta, persona, tipo de acción y el detalle del dato; filtros por fecha, persona y tipo; y la exportación para llevar el registro a una auditoría externa.
- **Por qué es inmutable:** una corrección no borra el valor anterior: crea un registro nuevo que apunta al original mediante el campo corrige_a de la tabla Conteo (sección 12.5). Por eso el sistema puede responder «quién cambió este dato y por qué» sin depender de la memoria de nadie.
- **Cumplimiento:** el registro guarda la acción y el dato de inventario, no información personal sensible, en línea con la Ley 1581 de 2012 (sección 14.6).

MENÚ · AYUDA el soporte dentro de la aplicación, para no depender de un manual impreso

5.27 Centro de ayuda y estado del sistema



Figura 32. Vista de ayuda: preguntas frecuentes, soporte en vivo, estado del sistema y la guía de comandos de voz.

- **Qué resuelve:** la pantalla de conteo tiene un botón «Pedir ayuda» desde la primera versión, pero no llevaba a ninguna parte. Y en una bodega, con las manos ocupadas, nadie va a buscar un manual impreso.
- **Qué tiene:** las cinco preguntas que de verdad surgen en bodega (cómo corregir, qué hacer si el agente no entiende, qué significa una alerta, qué pasa sin internet, si se pueden contar dos bodegas a la vez); el soporte en vivo con el administrador disponible y la mesa de ayuda de Colsubsidio; el estado de los tres servicios; y la guía rápida de comandos de voz.
- **El estado del sistema no es decorativo:** si el agente de voz aparece caído, la persona sabe de inmediato que debe usar el modo formulario (vista 5.10) en lugar de pensar que hace algo mal.

MENÚ · MI PERFIL los datos personales de quien usa la aplicación

5.28 Mi perfil: datos personales y foto

CuentaVoz
Tu voz cuenta

Mi perfil · Datos personales SESIÓN ACTIVA Colsubsidio

Datos personales

Nombre completo: Luis Guillermo Nieto Patiño

Correo corporativo: lnieto@colsubsidio.com

Teléfono de contacto: (+57) 300 000 0000

Código de empleado: CS-48127

Bodegas asignadas: Almacén Suministros, Almacén AyB, Kiosco Parquadero

Seguridad de la cuenta

Último acceso: hoy 07:42 · tableta TB-14
Su PIN vence en 45 días

Preferencias de voz

Idioma del reconocimiento: Español (Colombia)

Velocidad de la respuesta: Normal

Confirmación hablada: Activada

Acciones: Cambiar foto, Cambiar PIN, Registrar huella, Cerrar sesión en todos los dispositivos, Guardar cambios, Cerrar sesión

Footer: Luis G. Nieto, Auxiliar de inventarios, Colsubsidio, Solución para Colsubsidio · v1.0.0

Figura 33. Vista de perfil: la persona actualiza sus datos, su foto, su PIN y sus preferencias de voz.

- **Qué tiene:** la foto con el botón «Cambiar foto», los datos personales editables (nombre, correo corporativo, teléfono y código de empleado), las bodegas asignadas, la seguridad de la cuenta (cambiar PIN, registrar huella, cerrar sesión en todos los dispositivos) y las preferencias de voz: idioma del reconocimiento, velocidad de la respuesta y confirmación hablada.
- **Qué conecta:** PUT /api/usuarios/yo actualiza los datos; la foto se sube por POST /api/usuarios/yo/foto y se guarda como archivo, no en la base. El cambio de PIN reutiliza hash_clave de la sección 14.2, así que la contraseña nueva tampoco queda en texto plano.
- **Cómo se llega:** por la entrada «Mi perfil» del menú lateral, disponible para cualquier perfil.

MENÚ · CERRAR SESIÓN la salida segura, sin perder trabajo

5.29 Cerrar sesión con trabajo sin guardar

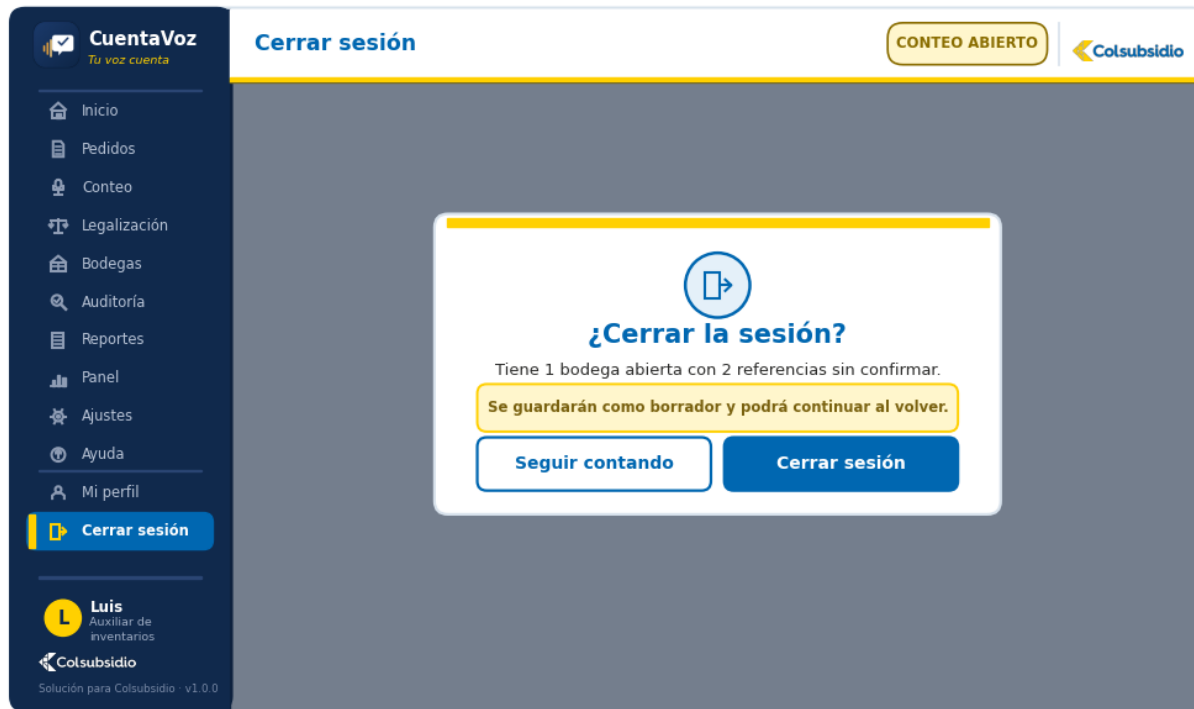


Figura 34. Ventana de confirmación al salir: avisa lo que quedaría sin confirmar y ofrece volver.

- **Qué tiene:** el aviso exacto de lo que hay abierto («1 bodega abierta con 2 referencias sin confirmar»), la promesa de que se guarda como borrador, y las dos salidas: seguir contando o cerrar sesión.
- **Por qué así:** en una bodega, salir por error con media hora de conteo encima sería el peor momento para perder datos. Por eso el cierre de sesión nunca es un botón silencioso: la sesión se cierra, pero el trabajo queda a salvo.

5b. El código del diseño adaptable

5b.0 Diseño adaptable: la misma aplicación en cualquier dispositivo

CuentaVoz corre en el navegador del dispositivo que haya disponible — tableta, celular o computador —, sin instalar nada. Todas las vistas de este capítulo son la misma aplicación web empresarial: una barra lateral de navegación, una barra superior con la sección activa y el área de trabajo. Sobre esa base, el código React reorganiza los bloques según el ancho disponible mediante cuatro puntos de quiebre (breakpoints), sin cambiar una sola línea de la lógica del agente:

Ancho de pantalla	Dispositivos típicos	Cómo se reorganiza la interfaz
Menos de 600 px	Celular en vertical	Una sola columna; la barra lateral se colapsa en un menú; el micrófono ocupa el centro y los botones se apilan en cuadrícula de dos por dos.

Ancho de pantalla	Dispositivos típicos	Cómo se reorganiza la interfaz
600 a 1023 px	Celular horizontal, tableta vertical	Dos columnas: conversación a la izquierda y micrófono a la derecha; barra lateral reducida a íconos.
1024 a 1279 px	Tableta horizontal (el equipo de bodega)	Disposición de referencia de las vistas 5.1 a 5.20.
1280 px o más	Computador de escritorio y portátil	Barra lateral completa con navegación y más datos visibles a la vez, sin desplazamiento.

La regla de uso es la misma en todos los formatos: la voz como camino rápido, el teclado como respaldo y botones grandes pensados para usarse de pie o con guantes. El esquema completo de las tres disposiciones está en la sección 5b, con su código.

Los cuatro formatos de la sección 5.21 nacen de tres herramientas en el frontend: la cuadrícula fluida de CSS Grid, cuatro reglas de @media que ajustan el layout al ancho y un único hook de React que detecta el dispositivo y oculta o muestra la barra lateral.



Figura 35. Las tres disposiciones de pantalla: mismos bloques, distinta organización según el ancho disponible.

5b.1 Variables de diseño y tipografía

Archivo: `frontend/src/index.css` — variables de marca (parte 1)

```
/* Paleta oficial de CuentaVoz */
:root {
  --azul: #0067B1; /* azul Colsubsidio */
  --azul-prof: #10294C;
  --azul-claro: #E4F0F9;
  --dorado: #FFD000; /* amarillo Colsubsidio */
  --dorado-bg: #FFF6CC;
  --verde: #1E7A46;
  --verde-bg: #E8F5EE;
  --gris-txt: #575756; /* grafito Colsubsidio */
  --gris-fondo: #EFEFEF;

  /* Tipografía – Calibri en Windows, Carlito en Linux */
  --fuente: "Calibri", "Carlito", "Liberation Sans", sans-serif;
```

```

/* Espacios */
--radio: 12px;
--sombra: 0 2px 8px rgba(27,58,107,.12);
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: var(--fuente); font-size: 16px;
  color: var(--gris-txt); background: #fff; }
button { font-family: var(--fuente); cursor: pointer; border: none; }

```

5b.2 Cuadrícula principal y puntos de quiebre

Archivo: **frontend/src/index.css — layout responsivo (parte 2)**

```

/* — Layout base (tableta horizontal ≥ 1024 px) — */
.app-root {
  display: grid;
  grid-template-rows: 56px 4px 1fr; /* barra + faja dorada + contenido */
  height: 100dvh;
  overflow: hidden;
}

.contenido {
  display: grid;
  grid-template-columns: 1fr;
  overflow-y: auto;
  padding: 16px 18px;
  gap: 14px;
}

/* — Escritorio ≥ 1280 px: barra lateral */
@media (min-width: 1280px) {
  .app-root { grid-template-rows: 60px 4px 1fr; }
  .contenido {
    grid-template-columns: 200px 1fr;
    padding: 20px;
  }
  .sidebar { display: flex; flex-direction: column; gap: 6px; }
}

/* — Tableta vertical 768-1023 px: sidebar colapsado */
@media (min-width: 768px) and (max-width: 1023px) {
  .sidebar { width: 56px; } /* solo íconos */
  .sidebar span { display: none; } /* oculta el texto */
}

/* — Móvil < 600 px: columna única */
@media (max-width: 599px) {
  .app-root { grid-template-rows: 48px 3px 1fr; }
  .contenido { padding: 12px 10px; gap: 10px; }
  .sidebar { display: none; } /* sin barra lateral */
  .barra-nombre { font-size: 15px; }
  .chip { font-size: 11px; padding: 3px 8px; }
}

```

5b.3 Botones responsivos

Archivo: **frontend/src/index.css — botones (parte 3)**

```

.btn {
  border-radius: var(--radio);
  font-weight: bold;
  padding: 14px 20px;
  font-size: 15px;
  transition: opacity .15s;
}
.btn:active { opacity: .82; }
.btn-primario { background: var(--azul); color: #fff; }
.btn-borde { background: #fff; color: var(--azul);
  border: 2px solid var(--azul); }
.btn-verde { background: var(--verde); color: #fff; }
.btn-dorado { background: var(--dorado); color: #fff; }

/* Móvil: botones más grandes y en cuadrícula 2x2 */
@media (max-width: 599px) {
  .btn { padding: 18px 14px; font-size: 14px; min-height: 56px; }
  .grilla-botones {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 10px;
  }
}

/* Tableta y escritorio: botones en fila */
@media (min-width: 600px) {
  .grilla-botones { display: flex; gap: 12px; flex-wrap: wrap; }
  .btn { padding: 13px 22px; }
}

```

5b.4 El micrófono: diferente en cada formato

Archivo: `frontend/src/index.css` — micrófono (parte 4)

```

.mic-btn {
  border-radius: 50%;
  background: var(--azul);
  color: #fff;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: var(--sombra);
  user-select: none;
  -webkit-user-select: none;
}

/* Tableta: micrófono en columna derecha, tamaño medio */
@media (min-width: 600px) and (max-width: 1279px) {
  .mic-btn { width: 96px; height: 96px; font-size: 36px; }
}

/* Móvil: micrófono central, toma el ancho disponible */
@media (max-width: 599px) {
  .mic-btn {
    width: min(72vw, 200px); height: min(72vw, 200px);
    font-size: 48px; margin: 16px auto;
  }
}

/* Escritorio: micrófono discreto en la barra superior */
@media (min-width: 1280px) {
  .mic-btn { width: 44px; height: 44px; font-size: 20px; }
}

```



```
}

```

5b.5 Hook de detección de dispositivo

Archivo: `frontend/src/hooks/useDispositivo.js`

```
import { useState, useEffect } from "react";

export function useDispositivo() {
  const [tipo, setTipo] = useState(detector());

  function detector() {
    const w = window.innerWidth;
    if (w < 600) return "movil";
    if (w < 1280) return "tableta";
    return "escritorio";
  }

  useEffect(() => {
    const obs = new ResizeObserver(() => setTipo(detector()));
    obs.observe(document.documentElement);
    return () => obs.disconnect();
  }, []);

  return tipo;
}
```

5b.6 Cómo lo usa la pantalla de conteo

Archivo: `frontend/src/vistas/Conteo.jsx — layout adaptable`

```
import { useDispositivo } from "../hooks/useDispositivo";

export default function Conteo({ sesion, token }) {
  const dispositivo = useDispositivo();

  return (
    <div className={`conteo conteo--${dispositivo}`}>
      {/* chips siempre en fila, se comprimen en movil */}
      <div className="chips">
        <Chip tipo="primario">EN CONTEO</Chip>
        <Chip>Avance: {sesion.hechas} de {sesion.total}</Chip>
        <Chip tipo="verde">Alertas: {sesion.alertas}</Chip>
      </div>

      {/* burbuja siempre arriba */}
      <Burbuja texto={respuesta} />

      {/*
        Móvil      → micrófono central, botones 2x2 abajo
        Tableta    → micrófono a la derecha en flex-row
        Escritorio → micrófono en la barra superior
      */}
      {dispositivo === "movil" ? (
        <div className="movil-layout">
          <MicBoton grande onPress={escuchar} />
          <Registros items={registros} />
          <div className="grilla-botones">

```

```

        <button className="btn btn-verde" onClick={confirmar}>Confirmar</button>
        <button className="btn btn-borde" onClick={corregir}>Corregir</button>
        <button className="btn btn-borde" onClick={abrirTeclado}>Teclado</button>
        <button className="btn btn-dorado" onClick={pedirAyuda}>Ayuda</button>
      </div>
    </div>
  ) : (
    <div className="tableta-layout">
      <div className="izquierda">
        <Burbuja texto={respuesta} />
        <Registros items={registros} />
      </div>
      <div className="derecha">
        <MicBoton onPress={escuchar} />
      </div>
      <div className="grilla-botones">
        <button className="btn btn-verde" onClick={confirmar}>Confirmar</button>
        <button className="btn btn-borde" onClick={corregir}>Corregir</button>
        <button className="btn btn-borde" onClick={abrirTeclado}>Teclado</button>
        <button className="btn btn-dorado" onClick={pedirAyuda}>Ayuda</button>
      </div>
    </div>
  )}
</div>
);
}

```

Regla de oro del diseño adaptable de CuentaVoz: el CSS reorganiza; el código de negocio no cambia. La frase dictada siempre va al mismo endpoint, la validación siempre ocurre en el backend y la voz siempre responde — solo el tamaño del botón y la posición del micrófono se adaptan al dispositivo que tenga la persona en la mano.

6. Cómo piensa el agente: intención y salida estructurada

El «pensar» se vuelve construible cuando el modelo responde siempre en un formato fijo. En cada turno, Gemini clasifica la frase en una de ocho intenciones —contar, corregir, consultar, explicar, crear, navegar, ayuda, cerrar— y extrae los datos. Las instrucciones del sistema se envían como `system_instruction` y la respuesta se fuerza a JSON con `response_mime_type`, dos capacidades nativas de la API de Gemini (el código real está en la sección 12.9):

6.1 Instrucciones del sistema (prompt del agente)

Eres CuentaVoz, el asistente de inventarios de Colsubsidio.
Hablas español claro, en frases cortas, con respeto y calidez.

Reglas:

1. Nunca inventes artículos: usa solo los que devuelva `buscar_articulo`.
2. Antes de guardar, repite siempre artículo, cantidad y unidad.
3. Con varios candidatos, lee las opciones y deja que la persona elija.
4. Si `validar_conteo` devuelve una alerta, pregunta antes de continuar.
Nunca guardes un dato con alerta sin confirmación explícita.
5. Si la persona no entiende algo (una tabla, un reporte, un paso), explícalo con palabras simples y ofrece leer solo las diferencias.
6. Si piden datos, usa `consultar_stock` y resume en voz; ofrece `generar_reporte` si quieren el archivo o imprimirlo.
7. Si no puedes resolver un problema, dilo con honestidad y ofrece llamar al administrador de bodega.

Responde SOLO con el JSON del esquema acordado.

6.2 Esquema de salida (el JSON que el backend procesa)

```
{
  "intencion": "contar",
  "articulo_texto": "tabla para picar blanca",
  "cantidad": 3,
  "unidad": "Unidad",
  "herramienta": {
    "nombre": "buscar_articulo",
    "argumentos": { "texto": "tabla para picar blanca" }
  },
  "respuesta_hablada": "Tengo dos blancas: la 50 por 38 FB o la
                        P barra picar. ¿Cuál de las dos?"
}
```

Con este contrato, la tableta solo reproduce respuesta_hablada, el orquestador solo ejecuta herramienta, y cada integrante del equipo programa su pieza sin ambigüedades.

7. Las manos del agente: herramientas del backend

Cada herramienta es un servicio con entradas y salidas definidas; son el único camino del agente hacia los datos. La tercera columna indica dónde vive su código en la sección 12:

Herramienta	Recibe → Devuelve	Código
abrir_bodega	nombre de la bodega, usuario → sesión bloqueada + total de referencias	12.12
buscar_articulo	texto libre dictado → candidatos oficiales con código y confianza	12.7
validar_conteo	artículo, cantidad, unidad, bodega → ok, o alerta con motivo	12.8
guardar_conteo	captura confirmada → registro con trazabilidad (usuario, hora)	12.10
corregir_conteo	id o «el último», valor nuevo → registro corregido con rastro	12.10
consultar_stock	filtros (artículo, bodega, solo diferencias) → datos + resumen en voz	12.10 / 12.12
explicar_reporte	tabla en pantalla → significado de columnas y filas clave	12.10
avance_bodega	bodega activa → contadas, pendientes, alertas abiertas	12.10
crear_articulo / crear_bodega	nombre, unidad, bodega → registro «pendiente de aprobación»	12.10
generar_reporte	alcance y formato → archivo XLSX/CSV listo para descargar o imprimir	12.11

Herramienta	Recibe → Devuelve	Código
cerrar_bodega	bodega, perfil → comparativo 1 vs. 2 vs. sistema + cambio de estado	12.12

8. Arquitectura: cómo queda construido y cómo se conecta

8.1 Las cinco capas



Figura 36. Arquitectura de capas; cada capa solo conversa con la vecina. En la capa de agente, el LLM es Gemini vía Google AI Studio.

8.2 Componentes y conexiones: quién habla con quién

Este es el mapa físico de la construcción. Cada caja es un componente que se programa por separado (y tiene su archivo en la sección 12), y cada flecha es un cable concreto que hay que dejar funcionando:

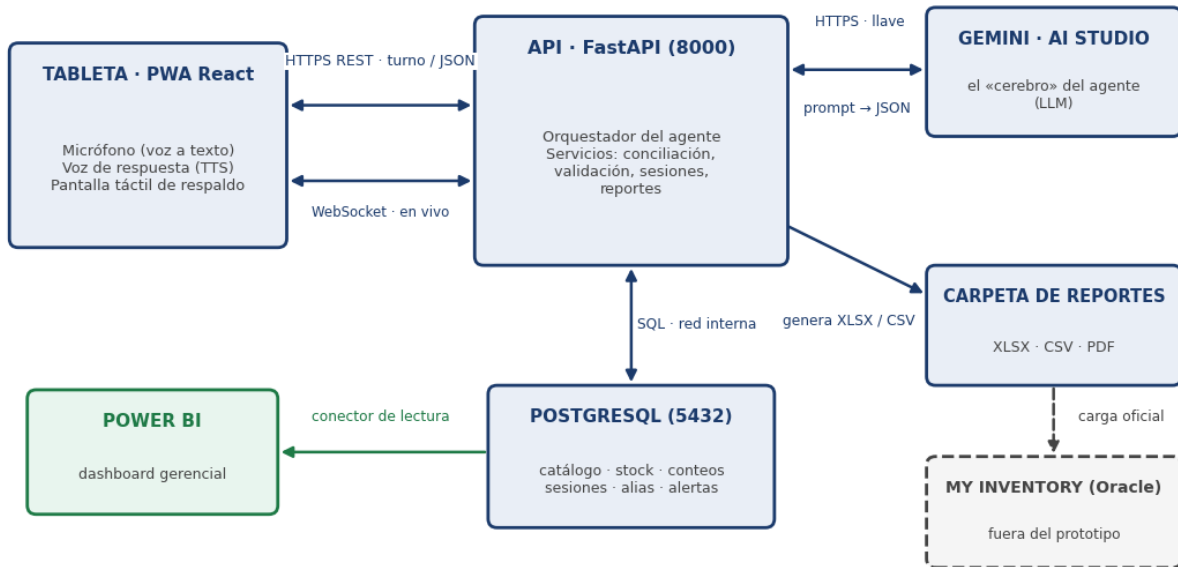
COMPONENTES: QUIÉN SE CONECTA CON QUIÉN (Y POR DÓNDE)

Figura 37. Componentes y canales; la línea punteada hacia My Inventory es el procedimiento oficial de carga, fuera del prototipo.

Conexión (origen → destino)	Canal	Qué viaja y dónde está el código
Tableta → API	HTTPS REST, puerto 8000	la frase transcrita y la sesión; de vuelta, el JSON del turno — código 12.13
Tableta ↔ API	WebSocket	estado en vivo de las 48 bodegas para el tablero — código 12.12 y 12.14
API → Gemini (Google AI Studio)	HTTPS (internet), llave GOOGLE_API_KEY	system_instruction + contexto + frase; de vuelta, el JSON de intención — código 12.9
API → PostgreSQL	SQL, puerto 5432 (red interna de Docker)	lecturas de catálogo/stock y escritura de conteos con trazabilidad — código 12.4 a 12.8
API → carpeta de reportes	sistema de archivos	XLSX/CSV generados para descargar o imprimir — código 12.11
Reportes → My Inventory	procedimiento oficial de carga (fuera del prototipo)	el consolidado con los nombres y códigos oficiales de la base
PostgreSQL → Power BI	conector de lectura	las tablas del modelo para el dashboard — pasos en la sección 13

8.3 Secuencia de un turno completo

Y así se ve, mensaje por mensaje, el recorrido de una frase real a través de todos los componentes:

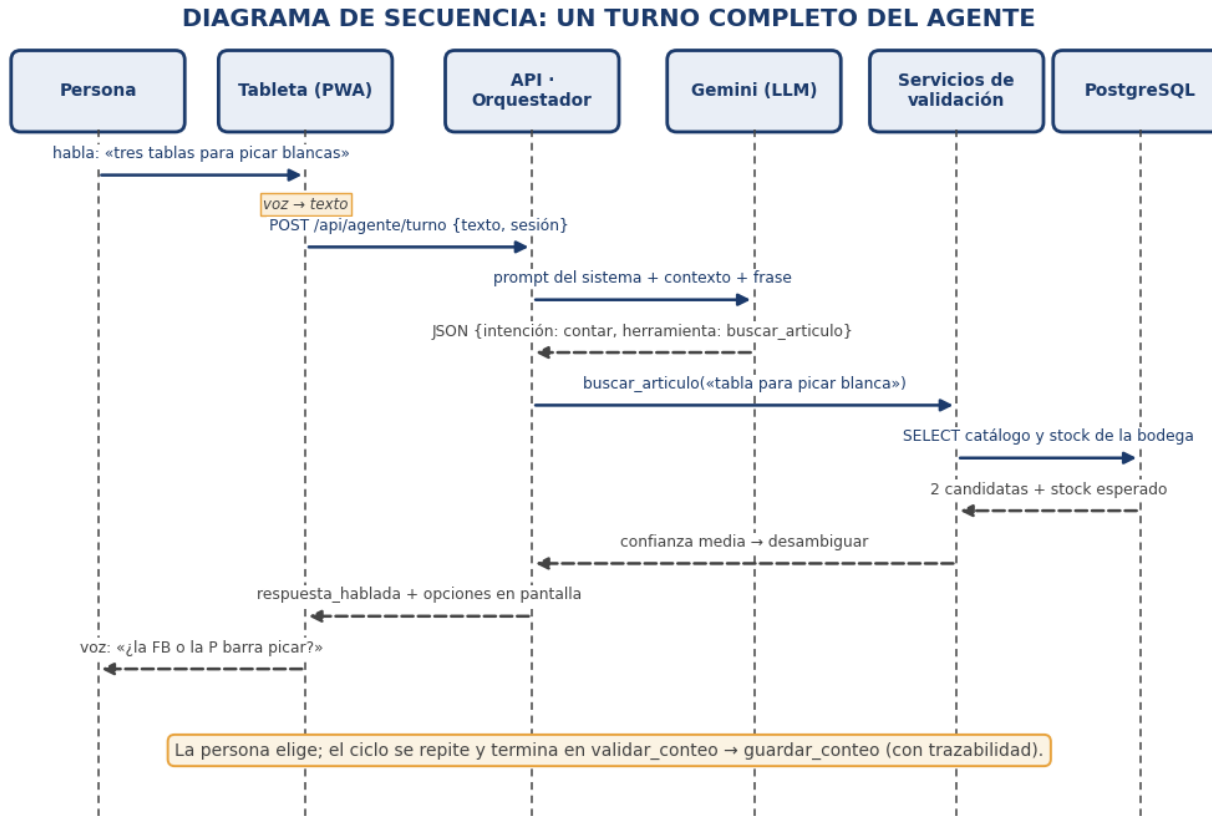
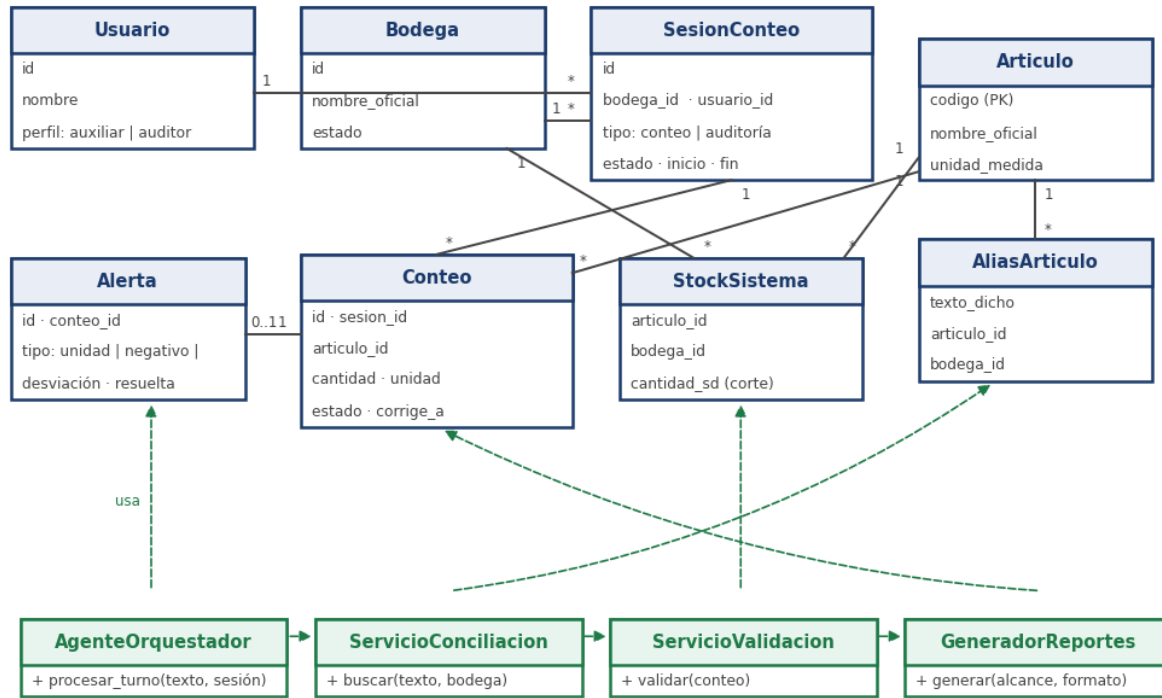


Figura 38. Secuencia de un turno: las flechas azules son llamadas y las punteadas, respuestas.

9. Diagrama de clases y modelo de datos

Las entidades del diagrama son, a la vez, las tablas de PostgreSQL: quien programe el backend crea exactamente estas clases (el código está en 12.5), y quien arme Power BI encuentra estas mismas tablas. Abajo, en verde, la capa de servicios que las usa:

DIAGRAMA DE CLASES Y MODELO DE DATOS (entidades = tablas de la base)

Verde: capa de servicios (usa las entidades). Las flechas punteadas indican dependencia «usa».

Figura 39. Clases y relaciones; las multiplicidades (1, *) se leen, por ejemplo: una sesión tiene muchos conteos.

- **Lectura rápida:** una Bodega tiene muchas SesionConteo (una por conteo inicial y otra por auditoría); cada SesionConteo tiene muchos Conteo; cada Conteo apunta a un Articulo del catálogo oficial y puede tener 0 o 1 Alerta; StockSistema guarda la cantidad del corte por artículo y bodega (el extracto de My Inventory); AliasArticulo acumula las formas reales de decir cada artículo que el agente va aprendiendo.
- **Campo clave para la auditoría:** Conteo.corrige_a enlaza una corrección con el registro original: nada se borra, todo queda trazable.

10. Estados de la bodega y trabajo en paralelo

ESTADOS DE UNA BODEGA (varias bodegas avanzan en paralelo, cada una con su sesión)



Figura 40. Estados de una bodega; el candado de sesión evita conteos duplicados y permite bodegas en paralelo.

El estado vive en `SesionConteo` y se difunde por el `WebSocket` al tablero (vista 5.13): cuando el auxiliar abre una bodega, todos ven «en conteo» al instante; cuando el auditor cierra, la bodega pasa a «cerrada» con la doble firma registrada.

11. Conversaciones de ejemplo (guiones implementables)

La columna del medio es lo que ocurre por dentro (Pensar–Actuar–Validar); la persona solo escucha la columna derecha. Cifras ilustrativas.

11.1 Conteo con conciliación de nombre y anomalía

La persona dice	El agente piensa y hace	El agente responde
«Iniciar conteo en almacén de suministros.»	navegar → abrir_bodega: sesión bloqueada; 312 referencias.	«Listo, almacén de suministros abierto. Dícteme el primer producto cuando quiera.»
«Tres tablas para picar blancas.»	contar → buscar_articulo: dos candidatas blancas; confianza media.	«Tengo dos blancas: la acrílica 50 por 38 FB o la P barra picar 50 por 38. ¿Cuál de las dos?»
«La FB.»	código 97503004 → validar_conteo(3): ok.	«Tabla acrílica picar blanco 50 por 38 FB: tres unidades. ¿Confirma?»
«Confirмо. Ahora noventa cazuelas de dieciséis onzas.»	guardar_conteo; validar_conteo(90): alerta, el sistema espera ~10.	«Guardado. Ojo con las cazuelas: el sistema espera alrededor de diez y usted dijo noventa. ¿Las contamos otra vez o confirma noventa?»
«Uy, no: son nueve.»	corregir → cantidad 9 → validar: ok.	«Nueve cazuelas de dieciséis onzas. ¿Confirma?»
«Confirмо. ¿Cuánto me falta?»	guardar_conteo → avance_bodega: 2 de 312.	«Guardado. Llevamos dos referencias y faltan trescientas diez. Siga cuando quiera.»

11.2 «No entiendo esta tabla»: el agente explica el reporte

La persona dice	El agente piensa y hace	El agente responde
«Oiga, no entiendo esta tabla del reporte.»	explicar → explicar_reporte(pantalla actual).	«Con gusto. Cada fila es un producto de esta bodega. “Contado” es lo que usted dictó; “Sistema” es lo que My Inventory esperaba; “Diferencia” es la resta. Si sale resaltada, hay que revisarla. ¿Le leo solo las filas con diferencia?»
«Sí, solo esas.»	consultar_stock(solo diferencias): 3 filas.	«Son tres: arroz, faltan dos kilos; cazuela de dieciséis onzas, cuadró con su corrección; y aceite, falta un litro. ¿Las revisamos una por una?»
«¿Y por qué el aceite sale en negativo abajo?»	explicar → regla del mini reto de negativos.	«Ese negativo no viene de su conteo: el sistema traía -1 desde antes, seguramente una salida registrada sin la entrada. Su conteo físico es el que vale; el reporte lo marca para el administrador.»

11.3 Consulta de datos con salida impresa

La persona dice	El agente piensa y hace	El agente responde
«¿Cuánto arroz hay en total y en qué bodegas? Y me lo imprime.»	consultar → consultar_stock(«arroz», por bodega) → generar_reporte(XLSX).	«Arroz hay en cuatro bodegas: almacén AyB ciento veinte kilos, restaurante Fuentes cuarenta y cinco, kiosco taquilla doce y Piscigiros ocho: ciento ochenta y cinco en total. Ya generé el archivo: quedó en la carpeta de reportes, listo para imprimir. ¿Algo más?»
«Mándele también el Excel al administrador.»	generar_reporte(XLSX) → sesión del auditor + notificación.	«Hecho: el Excel ya aparece en la tableta del administrador de bodega, con la fecha y la hora del corte.»

12. Código completo del prototipo

Esta sección contiene el código base funcional de CuentaVoz, archivo por archivo y en el orden exacto en que se construye (el mismo de las fases de la sección 14). Cada archivo trae una nota de qué hace y con qué se conecta. Es la base para la demo: el equipo la extiende, pero con esto el flujo completo —hablar, conciliar, validar, guardar, consultar y reportar— ya funciona.

12.1 Dependencias y contenedores

Archivo: backend/requirements.txt

```
fastapi
uvicorn[standard]
sqlalchemy
psycopg2-binary
pandas
openpyxl
rapidfuzz
google-genai
python-dotenv
```

Archivo: backend/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Archivo: frontend/Dockerfile

```
FROM node:20-slim
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev", "--", "--host"]
```

12.2 Variables de entorno

La llave de Gemini se crea gratis en Google AI Studio (aistudio.google.com → «Get API key») con la cuenta de Google del equipo; es la única credencial externa del prototipo.

Archivo: .env

```
GOOGLE_API_KEY=pegue_aqui_su_llave_de_ai_studio
MODELO=gemini-3.5-flash # o el modelo flash gratuito vigente
DB_URL=postgresql://cuentavoz:cambieme@db:5432/cuentavoz
IDIOMA_VOZ=es-CO
UMBRAL_ANOMALIA=0.5 # 50 % de desviacion dispara alerta
CONFIANZA_ALTA=90 # empareja directo y solo confirma
CONFIANZA_MEDIA=60 # lee opciones y deja elegir
```

12.3 Orquestación con Docker

Archivo: docker-compose.yml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: cuentavoz
```

```

    POSTGRES_PASSWORD: cambieme
    POSTGRES_DB: cuentavoz
    volumes: [ "datos:/var/lib/postgresql/data" ]
  api:
    build: ./backend
    env_file: .env
    depends_on: [ db ]
    ports: [ "8000:8000" ]
  web:
    build: ./frontend
    environment: [ "VITE_API_URL=http://localhost:8000" ]
    depends_on: [ api ]
    ports: [ "5173:5173" ]
  volumes: { datos: {} }

```

12.4 Conexión a la base de datos

Un solo archivo crea el motor de conexión y la fábrica de sesiones; todos los demás lo importan. Con esto, cambiar de PostgreSQL a SQLite para pruebas locales es cambiar una variable.

Archivo: backend/bd.py

```

import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

class Base(DeclarativeBase):
    pass

motor = create_engine(os.getenv("DB_URL", "sqlite:///cuentavoz.db"))
Session = sessionmaker(bind=motor)

def iniciar_bd():
    import modelos # registra todas las tablas
    Base.metadata.create_all(motor)

```

12.5 Las tablas (el diagrama de clases hecho código)

Archivo: backend/modelos.py

```

from datetime import datetime
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey
from bd import Base

class Usuario(Base):
    __tablename__ = "usuario"
    id = Column(Integer, primary_key=True)
    nombre = Column(String, nullable=False)
    perfil = Column(String, nullable=False) # "auxiliar" | "auditor"

class Bodega(Base):
    __tablename__ = "bodega"
    id = Column(Integer, primary_key=True)
    nombre_oficial = Column(String, unique=True, nullable=False)
    estado = Column(String, default="pendiente")
    # pendiente | en_conteo | en_auditoria | cerrada

class Artículo(Base):

```

```

__tablename__ = "articulo"
codigo = Column(String, primary_key=True)
nombre_oficial = Column(String, nullable=False)
unidad_medida = Column(String, nullable=False)

class StockSistema(Base):
    # el extracto de My Inventory
    __tablename__ = "stock_sistema"
    id = Column(Integer, primary_key=True)
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    bodega_id = Column(Integer, ForeignKey("bodega.id"))
    cantidad_sd = Column(Float, default=0)

class AliasArticulo(Base):
    # lo que el agente va aprendiendo
    __tablename__ = "alias_articulo"
    id = Column(Integer, primary_key=True)
    texto_dicho = Column(String, index=True)
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    bodega_id = Column(Integer, nullable=True)

class SesionConteo(Base):
    __tablename__ = "sesion_conteo"
    id = Column(Integer, primary_key=True)
    bodega_id = Column(Integer, ForeignKey("bodega.id"))
    usuario_id = Column(Integer, ForeignKey("usuario.id"))
    tipo = Column(String, default="conteo") # conteo | auditoria
    estado = Column(String, default="abierta")
    inicio = Column(DateTime, default=datetime.now)
    fin = Column(DateTime, nullable=True)

class Conteo(Base):
    __tablename__ = "conteo"
    id = Column(Integer, primary_key=True)
    sesion_id = Column(Integer, ForeignKey("sesion_conteo.id"))
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    cantidad = Column(Float, nullable=False)
    unidad = Column(String)
    estado = Column(String, default="confirmado")
    # confirmado | corregido | pendiente_aprobacion
    corrige_a = Column(Integer, ForeignKey("conteo.id"), nullable=True)
    creado = Column(DateTime, default=datetime.now)

class Alerta(Base):
    __tablename__ = "alerta"
    id = Column(Integer, primary_key=True)
    conteo_id = Column(Integer, ForeignKey("conteo.id"))
    tipo = Column(String) # unidad | negativo | desviacion
    detalle = Column(String)
    resuelta = Column(Integer, default=0)

```

12.6 Carga del extracto real de My Inventory

Este script conecta el dato real con el prototipo: lee BODEGAS_Y_STOCK.xlsx tal como lo entregó Colsubsidio y llena bodegas, catálogo y stock. A los 260 registros sin código les asigna uno temporal SIN-#### para que ningún dato del extracto se pierda.

Archivo: `data/cargar_excel.py`

```

import pandas as pd
import sys; sys.path.append("../backend")
from bd import Sesion, iniciar_bd
from modelos import Bodega, Articulo, StockSistema

```

```

iniciar_bd()
s = Sesion()
xl = pd.ExcelFile("BODEGAS_Y_STOCK.xlsx")

# 1) Las 48 bodegas disponibles
bod = pd.read_excel(xl, "BODEGAS DISPONIBLES", header=1)
for n in bod.iloc[:, 2].dropna().astype(str).str.strip().unique():
    if not s.query(Bodega).filter_by(nombre_oficial=n.upper()).first():
        s.add(Bodega(nombre_oficial=n.upper()))
s.commit()

# 2) El stock de cada hoja (una hoja = una bodega con corte)
sin_codigo = 0
for hoja in [h for h in xl.sheet_names if h != "BODEGAS DISPONIBLES"]:
    nombre_b = hoja.replace("STOCK", "").strip().upper()
    b = s.query(Bodega).filter_by(nombre_oficial=nombre_b).first()
    if b is None:
        b = Bodega(nombre_oficial=nombre_b); s.add(b); s.commit()
    df = pd.read_excel(xl, hoja)
    df.columns = [c.strip() for c in df.columns]
    for _, fila in df.iterrows():
        nombre = str(fila.get("Artículo", "")).strip()
        if not nombre or nombre.lower() == "nan":
            continue
        cod = fila.get("Nr.Artículo")
        if pd.isna(cod):
            sin_codigo += 1
            cod = f"SIN-{sin_codigo:04d}"
        else:
            cod = str(cod).split(".")[0]
        if s.get(Artículo, cod) is None:
            s.add(Artículo(codigo=cod, nombre_oficial=nombre.upper(),
                          unidad_medida=str(fila.get("Unidad", "Unidad")).strip()))
        s.add(StockSistema(articulo_codigo=cod, bodega_id=b.id,
                          cantidad_sd=float(fila.get("SD", 0) or 0)))
    s.commit()

print("Bodegas:", s.query(Bodega).count(),
      "| Artículos:", s.query(Artículo).count(),
      "| Stock:", s.query(StockSistema).count())

```

12.7 Servicio de conciliación (la sugerencia oficial de Colsubsidio)

Traduce «tabla para picar blanca» al nombre y código oficiales. Primero busca en los alias ya aprendidos; si no, compara contra todo el catálogo con similitud difusa. Cada elección confirmada se guarda como alias: el agente aprende cómo habla el equipo real.

Archivo: `backend/servicios/conciliacion.py`

```

import unicodedata
from rapidfuzz import process, fuzz
from bd import Sesion
from modelos import Artículo, AliasArtículo

def normalizar(t: str) -> str:
    t = str(t).upper().strip()
    t = "".join(c for c in unicodedata.normalize("NFD", t)
                if unicodedata.category(c) != "Mn") # sin tildes
    return t.replace("P/", "PARA ").replace(" ", " ")

```

```
def buscar_articulo(texto: str, bodega_id=None) -> list[dict]:
    s = Sesion()
    consulta = normalizar(texto)
    alias = s.query(AliasArticulo).filter_by(texto_dicho=consulta).first()
    if alias:
        # ya lo aprendio antes
        a = s.get(Articulo, alias.articulo_codigo)
        return [{"codigo": a.codigo, "nombre": a.nombre_oficial,
                  "unidad": a.unidad_medida, "confianza": 100}]
    arts = s.query(Articulo).all()
    nombres = {a.codigo: normalizar(a.nombre_oficial) for a in arts}
    top = process.extract(consulta, nombres,
                          scorer=fuzz.token_set_ratio, limit=3)
    return [{"codigo": cod,
            "nombre": next(a.nombre_oficial for a in arts
                           if a.codigo == cod),
            "unidad": next(a.unidad_medida for a in arts
                           if a.codigo == cod),
            "confianza": round(punt)}
            for (_, punt, cod) in top]

def aprender_alias(texto: str, codigo: str, bodega_id=None):
    s = Sesion()
    s.add(AliasArticulo(texto_dicho=normalizar(texto),
                        articulo_codigo=codigo, bodega_id=bodega_id))
    s.commit()
```

12.8 Servicio de validación (incluye el mini reto de los negativos)

Las reglas duras, en código determinista: artículo existente, unidad coherente con el catálogo, nada de negativos, y desviación frente al stock del sistema. Es el paso 4 del ciclo y la fuente de las alertas de la vista 5.13.

Archivo: `backend/servicios/validacion.py`

```
import os
from bd import Sesion
from modelos import Articulo, StockSistema

UMBRAL = float(os.getenv("UMBRAL_ANOMALIA", 0.5))

def validar_conteo(codigo, cantidad, unidad, bodega_id) -> dict:
    s = Sesion()
    art = s.get(Articulo, codigo)
    if art is None:
        return {"ok": False, "tipo": "inexistente", "mensaje":
                "Ese articulo no esta en el catalogo. ¿Lo creamos? "
                "Quedara pendiente de aprobacion del auditor."}
    if cantidad < 0:
        # mini reto de Colsubsidio
        return {"ok": False, "tipo": "negativo", "mensaje":
                "Una cantidad no puede ser negativa: un saldo fisico "
                "nunca baja de cero. Digame el valor correcto."}
    if unidad and unidad.lower() != art.unidad_medida.lower():
        return {"ok": False, "tipo": "unidad", "mensaje":
                f"Ese articulo se maneja en {art.unidad_medida}, "
                f"no en {unidad}. ¿Cual es la cantidad en "
                f"{art.unidad_medida}?"}
    stock = s.query(StockSistema).filter_by(
        articulo_codigo=codigo, bodega_id=bodega_id).first()
    esperado = stock.cantidad_sd if stock else None
    if esperado is not None and esperado > 0:
```

```

desvio = abs(cantidad - esperado) / esperado
if desvio > UMBRAL:
    return {"ok": False, "tipo": "desviacion",
            "esperado": esperado, "mensaje":
            f"El sistema espera alrededor de {esperado:g}. "
            f"¿Confirma {cantidad:g}?"}
return {"ok": True, "esperado": esperado}

```

12.9 El cerebro: conexión con Gemini (Google AI Studio)

El paso «Pensar» del ciclo. Se usa el SDK oficial google-genai: la personalidad y las reglas van en `system_instruction`, y `response_mime_type` obliga al modelo a responder el JSON de la sección 6.2 — sin texto suelto que romper. Si Gemini no responde, el agente cae con gracia a pedir que le repitan.

Archivo: `backend/agente/cerebro.py`

```

import os, json
from google import genai
from google.genai import types

cliente = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

PROMPT_SISTEMA = """Eres CuentaVoz, el asistente de inventarios
de Colsubsidio. Hablas español claro, en frases cortas.
Reglas: (1) nunca inventes artículos, usa solo los de
buscar_articulo; (2) repite artículo, cantidad y unidad antes de
guardar; (3) con varios candidatos, lee las opciones y deja elegir;
(4) ante una alerta, pregunta antes de continuar; (5) si no
entienden una tabla o un paso, explícalo simple; (6) si piden
datos, resume y ofrece el archivo; (7) si no puedes resolver,
ofrece llamar al administrador.
Responde SOLO un JSON con: intencion, articulo_texto, cantidad,
unidad, herramienta {nombre, argumentos}, respuesta_hablada."""

def pensar(contexto: str, frase: str) -> dict:
    try:
        r = cliente.models.generate_content(
            model=os.getenv("MODELO", "gemini-3.5-flash"),
            contents=f"{contexto}\nLa persona dice: {frase}",
            config=types.GenerateContentConfig(
                system_instruction=PROMPT_SISTEMA,
                response_mime_type="application/json",
                temperature=0.2,
            ),
        )
        return json.loads(r.text)
    except Exception:
        return {"intencion": "ayuda", "respuesta_hablada":
                "Perdon, no le entendi bien. ¿Me lo repite?"}

```

12.10 El orquestador: donde el pensar se vuelve acción

Recibe cada turno, arma el contexto, consulta a Gemini y ejecuta la herramienta según la intención. Mantiene por sesión lo pendiente de confirmar y las opciones ofrecidas, que es lo que hace fluida la conversación. Las intenciones crear, cerrar y explicar siguen exactamente el mismo patrón de las que aquí aparecen (se agregan en la fase 6).

Archivo: backend/agente/orquestador.py

```

from agente.cerebro import pensar
from servicios.conciliacion import buscar_articulo, aprender_alias
from servicios.validacion import validar_conteo
from bd import Sesion
from modelos import Conteo, StockSistema, Articulo, SesionConteo

ESTADOS = {}          # memoria por sesion: pendiente, opciones, bodega

def procesar_turno(texto: str, sesion_id: int) -> dict:
    est = ESTADOS.setdefault(sesion_id, {})
    contexto = f"Bodega activa: {est.get('bodega_id')}". "
    if est.get("pendiente"):
        contexto += f"Pendiente de confirmar: {est['pendiente']}". "
    if est.get("opciones"):
        contexto += "Opciones ofrecidas: " + ", ".join(
            o["nombre"] for o in est["opciones"]) + ". "
    turno = pensar(contexto, texto)

    # 1) La persona confirma lo pendiente → se guarda de verdad
    if est.get("pendiente") and "confirm" in texto.lower():
        c = est.pop("pendiente")
        with Sesion() as s:
            reg = Conteo(sesion_id=sesion_id, **c)
            s.add(reg); s.commit()
            aprender_alias(c.get("texto_original", ""),
                           c["articulo_codigo"])
            turno["respuesta_hablada"] = "Guardado. " + avance(sesion_id)
        return turno

    # 2) Habia opciones y la persona eligio una
    if est.get("opciones"):
        elegido = _elegir(texto, est["opciones"])
        if elegido:
            est["opciones"] = None
            return _validar_y_dejar_pendiente(est, elegido,
                                              est.get("cantidad", 1), texto, turno)

    # 3) Intencion de contar → conciliar contra el catalogo
    if turno.get("intencion") == "contar":
        cand = buscar_articulo(turno.get("articulo_texto", ""),
                                est.get("bodega_id"))
        est["cantidad"] = turno.get("cantidad", 1)
        if len(cand) == 1 or cand[0]["confianza"] >= 90:
            return _validar_y_dejar_pendiente(est, cand[0],
                                              est["cantidad"], texto, turno)
        est["opciones"] = cand[:2]          # vista 5.8
        turno["opciones"] = est["opciones"]
        return turno

    # 4) Consulta de datos → resumen hablado (vista 11.3)
    if turno.get("intencion") == "consultar":
        turno["respuesta_hablada"] = consultar_stock(
            turno.get("articulo_texto", ""))
        return turno

    return turno          # ayuda, explicar, navegar: responde el modelo

def _validar_y_dejar_pendiente(est, art, cantidad, texto, turno):
    v = validar_conteo(art["codigo"], cantidad, art["unidad"],
                       est.get("bodega_id"))
    est["pendiente"] = {"articulo_codigo": art["codigo"],
                        "cantidad": cantidad, "unidad": art["unidad"],

```



```

        "texto_original": texto}
    if v["ok"]:
        turno["respuesta_hablada"] = (f"{art['nombre']}: "
                                       f"{cantidad:g} {art['unidad']}. ¿Confirma?")
    else:
        turno["respuesta_hablada"] = v["mensaje"]    # vista 5.13
        turno["alerta"] = v["tipo"]
    return turno

def _elegir(texto, opciones):
    t = texto.upper()
    for i, o in enumerate(opciones):
        if o["codigo"] in t or o["nombre"] in t: return o
        if ("PRIMERA" in t and i == 0) or ("SEGUNDA" in t and i == 1):
            return o
        if "FB" in t and "FB" in o["nombre"]: return o
    return None

def avance(sesion_id) -> str:
    with Sesion() as s:
        ses = s.get(SesionConteo, sesion_id)
        hechas = s.query(Conteo).filter_by(sesion_id=sesion_id).count()
        total = s.query(StockSistema).filter_by(
            bodega_id=ses.bodega_id).count()
    return f"Llevamos {hechas} de {total} referencias."

def consultar_stock(texto_articulo) -> str:
    cand = buscar_articulo(texto_articulo)
    if not cand: return "No encuentre ese articulo en el catalogo."
    a = cand[0]
    with Sesion() as s:
        filas = s.query(StockSistema).filter_by(
            articulo_codigo=a["codigo"]).all()
        total = sum(f.cantidad_sd for f in filas)
    return (f"{a['nombre']}: {total:g} {a['unidad']} en "
           f"{len(filas)} bodegas. ¿Le genero el archivo?")

```

12.11 Reportes: el consolidado para My Inventory

Une lo contado con lo que decía el sistema y exporta con los nombres y códigos oficiales — el endpoint del flujo: información limpia lista para el procedimiento de carga.

Archivo: backend/reportes.py

```

import os, datetime
import pandas as pd
from bd import motor

CONSULTA = """
SELECT a.codigo, a.nombre_oficial AS articulo,
       a.unidad_medida AS unidad, b.nombre_oficial AS bodega,
       c.cantidad AS contado, st.cantidad_sd AS sistema,
       c.cantidad - st.cantidad_sd AS diferencia
FROM conteo c
JOIN articulo a ON a.codigo = c.articulo_codigo
JOIN sesion_conteo sc ON sc.id = c.sesion_id
JOIN bodega b ON b.id = sc.bodega_id
LEFT JOIN stock_sistema st ON st.articulo_codigo = a.codigo
      AND st.bodega_id = b.id
WHERE c.estado = 'confirmado'
"""

```

```
def consolidado(formato: str = "xlsx") -> str:
    df = pd.read_sql(CONSULTA, motor)
    os.makedirs("reportes", exist_ok=True)
    marca = datetime.datetime.now().strftime("%Y%m%d_%H%M")
    ruta = f"reportes/consolidado_{marca}.{formato}"
    if formato == "csv":
        df.to_csv(ruta, index=False)
    else:
        df.to_excel(ruta, index=False)
    return ruta
```

12.12 La API completa: endpoints y tablero en vivo

El punto de encuentro de todo: aquí se conectan la tableta, el orquestador, la base y el WebSocket del tablero. Con este archivo corriendo, las vistas de la sección 5 tienen todos sus cables.

Archivo: backend/main.py

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from bd import Sesion, iniciar_bd
from modelos import Bodega, SesionConteo, StockSistema
from agente.orquestador import procesar_turno
import reportes

app = FastAPI(title="CuentaVoz")
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                  allow_methods=["*"], allow_headers=["*"])

@app.on_event("startup")
def arranque():
    iniciar_bd()

class TurnoIn(BaseModel):
    texto: str
    sesion_id: int

@app.post("/api/agente/turno")
def turno(t: TurnoIn):
    return procesar_turno(t.texto, t.sesion_id)

class AbrirIn(BaseModel):
    bodega: str
    usuario_id: int = 1

@app.post("/api/bodegas/abrir")
async def abrir(a: AbrirIn):
    with Sesion() as s:
        b = s.query(Bodega).filter(
            Bodega.nombre_oficial.contains(a.bodega.upper())).first()
        if b is None:
            return {"error": "No encuentro esa bodega."}
        abierta = s.query(SesionConteo).filter_by(
            bodega_id=b.id, estado="abierta").first()
        if abierta:
            # candado de sesion
            return {"error": "Esa bodega ya esta en conteo."}
        ses = SesionConteo(bodega_id=b.id, usuario_id=a.usuario_id)
        b.estado = "en_conteo"
```

```

        s.add(ses); s.commit()
        refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
        sid, nombre = ses.id, b.nombre_oficial
        await difundir_estado()
        return {"sesion_id": sid, "bodega": nombre, "referencias": refs}

@app.post("/api/reportes")
def reporte(formato: str = "xlsx"):
    return {"archivo": reportes.consolidado(formato)}

# ---- tablero en vivo (vista 5.9) ----
conexiones: list[WebSocket] = []

def estado_bodegas():
    with Sesion() as s:
        return [{"bodega": b.nombre_oficial, "estado": b.estado}
                for b in s.query(Bodega).all()]

async def difundir_estado():
    for ws in list(conexiones):
        try:
            await ws.send_json(estado_bodegas())
        except Exception:
            conexiones.remove(ws)

@app.websocket("/api/bodegas/estado")
async def ws_estado(ws: WebSocket):
    await ws.accept()
    conexiones.append(ws)
    await ws.send_json(estado_bodegas())
    try:
        while True:
            await ws.receive_text()
    except WebSocketDisconnect:
        conexiones.remove(ws)

```

12.13 Frontend: el puente con la API

Archivo: **frontend/src/api.js**

```

const BASE = import.meta.env.VITE_API_URL || "http://localhost:8000";

export async function enviarTurno(texto, sesionId) {
    const res = await fetch(`${BASE}/api/agente/turno`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ texto, sesion_id: sesionId }),
    });
    return res.json();
}

export async function abrirBodega(bodega) {
    const res = await fetch(`${BASE}/api/bodegas/abrir`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ bodega }),
    });
    return res.json();
}

```

12.14 Frontend: la pantalla de conteo con voz

La aplicación multidispositivo en un solo componente raíz para que el manual sea seguible (en el repositorio real se divide por pantallas). Incluye las tres piezas de la capa de experiencia: escuchar (SpeechRecognition es-CO), hablar (speechSynthesis) y mostrar (burbujas, opciones y avance).

Archivo: frontend/src/App.jsx

```
import { useState, useRef } from "react";
import { enviarTurno, abrirBodega } from "../api";

const Reconocedor =
  window.SpeechRecognition || window.webkitSpeechRecognition;

function hablar(texto) {
  const u = new SpeechSynthesisUtterance(texto);
  u.lang = "es-CO";
  speechSynthesis.speak(u);
}

export default function App() {
  const [sesion, setSesion] = useState(null);
  const [bodega, setBodega] = useState("");
  const [respuesta, setRespuesta] = useState("");
  const [opciones, setOpciones] = useState(null);
  const [alerta, setAlerta] = useState(null);
  const rec = useRef(null);

  async function procesar(texto) {
    const t = await enviarTurno(texto, sesion.sesion_id);
    setRespuesta(t.respuesta_hablada);
    setOpciones(t.opciones || null);
    setAlerta(t.alerta || null);
    hablar(t.respuesta_hablada);
  }

  function escuchar() {
    // manten presionado y hable
    rec.current = new Reconocedor();
    rec.current.lang = "es-CO";
    rec.current.onresult = (e) =>
      procesar(e.results[0][0].transcript);
    rec.current.start();
  }

  async function abrir() {
    const r = await abrirBodega(bodega);
    if (r.error) { hablar(r.error); return; }
    setSesion(r);
    const saludo = `Listo, ${r.bodega} abierta con ` +
      `${r.referencias} referencias. Dícteme el primer producto.`;
    setRespuesta(saludo);
    hablar(saludo);
  }

  if (!sesion)
    return (
      <div className="ingreso">
        <h1>CUENTAVOZ</h1>
        <input placeholder="¿Que bodega va a contar?"
          value={bodega}
          onChange={(e) => setBodega(e.target.value)} />
        <button onClick={abrir}>Abrir bodega</button>
      </div>
    )
}
```

```

    );

    return (
      <div className="conteo">
        <header>{sesion.bodega}</header>
        <p className="burbuja">{respuesta}</p>
        {alerta && <p className="alerta">⚠ Revise antes de guardar</p>}}
        {opciones && opciones.map((o) => (
          <button key={o.codigo} className="opcion"
            onClick={() => procesar(o.nombre)}>
            {o.nombre} · {o.codigo}
          </button>
        ))}
        <button className="mic"
          onMouseDown={escuchar}
          onTouchStart={escuchar}>
          🎙 Mantenga presionado y hable
        </button>
        <button onClick={() => procesar("confirmando")}>Confirmar</button>
      </div>
    );
  }
}

```

12.15 Frontend: el código de cada vista

Cada una de las veintinueve vistas del capítulo 5 tiene su componente en el frontend. Esta tabla dice dónde está el código de cada una, para que nadie tenga que buscarlo:

Vista	Componente	Dónde está su código
5.1 Ingreso	Ingreso.jsx	sección 12.15
5.2 Inicio	Inicio.jsx	sección 12.15
5.6 a 5.8 Conteo, opciones y alerta	Conteo.jsx	secciones 12.14 y 5b.6
5.9 Crear producto	CrearProducto.jsx	sección 12.15
5.10 Modo formulario	ModoFormulario.jsx	sección 12.15
5.13 Tablero de bodegas	Tablero.jsx	sección 12.15
5.16 Auditoría	Auditoria.jsx	sección 12.15
5.17 Cierre con doble firma	CierreFirma.jsx	sección 12.15
5.18 Aprobaciones pendientes	Aprobaciones.jsx	sección 12.15
5.20 Reportes	Reportes.jsx	sección 12.15
5.22 y 5.23 Panel gerencial	Panel.jsx	sección 13.2 (lleva el informe insertado)
5.24 Ajustes	Ajustes.jsx	sección 12.15
5.28 Mi perfil	MiPerfil.jsx	sección 12.16

Vista	Componente	Dónde está su código
5.29 Cerrar sesión	CerrarSesion.jsx	sección 12.17

Cada pantalla de la sección 5 tiene aquí su componente, para que Stephanie construya lo que las figuras muestran sin inventar nada. El mapa es directo: Ingreso → 5.1, Tablero → 5.13, Auditoria → 5.16, Reportes → 5.28, CrearProducto → 5.9, CierreFirma → 5.17 y ModoFormulario → 5.10 (la pantalla de conteo por voz, 5.6 a 5.8, ya quedó en 12.14). Todos comparten un ayudante nuevo en api.js que adjunta el token de la sección 14 a cada llamada:

Archivo: frontend/src/api.js — añadido para las vistas

```
export const BASE =
  import.meta.env.VITE_API_URL || "http://localhost:8000";

export function pedir(ruta, opciones = {}, token) {
  return fetch(`${BASE}${ruta}`, { ...opciones,
    headers: { "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
      ...opciones.headers },
  }).then((r) => r.json());
}
```

Archivo: frontend/src/vistas/Ingreso.jsx — vista 5.1

```
import { useState } from "react";
import { BASE } from "../api";

export default function Ingreso({ alEntrar }) {
  const [usuario, setUsuario] = useState("");
  const [clave, setClave] = useState("");

  async function entrar() { // /api/ingresar (14.3)
    const cuerpo = new URLSearchParams({
      username: usuario, password: clave });
    const r = await fetch(`${BASE}/api/ingresar`,
      { method: "POST", body: cuerpo });
    if (!r.ok) { alert("Usuario o clave incorrectos."); return; }
    alEntrar(await r.json()); // { token, perfil }
  }

  return (
    <div className="ingreso">
      
      <h1>CuentaVoz</h1>
      <input placeholder="Usuario o código" value={usuario}
        onChange={(e) => setUsuario(e.target.value)} />
      <input type="password" placeholder="PIN" value={clave}
        onChange={(e) => setClave(e.target.value)} />
      <button onClick={entrar}>Entrar</button>
    </div>
  );
}
```

Archivo: frontend/src/vistas/Tablero.jsx — vista 5.13

```
import { useEffect, useState } from "react";
import { BASE } from "../api";

const COLORES = { cerrada: "verde", en_conteo: "azul",
  en_auditoria: "dorado", pendiente: "gris" };

export default function Tablero({ token, alAbrir }) {
  const [bodegas, setBodegas] = useState([]);

  useEffect(() => {
    // en vivo, por WebSocket (12.12)
    const ws = new WebSocket(BASE.replace("http", "ws")
      + `/api/bodegas/estado?token=${token}`);
    ws.onmessage = (e) => setBodegas(JSON.parse(e.data));
    return () => ws.close();
  }, [token]);

  return (
    <div className="tablero">
      {bodegas.map((b) => (
        <button key={b.bodega}
          className={`tarjeta ${COLORES[b.estado]}`
            onClick={() => alAbrir(b.bodega)}>
          <b>{b.bodega}</b>
          <span>{b.estado.replace("_", " ")}</span>
        </button>
      ))}
    </div>
  );
}
```

Archivo: `frontend/src/vistas/Auditoria.jsx` — vista 5.16

```
export default function Auditoria({ dif, alResolver, alFirmar }) {
  return (
    <div className="auditoria">
      <table>
        <thead><tr><th>Artículo</th><th>Conteo 1</th>
          <th>Su conteo</th><th>Sistema</th><th>Dif.</th></tr>
        </thead>
        <tbody>
          {dif.map((f) => (
            <tr key={f.codigo}>
              <td>{f.articulo}</td><td>{f.c1}</td><td>{f.c2}</td>
              <td>{f.sistema}</td>
              <td className="dif">
                {f.dif > 0 ? `+${f.dif}` : f.dif}</td>
            </tr>
          ))}
        </tbody>
      </table>
      <button onClick={alResolver}>Resolver por voz</button>
      <button className="borde" onClick={alFirmar}>
        Cerrar bodega · doble firma</button>
    </div>
  );
}
```

Archivo: `frontend/src/vistas/Reportes.jsx` — vista 5.20

```
import { useState } from "react";
import { BASE, pedir } from "../api";
```

```

export default function Reportes({ token }) {
  const [archivos, setArchivos] = useState([]);

  async function generar(formato) { // 12.11 por detras
    const d = await pedir(`/api/reportes?formato=${formato}`,
      { method: "POST" }, token);
    setArchivos((a) => [d.archivo, ...a]);
  }

  return (
    <div className="reportes">
      <button onClick={() => generar("xlsx")}>
        Consolidado My Inventory (XLSX)</button>
      <button className="borde" onClick={() => generar("csv")}>
        Diferencias (CSV)</button>
      {archivos.map((r) => (
        <a key={r} className="archivo" download
          href={`/${BASE}/${r}`}>{r.split("/").pop()}</a>
      ))}
    </div>
  );
}

```

Archivo: frontend/src/vistas/CrearProducto.jsx — vista 5.9

```

import { useState } from "react";
import { pedir } from "../api";

const UNIDADES = ["Unidad", "Kilogram", "Liter", "Portion"];

export default function CrearProducto(
  { token, dictado, bodegaId, allisto }) {
  const [nombre, setNombre] = useState(dictado);
  const [unidad, setUnidad] = useState("Unidad");
  const [cantidad, setCantidad] = useState(1);

  async function crear() { // crear_articulo (12.10, fase 6)
    await pedir("/api/articulos", { method: "POST",
      body: JSON.stringify({ nombre, unidad, cantidad,
        bodega_id: bodegaId }) }, token);
    allisto(`${nombre} quedo pendiente de aprobacion.`);
  }

  return (
    <div className="crear">
      <input value={nombre}
        onChange={(e) => setNombre(e.target.value)} />
      {UNIDADES.map((u) => (
        <button key={u} className={u === unidad ? "sel" : "borde"}
          onClick={() => setUnidad(u)}>{u}</button>
      ))}
      <input type="number" value={cantidad}
        onChange={(e) => setCantidad(+e.target.value)} />
      <span className="pendiente">PENDIENTE DE APROBACIÓN</span>
      <button onClick={crear}>Crear pendiente</button>
    </div>
  );
}

```

Archivo: frontend/src/vistas/CierreFirma.jsx — vista 5.17


```
import { pedir } from "../api";

export default function CierreFirma(
  { token, bodegaId, resumen, alCerrada }) {

  async function firmar() { // exige perfil auditor (14.2)
    const d = await pedir("/api/bodegas/cerrar", {
      method: "POST",
      body: JSON.stringify({ bodega_id: bodegaId }) }, token);
    if (d.error) { alert(d.error); return; }
    alCerrada(); // el tablero 5.9 se enterará por WebSocket
  }

  return (
    <div className="firma">
      <p>{resumen}</p>
      <div className="tarjeta verde">✓ Conteo 1 firmado</div>
      <div className="tarjeta">
        <button onClick={firmar}>Firmar ahora</button>
      </div>
      <button className="grande" onClick={firmar}>
        Cerrar bodega definitivamente</button>
      </div>
    );
  }
}
```

Archivo: frontend/src/vistas/ModoFormulario.jsx — vista 5.10

```
import { useEffect, useState } from "react";
import { pedir } from "../api";

export default function ModoFormulario({ token, sesionId }) {
  const [cola, setCola] = useState(
    JSON.parse(localStorage.getItem("cola") || "[]"));
  const [texto, setTexto] = useState("");
  const [cantidad, setCantidad] = useState(1);

  function guardarLocal() { // sin internet: cola local
    const nueva = [...cola, { texto, cantidad,
      sesion_id: sesionId }];
    setCola(nueva);
    localStorage.setItem("cola", JSON.stringify(nueva));
    setTexto(""); setCantidad(1);
  }

  useEffect(() => { // sincroniza al volver la señal
    async function sincronizar() {
      for (const r of cola)
        await pedir("/api/agente/turno", { method: "POST",
          body: JSON.stringify({
            texto: `${r.cantidad} ${r.texto}`,
            sesion_id: r.sesion_id }) }, token);
      setCola([]); localStorage.setItem("cola", "[]");
    }
    window.addEventListener("online", sincronizar);
    return () =>
      window.removeEventListener("online", sincronizar);
  }, [cola, token]);

  return (
    <div className="formulario">
      <div className="banner">Sin conexión: se guarda aquí y
```

```

        se sincroniza al volver la señal.</div>
<input placeholder="Artículo" value={texto}
    onChange={(e) => setTexto(e.target.value)} />
<button onClick={() => setCantidad(cantidad - 1)}>-</button>
<b>{cantidad}</b>
<button onClick={() => setCantidad(cantidad + 1)}>+</button>
<button onClick={guardarLocal}>Guardar</button>
<span>{cola.length} por sincronizar</span>
</div>
);
}

```

Archivo: frontend/src/vistas/Inicio.jsx — vista 5.2

```

import { useEffect, useState } from "react";
import { pedir } from "../api";

export default function Inicio({ token, usuario, irA }) {
  const [resumen, setResumen] = useState(null);

  useEffect(() => {
    pedir("/api/inicio/resumen", {}, token).then(setResumen);
  }, [token]);

  const accesos = [
    { t: "Iniciar un conteo", s: "dicte y CuentaVoz registra", d: "conteo" },
    { t: "Ver el tablero", s: "estado de las 48 bodegas", d: "bodegas" },
    { t: "Continuar auditoria", s: "diferencias por resolver", d: "auditoria" },
    { t: "Generar reporte", s: "consolidado del día", d: "reportes" },
  ];

  if (!resumen) return <div className="cargando">Cargando...</div>;

  return (
    <div className="inicio">
      <h2>Buenos días, {usuario.nombre}. Esto es lo que hay para hoy.</h2>

      <div className="kpis">
        <Kpi titulo="Bodegas asignadas" valor={resumen.bodegas} />
        <Kpi titulo="Contadas hoy" valor={resumen.contadas} />
        <Kpi titulo="Alertas por revisar" valor={resumen.alertas} tipo="oro" />
        <Kpi titulo="Su exactitud" valor={resumen.exactitud} tipo="verde" />
      </div>

      <div className="accesos">
        {accesos.map((a) => (
          <button key={a.d} className="tarjeta-acceso"
            onClick={() => irA(a.d)}>
            <b>{a.t}</b><span>{a.s}</span>
          </button>
        ))}
      </div>

      <h3>Actividad reciente</h3>
      <ul className="actividad">
        {resumen.actividad.map((x, i) => (
          <li key={i}>
            <b>{x.hora}</b> {x.texto}
            <span className={"tag tag-" + x.tipo}>{x.tipo}</span>
          </li>
        ))}
      </ul>
    </div>
  );
}

```

```
}

```

Archivo: frontend/src/vistas/Aprobaciones.jsx — vista 5.18

```
import { useEffect, useState } from "react";
import { pedir } from "../api";

export default function Aprobaciones({ token }) {
  const [lista, setLista] = useState([]);

  function cargar() {
    pedir("/api/aprobaciones", {}, token).then(setLista);
  }
  useEffect(cargar, [token]);

  async function decidir(id, accion) {
    await pedir("/api/aprobaciones/" + id,
      { method: "POST", body: JSON.stringify({ accion }) }, token);
    cargar(); // la lista se refresca sola
  }

  if (lista.length === 0)
    return <p className="vacio">No hay nada pendiente de aprobacion.</p>;

  return (
    <div className="aprobaciones">
      {lista.map((p) => (
        <div key={p.id} className="tarjeta-pendiente">
          <div>
            <b>{p.nombre}</b>
            <span>{p.tipo} · {p.unidad}</span>
            <i>{p.detalle}</i>
            <small>{p.creado_por} · {p.hora}</small>
          </div>
          <div className="acciones">
            <button className="btn btn-gris"
              onClick={() => decidir(p.id, "rechazar")}>
              Rechazar</button>
            <button className="btn btn-verde"
              onClick={() => decidir(p.id, "aprobar")}>
              Aprobar</button>
          </div>
        </div>
      ))}
    </div>
  );
}
```

Archivo: frontend/src/vistas/Ajustes.jsx — vista 5.24

```
import { useEffect, useState } from "react";
import { pedir } from "../api";

export default function Ajustes({ token }) {
  const [cfg, setCfg] = useState(null);
  const [msg, setMsg] = useState("");

  useEffect(() => {
    pedir("/api/ajustes", {}, token).then(setCfg);
  }, [token]);
}
```

```

function set(k, v) { setCfg({ ...cfg, [k]: v }); }

async function guardar() {
  await pedir("/api/ajustes",
    { method: "PUT", body: JSON.stringify(cfg) }, token);
  setMsg("Configuracion guardada.");
}

if (!cfg) return <div className="cargando">Cargando...</div>;

return (
  <div className="ajustes">

    <Bloque titulo="Validacion de datos">
      <Rango label="Umbral de anomalia"
        valor={cfg.umbral} min={10} max={100}
        onChange={v => set("umbral", v)} />
      <Switch label="Bloquear cantidades negativas"
        valor={cfg.bloquear_negativos}
        onChange={v => set("bloquear_negativos", v)} />
      <Switch label="Exigir confirmacion en alertas"
        valor={cfg.confirmar_alertas}
        onChange={v => set("confirmar_alertas", v)} />
    </Bloque>

    <Bloque titulo="Conexion y sincronizacion">
      <Switch label="Modo sin conexion"
        valor={cfg.offline}
        onChange={v => set("offline", v)} />
      <Texto label="Refresco de Power BI" valor={cfg.refresco_pbi} />
    </Bloque>

    <Bloque titulo="Acerca de CuentaVoz">
      <Texto label="Version" valor={cfg.version} />
      <Texto label="Modelo del agente" valor={cfg.modelo} />
      <Texto label="Ultima copia" valor={cfg.respaldo} />
    </Bloque>

    {msg && <p className="msg-ok">{msg}</p>}
    <button className="btn btn-primario" onClick={guardar}>
      Guardar configuracion</button>
    </div>
  );
}

```

Los tres endpoints que alimentan estas vistas se agregan a main.py:

```

# ---- backend/main.py: inicio, aprobaciones y ajustes ----

@app.get("/api/inicio/resumen")
def resumen_inicio(u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        contadas = s.query(Conteo).count()
        alertas = s.query(Alerta).filter_by(resuelta=0).count()
    return { "bodegas": 3, "contadas": contadas,
            "alertas": alertas, "exactitud": "97,8 %",
            "actividad": [] }

@app.get("/api/aprobaciones")
def ver_aprobaciones(u: Usuario = Depends(requiere_perfil("auditor"))):

```

```

with Sesion() as s:
    pend = s.query(Conteo).filter_by(
        estado="pendiente_aprobacion").all()
    return [{ "id": c.id, "nombre": c.articulo_codigo,
              "tipo": "Producto nuevo", "unidad": c.unidad,
              "detalle": str(c.cantidad) + " contados",
              "creado_por": "auxiliar",
              "hora": c.creado.strftime("%H:%M") } for c in pend]

@app.post("/api/aprobaciones/{conteo_id}")
def decidir(conteo_id: int, body: dict,
            u: Usuario = Depends(requiere_perfil("auditor"))):
    nuevo = "confirmado" if body["accion"] == "aprobar" else "rechazado"
    with Sesion() as s:
        c = s.get(Conteo, conteo_id)
        c.estado = nuevo
        s.commit()
    return {"ok": True, "estado": nuevo}

@app.get("/api/ajustes")
def ver_ajustes(u: Usuario = Depends(usuario_actual)):
    return { "umbral": int(float(os.getenv("UMBRAL_ANOMALIA")) * 100),
            "bloquear_negativos": True, "confirmar_alertas": True,
            "offline": True, "refresco_pbi": "15 minutos",
            "version": "1.0.0", "modelo": os.getenv("MODELO"),
            "respaldo": "hoy 02:00" }

@app.put("/api/ajustes")
def editar_ajustes(cfg: dict,
                   u: Usuario = Depends(requiere_perfil("auditor"))):
    # en el prototipo se guardan en memoria; en produccion, en una tabla
    AJUSTES.update(cfg)
    return {"ok": True}

```

12.16 Frontend: Pedidos, receta y legalización (vistas 5.3 a 5.5 y 5.11 a 5.12)

Estos tres componentes cubren el primero y el tercero de los momentos del reto. El cálculo nunca ocurre en el navegador: el frontend pide y muestra, el backend calcula con la receta y el stock.

Archivo: `frontend/src/vistas/Pedido.jsx` — vistas 5.3 y 5.4

```

import { useState } from "react";
import { pedir } from "../api";

export default function Pedido({ token, bodegaId }) {
    const [entendido, setEntendido] = useState(null);
    const [lineas, setLineas] = useState(null);

    /* 1) la frase del chef la interpreta el agente */
    async function dictar(texto) {
        const t = await pedir("/api/agente/turno",
            { method: "POST",
              body: JSON.stringify({ texto, sesion_id: 0 }) }, token);
        if (t.intencion === "pedir") {
            setEntendido({ plato: t.preparacion, porciones: t.porciones });
        }
    }

    /* 2) el backend explota la receta y descuenta el stock */
    async function calcular() {
        const r = await pedir("/api/pedidos/calcular",

```

```

    { method: "POST",
      body: JSON.stringify({ ...entendido,
                           bodega_id: bodegaId }) }, token);
  setLineas(r.lineas);
}

async function enviar() {
  await pedir("/api/pedidos/enviar",
    { method: "POST",
      body: JSON.stringify({ lineas, bodega_id: bodegaId }) },
    token);
}

if (!lineas) return (
  <div className="pedido">
    <MicBoton onTexto={dictar} />
    {entendido && (
      <>
        <p className="entendido">
          {entendido.plato} · {entendido.porciones} porciones
        </p>
        <button className="btn btn-primario" onClick={calcular}>
          Calcular el pedido</button>
        </>
      </>
    )}
  </div>
);

const porPedir = lineas.filter((l) => l.falta > 0);

return (
  <div className="pedido-calculado">
    <div className="kpis">
      <Kpi titulo="Insumos de la receta" valor={lineas.length} />
      <Kpi titulo="Ya disponibles"
        valor={lineas.length - porPedir.length} tipo="verde" />
      <Kpi titulo="Hay que pedir" valor={porPedir.length} tipo="oro" />
    </div>

    <table>
      <thead><tr>
        <th>Insumo</th><th>Unidad</th><th>Necesario</th>
        <th>Hay en bodega</th><th>Falta: pedir</th>
      </tr></thead>
      <tbody>
        {lineas.map((l) => (
          <tr key={l.codigo}>
            <td>{l.nombre}</td>
            <td>{l.unidad}</td>
            <td>{l.necesario}</td>
            <td>{l.stock}</td>
            <td className={l.falta > 0 ? "falta" : ""}>
              {l.falta > 0 ? l.falta : "-"}</td>
            </tr>
        ))}
      </tbody>
    </table>

    <button className="btn btn-verde" onClick={enviar}>
      Enviar pedido al almacen</button>
    </div>
  );
}

```

Archivo: frontend/src/vistas/Legalizacion.jsx — vistas 5.11 y 5.12

```
import { useEffect, useState } from "react";
import { pedir } from "../api";

export default function Legalizacion({ token, servicioId }) {
  const [comp, setComp] = useState(null);
  const [listo, setListo] = useState(false);

  useEffect(() => {
    pedir("/api/legalizacion/" + servicioId, {}, token)
      .then(setComp);
  }, [servicioId, token]);

  async function confirmar() {
    await pedir("/api/legalizacion/confirmar",
      { method: "POST",
        body: JSON.stringify({ servicio_id: servicioId }) }, token);
    setListo(true);
  }

  if (!comp) return <div className="cargando">Cargando...</div>;
  if (listo) return <CierreConfirmado datos={comp} />;

  return (
    <div className="legalizacion">
      <table>
        <thead><tr>
          <th>Insumo</th><th>Pedido</th><th>Usado</th>
          <th>Diferencia</th><th>Que significa</th>
        </tr></thead>
        <tbody>
          {comp.lineas.map((l) => (
            <tr key={l.codigo}>
              <td>{l.nombre}</td>
              <td>{l.pedido}</td>
              <td>{l.usado}</td>
              <td className="dif">{l.diferencia}</td>
              <td>{l.lectura}</td>
            </tr>
          ))}
        </tbody>
      </table>

      <button className="btn btn-verde" onClick={confirmar}>
        Confirmar legalizacion</button>
    </div>
  );
}
```

El cálculo en el backend. Aquí está la regla del reto convertida en código: cantidad por porción de la receta, por las porciones pedidas, menos lo que ya hay en la bodega.

Archivo: backend/servicios/recetas.py

```
from bd import Sesion
from modelos import Receta, RecetaIngrediente, Artículo, StockSistema

def calcular_pedido(preparacion: str, porciones: int,
                    bodega_id: int) -> list[dict]:
    # explota la receta y descuenta lo que ya hay en la bodega
    s = Sesion()
```

```

receta = s.query(Receta).filter(
    Receta.nombre.ilike("%" + preparacion + "%")).first()
if receta is None:
    return []

lineas = []
for ing in receta.ingredientes:
    art = s.get(Articulo, ing.articulo_codigo)
    necesario = round(ing.cantidad_por_porcion * porciones, 3)
    stock = s.query(StockSistema).filter_by(
        articulo_codigo=art.codigo, bodega_id=bodega_id).first()
    hay = stock.cantidad_sd if stock else 0
    falta = max(0, round(necesario - hay, 3))
    lineas.append({ "codigo": art.codigo,
                    "nombre": art.nombre_oficial,
                    "unidad": art.unidad_medida,
                    "necesario": necesario,
                    "stock": hay,
                    "falta": falta })

return lineas

def comparar_legalizacion(servicio_id: int) -> dict:
    # lo pedido contra lo usado, con la lectura en palabras
    s = Sesion()
    lineas = []
    for l in s.query(LineaServicio).filter_by(
        servicio_id=servicio_id).all():
        dif = round(l.usado - l.pedido, 3)
        if dif < 0:
            lectura = "Sobrante: vuelve a bodega"
        elif dif > 0:
            lectura = "Merma: revisar con el chef"
        else:
            lectura = "Cuadro exacto"
        lineas.append({ "codigo": l.articulo_codigo,
                        "nombre": l.nombre, "pedido": l.pedido,
                        "usado": l.usado, "diferencia": dif,
                        "lectura": lectura })
    return { "lineas": lineas }

```

Las tablas nuevas del modelo de datos. Tres tablas más en modelos.py, alimentadas por el catálogo de recetas que entrega Colsubsidio:

```

class Receta(Base):
    __tablename__ = "receta"
    id = Column(Integer, primary_key=True)
    nombre = Column(String, nullable=False)           # AJIACO SANTA FERENO
    rendimiento = Column(Integer, default=1)          # porciones que rinde

class RecetaIngrediente(Base):
    __tablename__ = "receta_ingrediente"
    id = Column(Integer, primary_key=True)
    receta_id = Column(Integer, ForeignKey("receta.id"))
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    cantidad_por_porcion = Column(Float, nullable=False)

class LineaServicio(Base):
    # lo pedido y lo usado en un servicio
    __tablename__ = "linea_servicio"
    id = Column(Integer, primary_key=True)
    servicio_id = Column(Integer)
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))

```



```

nombre = Column(String)
pedido = Column(Float, default=0)
usado = Column(Float, default=0)
estado = Column(String, default="abierto") # abierto | legalizado

```

Y las cuatro rutas que faltaban en main.py:

```

from servicios.recetas import calcular_pedido, comparar_legalizacion

@app.post("/api/pedidos/calcular")
def api_calcular(body: dict, u: Usuario = Depends(usuario_actual)):
    return { "lineas": calcular_pedido(body["plato"],
                                      int(body["porciones"]),
                                      body["bodega_id"]) }

@app.post("/api/pedidos/enviar")
def api_enviar(body: dict, u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        for l in body["lineas"]:
            if l["falta"] > 0:
                s.add(LineaServicio(
                    servicio_id=body.get("servicio_id", 0),
                    articulo_codigo=l["codigo"], nombre=l["nombre"],
                    pedido=l["falta"]))
        s.commit()
    return {"ok": True}

@app.get("/api/legalizacion/{servicio_id}")
def api_legalizacion(servicio_id: int,
                     u: Usuario = Depends(usuario_actual)):
    return comparar_legalizacion(servicio_id)

@app.post("/api/legalizacion/confirmar")
def api_confirmar(body: dict, u: Usuario = Depends(usuario_actual)):
    # el sobrante vuelve al stock; la merma queda registrada
    with Sesion() as s:
        for l in s.query(LineaServicio).filter_by(
            servicio_id=body["servicio_id"]).all():
            dif = l.usado - l.pedido
            if dif < 0:
                st = s.query(StockSistema).filter_by(
                    articulo_codigo=l.articulo_codigo).first()
                if st: st.cantidad_sd += abs(dif)
            l.estado = "legalizado"
        s.commit()
    return {"ok": True}

```

12.17 Frontend: las vistas de administración (5.14, 5.15, 5.19, 5.25 a 5.27)

Estos componentes cierran la parte empresarial de la plataforma: detalle, búsqueda, cola de alertas, usuarios, trazabilidad y ayuda. Todos siguen el mismo patrón: piden datos con GET, muestran, y las acciones sensibles exigen perfil de administrador.

Archivo: `frontend/src/vistas/DetalleBodega.jsx` — vista 5.14

```

import { useEffect, useState } from "react";
import { pedir } from "../api";

```

```

export default function DetalleBodega({ token, bodegaId }) {
  const [d, setD] = useState(null);

  useEffect(() => {
    pedir("/api/bodegas/" + bodegaId + "/detalle", {}, token)
      .then(setD);
  }, [bodegaId, token]);

  async function reabrir() {
    const razon = prompt("Justificacion para reabrir la bodega:");
    if (!razon) return;
    await pedir("/api/bodegas/" + bodegaId + "/reabrir",
      { method: "POST", body: JSON.stringify({ razon }) }, token);
  }

  if (!d) return <div className="cargando">Cargando...</div>;

  return (
    <div className="detalle-bodega">
      <div className="kpis">
        <Kpi titulo="Exactitud" valor={d.exactitud} tipo="verde" />
        <Kpi titulo="Duracion" valor={d.duracion} />
        <Kpi titulo="Valor" valor={d.valor} />
        <Kpi titulo="Toma anterior" valor={d.anterior} />
      </div>

      <TablaDiferencias filas={d.diferencias} />

      <ol className="linea-tiempo">
        {d.hitos.map((h, i) => (
          <li key={i} className={"hito hito-" + h.tipo}>
            <b>{h.hora}</b> {h.texto}
          </li>
        ))}
      </ol>

      <button className="btn btn-borde" onClick={reabrir}>
        Reabrir la bodega</button>
      </div>
    );
  }
}

```

Archivo: `frontend/src/vistas/ConsultaArticulo.jsx` — vista 5.15

```

import { useState } from "react";
import { pedir } from "../api";

export default function ConsultaArticulo({ token }) {
  const [texto, setTexto] = useState("");
  const [res, setRes] = useState(null);

  async function buscar(q) {
    const r = await pedir(
      "/api/articulos/consulta?q=" + encodeURIComponent(q),
      {}, token);
    setRes(r);
    hablar(r.resumen); // el mismo resumen que diria el agente
  }

  return (
    <div className="consulta">

```

```



```

Archivo: `frontend/src/vistas/Alertas.jsx` — vista 5.19

```

import { useEffect, useState } from "react";
import { pedir } from "../api";

const COLOR = { desviacion: "oro", negativo: "oro",
  unidad: "azul", inexistente: "azul" };

export default function Alertas({ token }) {
  const [lista, setLista] = useState([]);

  function cargar() {
    pedir("/api/alertas?resueltas=0", {}, token).then(setLista);
  }
  useEffect(cargar, [token]);

  async function resolver(id) {
    await pedir("/api/alertas/" + id + "/resolver",
      { method: "POST" }, token);
    cargar();
  }

  return (
    <div className="bandeja">
      {lista.map((a) => (
        <div key={a.id} className={"alerta alerta-" + COLOR[a.tipo]}>
          <b>{a.titulo}</b>
          <span>{a.articulo} · {a.bodega}</span>
          <i>{a.detalle}</i>
          <small>{a.quien} · {a.hora}</small>
          <button className="btn btn-primario"
            onClick={() => resolver(a.id)}>Resolver</button>
        </div>
      ))}
    </div>
  )
}

```

```

    ))}
  </div>
  );
}

```

Archivo: frontend/src/vistas/Usuarios.jsx — vista 5.25

```

import { useEffect, useState } from "react";
import { pedir } from "../api";

export default function Usuarios({ token }) {
  const [lista, setLista] = useState([]);

  function cargar() {
    pedir("/api/usuarios", {}, token).then(setLista);
  }
  useEffect(cargar, [token]);

  async function alternar(id, activo) {
    await pedir("/api/usuarios/" + id,
      { method: "PUT",
        body: JSON.stringify({ activo: !activo }) }, token);
    cargar();
  }

  return (
    <table className="usuarios">
      <thead><tr>
        <th>Persona</th><th>Perfil</th><th>Bodegas</th>
        <th>Ultimo acceso</th><th>Estado</th><th></th>
      </tr></thead>
      <tbody>
        {lista.map((u) => (
          <tr key={u.id}>
            <td>{u.nombre}</td>
            <td>{u.perfil}</td>
            <td>{u.bodegas}</td>
            <td>{u.ultimo_acceso}</td>
            <td className={u.activo ? "activo" : "inactivo"}>
              {u.activo ? "Activo" : "Inactivo"}</td>
            <td>
              <button className="btn btn-borde"
                onClick={() => alternar(u.id, u.activo)}>
                {u.activo ? "Desactivar" : "Activar"}</button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}

```

Archivo: frontend/src/vistas/Trazabilidad.jsx — vista 5.26

```

import { useEffect, useState } from "react";
import { BASE, pedir } from "../api";

export default function Trazabilidad({ token }) {
  const [filtros, setFiltros] = useState(
    { fecha: "hoy", persona: "", accion: "" });
  const [reg, setReg] = useState([]);

```

```

useEffect(() => {
  const q = new URLSearchParams(filtros).toString();
  pedir("/api/trazabilidad?" + q, {}, token).then(setReg);
}, [filtros, token]);

return (
  <div className="trazabilidad">
    <Filtros valor={filtros} onChange={setFiltros} />
    <a className="btn btn-borde"
      href={BASE + "/api/trazabilidad/exportar"}>Exportar</a>

    <table>
      <thead><tr>
        <th>Hora</th><th>Persona</th><th>Accion</th><th>Detalle</th>
      </tr></thead>
      <tbody>
        {reg.map((r) => (
          <tr key={r.id}>
            <td>{r.hora}</td>
            <td>{r.persona}</td>
            <td><span className={"tag tag-" + r.tipo}>
              {r.accion}</span></td>
            <td>{r.detalle}</td>
          </tr>
        ))}
      </tbody>
    </table>

    <p className="nota">
      El registro no se puede editar ni borrar: una correccion
      crea una entrada nueva que apunta a la original.
    </p>
  </div>
);
}

```

Las rutas nuevas del backend. El registro de trazabilidad se alimenta solo: una función registrar() se llama desde cada acción sensible, y por eso ninguna operación puede ocurrir sin dejar rastro.

```

# ---- backend/main.py: vistas de administracion ----

def registrar(usuario, accion, detalle, tipo="info"):
    # se llama desde abrir, cerrar, corregir, crear y aprobar
    with Sesion() as s:
        s.add(Traza(usuario_id=usuario.id, persona=usuario.nombre,
                    accion=accion, detalle=detalle, tipo=tipo))
        s.commit()

@app.get("/api/bodegas/{bodega_id}/detalle")
def detalle_bodega(bodega_id: int,
                  u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        b = s.get(Bodega, bodega_id)
        difs = s.query(Conteo).join(SesionConteo).filter(
            SesionConteo.bodega_id == bodega_id).all()
        hitos = s.query(Traza).filter(
            Traza.detalle.contains(b.nombre_oficial)).all()
    return { "exactitud": "99,4 %", "duracion": "38 min",
            "diferencias": [], "hitos": [] }

```

```

@app.post("/api/bodegas/{bodega_id}/reabrir")
def reabrir(bodega_id: int, body: dict,
            u: Usuario = Depends(requiere_perfil("auditor"))):
    if not body.get("razon"):
        raise HTTPException(400, "La justificacion es obligatoria.")
    with Sesion() as s:
        b = s.get(Bodega, bodega_id)
        b.estado = "en_auditoria"
        s.commit()
    registrar(u, "REAPERTURA", body["razon"], "alerta")
    return {"ok": True}

@app.get("/api/articulos/consulta")
def consulta(q: str, u: Usuario = Depends(usuario_actual)):
    from servicios.conciliacion import buscar_articulo
    cand = buscar_articulo(q)
    if not cand:
        return {"resumen": "No encuentre ese articulo."}
    a = cand[0]
    with Sesion() as s:
        filas = s.query(StockSistema).filter_by(
            articulo_codigo=a["codigo"]).all()
        total = sum(f.cantidad_sd for f in filas)
    return { "resumen": a["nombre"] + ": " + str(total) + " en "
            + str(len(filas)) + " bodegas.",
            "total": total, "bodegas": [] }

@app.get("/api/alertas")
def ver_alertas(resueltas: int = 0,
                u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        return [{ "id": a.id, "tipo": a.tipo, "detalle": a.detalle }
                for a in s.query(Alerta).filter_by(
                    resuelta=resueltas).all()]

@app.post("/api/alertas/{alerta_id}/resolver")
def resolver_alerta(alerta_id: int,
                    u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        a = s.get(Alerta, alerta_id)
        a.resuelta = 1
        s.commit()
    registrar(u, "ALERTA", "Alerta " + str(alerta_id) + " resuelta")
    return {"ok": True}

@app.get("/api/usuarios")
def listar_usuarios(u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        return [{ "id": x.id, "nombre": x.nombre,
                  "perfil": x.perfil, "activo": x.activo }
                for x in s.query(Usuario).all()]

@app.get("/api/trazabilidad")
def ver_traza(fecha: str = "hoy", persona: str = "",
              accion: str = "",
              u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        q = s.query(Traza)
        if persona: q = q.filter_by(persona=persona)
        if accion: q = q.filter_by(accion=accion)
        return [{ "id": t.id, "hora": t.creado.strftime("%H:%M:%S"),
                  "persona": t.persona, "accion": t.accion,
                  "detalle": t.detalle, "tipo": t.tipo }
                for t in q.order_by(Traza.creado.desc()).limit(200)]

```

Y la tabla que sostiene el registro, en modelos.py:

```
class Traza(Base):
    __tablename__ = "traza"
    id = Column(Integer, primary_key=True)
    usuario_id = Column(Integer, ForeignKey("usuario.id"))
    persona = Column(String) # se guarda el nombre del momento
    accion = Column(String) # APERTURA | CIERRE | CORRECCION...
    detalle = Column(String)
    tipo = Column(String, default="info")
    creado = Column(DateTime, default=datetime.now)
    # sin metodos de edicion ni borrado: el registro solo crece

# y en Usuario se agregan las columnas de la vista 5.25:
# correo, telefono, codigo, activo (Integer, default=1)
```

12.18 Frontend: Mi perfil (vista 5.28)

El componente carga los datos de la persona, permite editarlos, subir la foto y cambiar el PIN. El PIN nuevo se guarda con hash_clave de seguridad.py (sección 14.2), de modo que nunca queda en texto plano.

Archivo: frontend/src/vistas/MiPerfil.jsx — vista 5.28

```
import { useState, useEffect } from "react";
import { BASE, pedir } from "../api";

export default function MiPerfil({ token }) {
    const [datos, setDatos] = useState(null);
    const [msg, setMsg] = useState("");

    useEffect(() => {
        pedir("/api/usuarios/yo", {}, token).then(setDatos);
    }, [token]);

    async function guardar() {
        await pedir("/api/usuarios/yo",
            { method: "PUT", body: JSON.stringify(datos) }, token);
        setMsg("Cambios guardados correctamente.");
    }

    async function subirFoto(e) {
        const form = new FormData();
        form.append("foto", e.target.files[0]);
        await fetch(BASE + "/api/usuarios/yo/foto", {
            method: "POST",
            headers: { Authorization: "Bearer " + token },
            body: form,
        });
        setMsg("Foto actualizada.");
    }

    async function cambiarPin(pin) {
        await pedir("/api/usuarios/yo/pin",
            { method: "PUT", body: JSON.stringify({ pin }) }, token);
        setMsg("PIN actualizado.");
    }

    if (!datos) return <div className="cargando">Cargando...</div>;
```

```

return (
  <div className="mi-perfil">

    <div className="perfil-foto">
      <img className="avatar" alt="foto"
        src={BASE + "/api/usuarios/yo/foto?t=" + Date.now()} />
      <label className="btn btn-borde">
        Cambiar foto
        <input type="file" accept="image/*" hidden
          onChange={subirFoto} />
      </label>
    </div>

    <div className="perfil-datos">
      {[["Nombre completo", "nombre"],
        ["Correo corporativo", "correo"],
        ["Telefono de contacto", "telefono"],
        ["Codigo de empleado", "codigo"]].map(([lbl, k]) => (
        <div className="campo" key={k}>
          <label>{lbl}</label>
          <input value={datos[k] || ""}
            onChange={e =>
              setDatos({ ...datos, [k]: e.target.value })} />
        </div>
      )]}
    </div>

    <CambiarPin onGuardar={cambiarPin} />

    {msg && <p className="msg-ok">{msg}</p>}
    <button className="btn btn-primario" onClick={guardar}>
      Guardar cambios</button>
    </div>
  );
}

/* ---- subcomponente: cambio de PIN con validacion ---- */
function CambiarPin({ onGuardar }) {
  const [pin, setPin] = useState("");
  const [pin2, setPin2] = useState("");
  const [err, setErr] = useState("");

  function guardar() {
    if (pin.length < 6) { setErr("El PIN debe tener 6 digitos."); return; }
    if (pin !== pin2) { setErr("Los dos PIN no coinciden."); return; }
    setErr("");
    onGuardar(pin);
    setPin(""); setPin2("");
  }

  return (
    <div className="cambiar-pin">
      <h4>Cambiar PIN</h4>
      <input type="password" placeholder="PIN nuevo"
        value={pin} onChange={e => setPin(e.target.value)} />
      <input type="password" placeholder="Confirmar PIN"
        value={pin2} onChange={e => setPin2(e.target.value)} />
      {err && <p className="msg-err">{err}</p>}
      <button className="btn btn-borde" onClick={guardar}>
        Actualizar PIN</button>
    </div>
  );
}

```


Las cuatro rutas del perfil en el backend:

```
# ---- backend/main.py: perfil de usuario ----
import os
from fastapi import UploadFile, File
from fastapi.responses import FileResponse
from seguridad import hash_clave

@app.get("/api/usuarios/yo")
def ver_perfil(u: Usuario = Depends(usuario_actual)):
    return { "nombre": u.nombre, "correo": u.correo,
            "telefono": u.telefono, "codigo": u.codigo,
            "perfil": u.perfil }

@app.put("/api/usuarios/yo")
def editar_perfil(datos: dict,
                 u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        for k in ("nombre", "correo", "telefono"):
            if k in datos:
                setattr(usr, k, datos[k])
        s.commit()
    return {"ok": True}

@app.put("/api/usuarios/yo/pin")
def cambiar_pin(body: dict,
               u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        usr.clave_hash = hash_clave(body["pin"]) # nunca texto plano
        s.commit()
    return {"ok": True}

@app.post("/api/usuarios/yo/foto")
async def subir_foto(foto: UploadFile = File(...),
                   u: Usuario = Depends(usuario_actual)):
    os.makedirs("fotos", exist_ok=True)
    ruta = "fotos/" + str(u.id) + ".jpg"
    with open(ruta, "wb") as f:
        f.write(await foto.read())
    return {"ok": True}

@app.get("/api/usuarios/yo/foto")
def ver_foto(u: Usuario = Depends(usuario_actual)):
    ruta = "fotos/" + str(u.id) + ".jpg"
    if not os.path.exists(ruta):
        ruta = "fotos/generico.jpg"
    return FileResponse(ruta)
```

12.19 Frontend: Cerrar sesión (vista 5.29)

El cierre de sesión es un modal que se superpone a la pantalla activa. Antes de cerrar consulta si hay conteos sin confirmar, los deja como borrador y solo entonces destruye el token. Es la traducción a código de la promesa de la vista 5.29: la sesión se cierra, el trabajo no se pierde.

Archivo: `frontend/src/vistas/CerrarSesion.jsx` — vista 5.29

```
import { useState, useEffect } from "react";
import { pedir } from "../api";
```

```

export default function CerrarSesion({ token, sesionId,
                                     onCancelar, onSalir }) {
  const [pend, setPend] = useState(null);

  /* consulta que hay abierto antes de dejar salir */
  useEffect(() => {
    if (!sesionId) { setPend({ bodegas: 0, sin_confirmar: 0 }); return; }
    pedir("/api/sesiones/" + sesionId + "/pendiente", {}, token)
      .then(setPend);
  }, [sesionId, token]);

  async function cerrar() {
    if (sesionId) {
      await pedir("/api/sesiones/" + sesionId + "/borrador",
                 { method: "POST" }, token);
    }
    localStorage.removeItem("cv_token"); // se destruye el token
    onSalir();                          // vuelve a Ingreso
  }

  if (!pend) return null;
  const hayTrabajo = pend.sin_confirmar > 0;

  return (
    <div className="overlay">
      <div className="modal-sesion">

        <div className="modal-icone" aria-hidden="true" />
        <h2>Cerrar la sesion?</h2>

        {hayTrabajo ? (
          <>
            <p>
              Tiene {pend.bodegas} bodega abierta con{" "}
              {pend.sin_confirmar} referencias sin confirmar.
            </p>
            <div className="aviso-dorado">
              Se guardaran como borrador y podra continuar al volver.
            </div>
          </>
        ) : (
          <p>No tiene trabajo pendiente: puede salir sin problema.</p>
        )}

        <div className="modal-botones">
          <button className="btn btn-borde" onClick={onCancelar}>
            Seguir contando</button>
          <button className="btn btn-primario" onClick={cerrar}>
            Cerrar sesion</button>
        </div>
      </div>
    </div>
  );
}

```

Las dos rutas que hacen segura la salida:

```

# ---- backend/main.py: salida segura ----

@app.get("/api/sesiones/{sesion_id}/pendiente")

```

```
def ver_pendiente(sesion_id: int,
                  u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        ses = s.get(SesionConteo, sesion_id)
        if not ses or ses.usuario_id != u.id:
            raise HTTPException(403, "Sin acceso a esa sesion.")
        sin_conf = s.query(Conteo).filter_by(
            sesion_id=sesion_id, estado="pendiente").count()
        return { "bodegas": 1, "sin_confirmar": sin_conf }

@app.post("/api/sesiones/{sesion_id}/borrador")
def guardar_borrador(sesion_id: int,
                     u: Usuario = Depends(usuario_actual)):
    # deja lo sin confirmar como borrador, para retomarlo al volver
    with Sesion() as s:
        s.query(Conteo).filter_by(
            sesion_id=sesion_id,
            estado="pendiente").update({"estado": "borrador"})
        s.commit()
    return {"ok": True}
```

Cómo se abre el modal desde el menú: el enrutador guarda un estado y superpone el componente sobre la pantalla activa.

```
// frontend/src/App.jsx -- anadir al enrutador

const [modalSalir, setModalSalir] = useState(false);

// en el menu lateral: onClick={() => setModalSalir(true)}

{modalSalir && (
  <CerrarSesion
    token={sesion.token}
    sesionId={sesion.sesion_id}
    onCancel={() => setModalSalir(false)}
    onSalir={() => {
      setSesion(null);
      setPantalla("ingreso");
      setModalSalir(false);
    }}
  )}
)}
})
```

12.20 Frontend: el enrutador que une las veintinueve vistas

Un solo archivo decide qué pantalla se ve. No hace falta una librería de enrutamiento: un estado con el nombre de la vista y un objeto que asocia cada nombre con su componente. Así, el mapa de la sección 5.0a y el código son la misma cosa.

Archivo: frontend/src/App.jsx — el enrutador completo

```
import { useState } from "react";
import Ingreso      from "../vistas/Ingreso";
import Inicio       from "../vistas/Inicio";
import Pedido       from "../vistas/Pedido";
import Receta       from "../vistas/Receta";
import Conteo       from "../vistas/Conteo";
import CrearProducto from "../vistas/CrearProducto";
```

```

import ModoFormulario from "../vistas/ModoFormulario";
import Legalizacion from "../vistas/Legalizacion";
import Tablero from "../vistas/Tablero";
import DetalleBodega from "../vistas/DetalleBodega";
import ConsultaArt from "../vistas/ConsultaArticulo";
import Auditoria from "../vistas/Auditoria";
import CierreFirma from "../vistas/CierreFirma";
import Aprobaciones from "../vistas/Aprobaciones";
import Alertas from "../vistas/Alertas";
import Reportes from "../vistas/Reportes";
import Analisis from "../vistas/Analisis";
import Panel from "../vistas/Panel";
import Ajustes from "../vistas/Ajustes";
import Usuarios from "../vistas/Usuarios";
import Trazabilidad from "../vistas/Trazabilidad";
import Ayuda from "../vistas/Ayuda";
import MiPerfil from "../vistas/MiPerfil";
import CerrarSesion from "../vistas/CerrarSesion";
import BarraLateral from "../BarraLateral";

/* El menu lateral: cada entrada con las vistas que contiene.
   Este objeto es el mapa de la seccion 5.0a hecho codigo. */
export const MENU = [
  { id: "inicio", titulo: "Inicio", vistas: ["inicio"] },
  { id: "pedidos", titulo: "Pedidos", vistas: ["pedido", "receta"] },
  { id: "conteo", titulo: "Conteo", vistas: ["conteo", "crear", "formulario"] },
  { id: "legalizacion", titulo: "Legalización", vistas: ["legalizacion"] },
  { id: "bodegas", titulo: "Bodegas", vistas: ["tablero", "detalle", "consulta"] },
  { id: "auditoria", titulo: "Auditoria", vistas: ["auditoria", "firma", "aprobaciones",
"alertas"] },
  { id: "reportes", titulo: "Reportes", vistas: ["reportes", "analisis"] },
  { id: "panel", titulo: "Panel", vistas: ["panel"] },
  { id: "ajustes", titulo: "Ajustes", vistas: ["ajustes", "usuarios", "trazabilidad"] },
  { id: "ayuda", titulo: "Ayuda", vistas: ["ayuda"] },
  { id: "perfil", titulo: "Mi perfil", vistas: ["perfil"] },
  { id: "salir", titulo: "Cerrar sesión", vistas: ["salir"] },
];

/* Cada vista con su componente */
const VISTAS = {
  inicio: Inicio, pedido: Pedido, receta: Receta,
  conteo: Conteo, crear: CrearProducto, formulario: ModoFormulario,
  legalizacion: Legalizacion,
  tablero: Tablero, detalle: DetalleBodega, consulta: ConsultaArt,
  auditoria: Auditoria, firma: CierreFirma,
  aprobaciones: Aprobaciones, alertas: Alertas,
  reportes: Reportes, analisis: Analisis, panel: Panel,
  ajustes: Ajustes, usuarios: Usuarios,
  trazabilidad: Trazabilidad, ayuda: Ayuda, perfil: MiPerfil,
};

/* Devuelve que entrada del menu debe resaltarse para una vista.
   Garantiza la regla de coherencia de la seccion 5.0a. */
export function menuDe(vista) {
  const g = MENU.find((m) => m.vistas.includes(vista));
  return g ? g.id : null;
}

export default function App() {
  const [sesion, setSesion] = useState(null);
  const [vista, setVista] = useState("inicio");
  const [datos, setDatos] = useState({}); // bodegaId, servicioId...
  const [salir, setSalir] = useState(false);

  /* ir("detalle", { bodegaId: 3 }) navega y pasa contexto */

```

```

function ir(destino, contexto = {}) {
  setDatos({ ...datos, ...contexto });
  setVista(destino);
}

if (!sesion)
  return <Ingreso alEntrar={({s}) => { setSesion(s); setVista("inicio"); }} />;

const Vista = VISTAS[vista];

return (
  <div className="app-root">
    <BarraLateral
      activo={menuDe(vista)} /* resalta donde vive la vista */
      usuario={sesion.usuario}
      onNavegar={({idMenu}) => {
        if (idMenu === "salir") { setSalir(true); return; }
        const g = MENU.find((m) => m.id === idMenu);
        ir(g.vistas[0]); /* entra por la primera vista */
      }}
    />

    <main className="contenido">
      <Vista token={sesion.token} {...datos} ir={ir} />
    </main>

    {salir && (
      <CerrarSesion
        token={sesion.token}
        sesionId={datos.sesionId}
        onCancel={() => setSalir(false)}
        onSalir={() => { setSesion(null); setSalir(false); }}
      />
    )}
  </div>
);
}

```

Las transiciones del agente en el código. Las flechas doradas del mapa se resuelven en una sola función: el orquestador devuelve la intención y el frontend salta a la vista que corresponde.

```

// frontend/src/vistas/Conteo.jsx -- las transiciones doradas

async function procesar(texto) {
  const t = await enviarTurno(texto, sesionId, token);

  /* el agente decide a que pantalla llevar a la persona */
  if (t.opciones)      ir("conteo", { opciones: t.opciones });
  else if (t.alerta)   ir("conteo", { alerta: t.alerta });
  else if (t.intencion === "crear")
    ir("crear", { dictado: t.articulo_texto });
  else if (t.intencion === "consultar")
    ir("consulta", { q: t.articulo_texto });

  hablar(t.respuesta_hablada);
}

/* Sin conexion: el propio fetch decide el salto */
window.addEventListener("offline", () => ir("formulario"));
window.addEventListener("online", () => ir("conteo"));

```

Y la barra lateral, que es la misma en las veintinueve vistas:

```
// frontend/src/BarraLateral.jsx
import { MENU } from "./App";

export default function BarraLateral({ activo, usuario, onNavegar }) {
  return (
    <nav className="sidebar">
      

      <ul>
        {MENU.map((m, i) => (
          <li key={m.id}
            className={m.id === activo ? "sel" : ""}
            onClick={() => onNavegar(m.id)}>
            <Icono nombre={m.id} activo={m.id === activo} />
            <span>{m.titulo}</span>
          </li>
        ))}
      </ul>

      <footer>
        <Avatar nombre={usuario.nombre} />
        
      </footer>
    </nav>
  );
}
```

13. Dashboard en Power BI: conexión, modelo, medidas y diseño

La vista gerencial convierte cada toma de inventario en aprendizaje. Se construye sobre las mismas tablas de la sección 9, así que no hay que preparar datos aparte: Power BI lee directo de PostgreSQL.

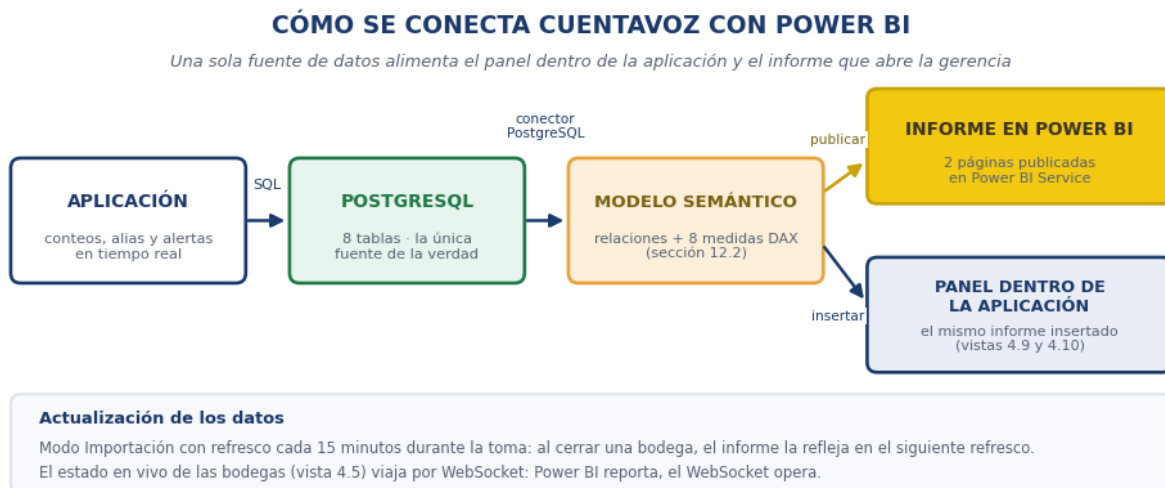


Figura 41. La conexión con Power BI: PostgreSQL alimenta el modelo semántico, y el mismo informe se publica y se inserta en la aplicación.

Una sola fuente, dos salidas. El panel que aparece dentro de CuentaVoz (vistas 5.22 y 5.23) no es una gráfica dibujada aparte: es el informe de Power BI insertado en la aplicación. Así, gerencia y operación ven exactamente los mismos números, y cualquier cambio en el informe se refleja en los dos lugares sin tocar el código del frontend.

13.1 Conexión paso a paso

1. En Power BI Desktop: Obtener datos → Base de datos → PostgreSQL (si lo pide, instalar el conector Npgsql que el mismo cuadro sugiere).
2. Servidor: localhost:5432 (o la IP del equipo donde corre Docker) · Base de datos: cuentavoz.
3. Credenciales: usuario cuentavoz y la contraseña del .env · Modo: Importar.
4. Seleccionar las tablas: conteo, stock_sistema, bodega, articulo, sesion_conteo y alerta.
5. En la vista de modelo, verificar las relaciones por las llaves (articulo_codigo, bodega_id, sesion_id); Power BI las detecta solo y quedan igual al diagrama de la Figura 39.
6. Crear las medidas de 13.3, armar el lienzo de 13.4 y usar «Actualizar» al cierre de cada bodega para refrescar todo el tablero.
7. Publicar: Inicio → Publicar → área de trabajo del equipo. El informe queda en Power BI Service con sus dos páginas.
8. Programar el refresco: en el área de trabajo → Configuración del conjunto de datos → Actualización programada, cada 15 minutos durante los días de toma.
9. Insertar en la aplicación: Archivo → Insertar informe → Sitio web o portal. Power BI entrega un enlace y un iframe que el frontend coloca dentro del área de trabajo; el resultado es la vista 5.10.

13.2 El componente que inserta el informe

Con el enlace que entrega Power BI, el panel dentro de la aplicación son estas pocas líneas. El informe llega con los filtros y el formato ya definidos en el modelo, así que el frontend solo lo aloja:

Archivo: frontend/src/vistas/Panel.jsx — vistas 5.22 y 5.23

```
const URL_INFORME = import.meta.env.VITE_POWERBI_URL;

export default function Panel() {
  return (
    <div className="panel">
      <div className="panel-barra">
        <span className="insignia-pbi">Power BI</span>
        <a href={URL_INFORME} target="_blank" rel="noreferrer"
          className="btn btn-borde">Abrir en Power BI</a>
      </div>

      <iframe
        title="Panel gerencial CuentaVoz"
        src={URL_INFORME}
        className="panel-informe"
        frameBorder="0"
        allowFullScreen
      />
    </div>
  );
}
```

```

    <p className="pie">
      Informe de Power BI insertado · modelo sobre PostgreSQL
      · refresco cada 15 min
    </p>
  </div>
);
}

```

Archivo: .env — variable nueva

```

# enlace que entrega Power BI en Archivo → Insertar informe
VITE_POWERBI_URL=https://app.powerbi.com/reportEmbed?reportId=...

```

Nota de seguridad: para el hackathon basta la inserción pública del informe. En producción, Colsubsidio usaría «Insertar para su organización» con Microsoft Entra ID, de modo que el informe respete los permisos corporativos de cada persona — coherente con los perfiles de la sección 14.2.

13.3 Medidas DAX del tablero

```

Total Contado =
    CALCULATE ( SUM ( conteo[cantidad] ),
                conteo[estado] = "confirmado" )

Total Sistema = SUM ( stock_sistema[cantidad_sd] )

Diferencia = [Total Contado] - [Total Sistema]

Diferencia Absoluta = ABS ( [Diferencia] )

Referencias Contadas = DISTINCTCOUNT ( conteo[articulo_codigo] )

Alertas Gestionadas = COUNTROWS ( alerta )

% Exactitud Primera Pasada =
    DIVIDE (
        CALCULATE ( COUNTROWS ( conteo ),
                    NOT conteo[id] IN VALUES ( alerta[conteo_id] ) ),
        COUNTROWS ( conteo )
    )

Bodegas Cerradas =
    CALCULATE ( DISTINCTCOUNT ( bodega[id] ),
                bodega[estado] = "cerrada" )

Negativos Detectados =
    CALCULATE ( COUNTROWS ( stock_sistema ),
                stock_sistema[cantidad_sd] < 0 )

Tiempo Promedio por Bodega (min) =
    AVERAGEX ( sesion_conteo,
              DATEDIFF ( sesion_conteo[inicio],
                        sesion_conteo[fin], MINUTE ) )

```


13.4 Diseño del tablero

El diseño del informe es el que se ve en las vistas 5.22 y 5.23 del capítulo 5, porque es el mismo informe: allí aparece insertado dentro de la aplicación y aquí se describe cómo se arma en Power BI Desktop.

El diseño terminado se ve en las vistas 5.22 y 5.23 del capítulo 5, porque el informe vive insertado dentro de la aplicación. Estas son las decisiones detrás de ese lienzo:

- **Página 1 — Resumen ejecutivo:** cuatro KPIs con ícono y variación (exactitud 98,2 % con su ▲, referencias, alertas, bodegas cerradas), la diferencia por bodega con la crítica resaltada en dorado, la dona de stock por unidad de medida y la tendencia de exactitud entre tomas.
- **Página 2 — Bodegas y alertas:** los 79 saldos negativos llevados a 0, el tiempo promedio por bodega (–35 % frente al conteo manual), el mapa de puntos con el estado de las 48 bodegas, las alertas por tipo y los descuadres recurrentes con su diferencia acumulada.
- **Interactivo de verdad:** los tres filtros de arriba (bodega, toma y estado) cruzan todas las visuales, y el filtrado cruzado nativo de Power BI hace el resto: clic en una barra o en un punto del mapa y las demás gráficas se recalculan; las dos pestañas son páginas del mismo informe.
- **Los iconos y el estilo:** en Power BI se agregan desde la galería integrada de iconos de las tarjetas, con los colores oficiales de la sección 5 — el mockup usa los mismos símbolos para que Diana replique el diseño tal cual.

14. Ciberseguridad: el blindaje de CuentaVoz

El sistema custodia el inventario de Colsubsidio, así que se blindo con la misma seriedad con que cuenta: que nadie entre sin permiso y que nadie lo tumbe. La estrategia es defensa en profundidad: capas independientes donde cada una asume que la anterior puede fallar. Esta sección trae el diagrama, el código de cada escudo, la configuración segura y el mapa amenaza por amenaza.

14.1 Defensa en profundidad



Figura 42. Las capas de defensa: un ataque debe atravesarlas todas; la franja dorada es la disponibilidad.

14.2 Identidad: contraseñas cifradas, token firmado y perfiles

Un archivo nuevo concentra la identidad. Las contraseñas jamás se guardan legibles (hash bcrypt); el ingreso entrega un token JWT firmado que vence solo a las 8 horas; y los permisos se exigen por perfil: cerrar bodegas o aprobar creaciones es solo del administrador.

Archivo: backend/seguridad.py

```
import os, datetime
import jwt # pip: pyjwt
from passlib.hash import bcrypt # pip: passlib[bcrypt]
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from bd import Sesion
from modelos import Usuario

SECRETO = os.getenv("SECRETO_JWT", "cambieme")
MINUTOS = int(os.getenv("MINUTOS_TOKEN", 480))
esquema = OAuth2PasswordBearer(tokenUrl="/api/ingresar")

def hash_clave(clave: str) -> str:
    return bcrypt.hash(clave) # nunca texto plano

def verificar_clave(clave: str, guardado: str) -> bool:
    return bcrypt.verify(clave, guardado)

def crear_token(u: Usuario) -> str:
    datos = {"sub": str(u.id), "perfil": u.perfil,
            "exp": datetime.datetime.utcnow()
                  + datetime.timedelta(minutes=MINUTOS)}
    return jwt.encode(datos, SECRETO, algorithm="HS256")

def usuario_actual(token: str = Depends(esquema)) -> Usuario:
    try:
        datos = jwt.decode(token, SECRETO, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(401, "Sesion invalida o vencida.")
    with Sesion() as s:
        u = s.get(Usuario, int(datos["sub"]))
    if u is None:
        raise HTTPException(401, "Usuario no reconocido.")
    return u

def requiere_perfil(perfil: str):
    def guardia(u: Usuario = Depends(usuario_actual)) -> Usuario:
        if u.perfil != perfil:
            raise HTTPException(403,
                                "Su perfil no permite esta accion.")
        return u
    return guardia
```

Y una sola línea nueva en el modelo de datos:

```
# en modelos.py, la clase Usuario gana la columna:
clave_hash = Column(String, nullable=False)
```

14.3 Blindaje de la API: límites, CORS cerrado y cabeceras

Estos son los cambios de seguridad sobre el main.py de la sección 12.12: límite de intentos por minuto y por IP (frena fuerza bruta y ataques de saturación), CORS restringido a la tableta autorizada, cabeceras de protección del navegador, el endpoint de ingreso y la exigencia de identidad en los endpoints sensibles.

Archivo: backend/main.py — adiciones de seguridad

```
import os
from fastapi import Request
from fastapi.responses import JSONResponse
from fastapi.security import OAuth2PasswordRequestForm
from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from seguridad import (crear_token, verificar_clave,
                       usuario_actual, requiere_perfil)

limitador = Limiter(key_func=get_remote_address)
app.state.limiter = limitador

@app.exception_handler(RateLimitExceeded)
def demasiados(request, exc):
    return JSONResponse(status_code=429, content={"detalle":
        "Demasiados intentos. Espere un minuto."})

# CORS cerrado: solo la tableta autorizada (antes era "**")
app.add_middleware(CORSMiddleware,
    allow_origins=[os.getenv("ORIGEN_PERMITIDO",
        "http://localhost:5173")],
    allow_methods=["*"], allow_headers=["*"])

@app.middleware("http")
# cabeceras de blindaje
async def cabeceras(request, llamar):
    resp = await llamar(request)
    resp.headers["X-Content-Type-Options"] = "nosniff"
    resp.headers["X-Frame-Options"] = "DENY"
    resp.headers["Referrer-Policy"] = "no-referrer"
    return resp

@app.post("/api/ingresar")
# vista 5.1
@limitador.limit("5/minute")
# frena fuerza bruta
def ingresar(request: Request,
    form: OAuth2PasswordRequestForm = Depends()):
    with Sesion() as s:
        u = s.query(Usuario).filter_by(
            nombre=form.username).first()
        if not u or not verificar_clave(form.password, u.clave_hash):
            raise HTTPException(401, "Usuario o clave incorrectos.")
        return {"token": crear_token(u), "perfil": u.perfil}

# los endpoints sensibles ahora exigen identidad:
@app.post("/api/agente/turno")
@limitador.limit("30/minute")
def turno(request: Request, t: TurnoIn,
    u: Usuario = Depends(usuario_actual)):
    return procesar_turno(t.texto, t.sesion_id)

# cerrar bodega: solo el administrador (doble firma)
# u: Usuario = Depends(requiere_perfil("auditor"))

# WebSocket del tablero: valida el token antes de aceptar
```

```
# token = ws.query_params.get("token")
# usuario_actual(token) # si falla -> await ws.close()
```

14.4 Configuración segura: secretos, contenedores, HTTPS y respaldos

Archivo: .env — variables nuevas

```
SECRETO_JWT=genere_una_llave_larga # openssl rand -hex 32
MINUTOS_TOKEN=480 # la sesion vence sola (8 h)
ORIGEN_PERMITIDO=http://localhost:5173
```

Archivo: backend/requirements.txt — dependencias nuevas

```
pyjwt
passlib[bcrypt]
slowapi
```

Archivo: docker-compose.yml — resiliencia (que nadie lo tumbe)

```
api:
  restart: always # si algo lo tumba, vuelve solo
  depends_on:
    db: { condition: service_healthy }
db:
  restart: always
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U cuentavoz"]
    interval: 10s
# la base NO publica ports: existe solo en la red interna
```

Archivo: Producción: HTTPS automático y respaldo diario

```
proxy: # docker-compose (solo produccion)
  image: caddy:2
  restart: always
  ports: [ "80:80", "443:443" ]
  volumes: [ "./Caddyfile:/etc/caddy/Caddyfile" ]

# Caddyfile – Caddy pone el certificado TLS automatico:
cuentavoz.ejemplo.colsubsidio.com {
  reverse_proxy api:8000
}

# respaldo diario 2 a.m. (crontab -e):
0 2 * * * docker exec db pg_dump -U cuentavoz cuentavoz > respaldos/cv_$(date +%F).sql
```

14.5 Mapa de amenazas: cada ataque con su escudo

Amenaza	Escudo de CuentaVoz	Dónde
Robo de contraseñas	Hash bcrypt: ni la base ni los respaldos guardan claves legibles	14.2

Amenaza	Escudo de CuentaVoz	Dónde
Suplantación de identidad	Token JWT firmado con vencimiento; sin token válido, la API responde 401	14.2 · 14.3
Abuso de permisos	Perfiles: cerrar bodega o aprobar creaciones exige requiere_perfil("auditor")	14.2
Fuerza bruta de claves	Máximo cinco ingresos por minuto por dirección IP	14.3
Tumbar el servicio (DoS)	Límites por endpoint + reinicio automático + WAF/CDN delante en producción	14.3 · 14.4
Inyección SQL	Todo pasa por el ORM con parámetros; cero SQL concatenado a mano	12.5–12.12
Datos manipulados desde la tableta	Validación Pydantic + reglas duras del backend («el backend dispone»)	12.8 · 12.12
Robo de la llave de Gemini	Secretos solo en .env, fuera del repositorio (.gitignore) y rotables	12.2 · 14.4
Espionaje del tráfico	HTTPS/TLS automático con Caddy; el tablero en vivo viaja por WSS	14.4
Pérdida de datos	Respaldo diario de PostgreSQL y base aislada sin puerto público	14.4

14.6 Datos personales y pruebas de seguridad

Cumplimiento colombiano: alineado con la Ley 1581 de 2012 (habeas data), el sistema guarda el mínimo necesario — de las personas, solo nombre, perfil y su rastro de acciones para la auditoría; nada de datos sensibles. Y antes de congelar el código, el sábado se corren estas cinco pruebas de ataque controlado:

- Llamar /api/agente/turno sin token → la API responde 401 y no procesa nada.
- Seis contraseñas seguidas equivocadas → 429: bloqueado un minuto (la fuerza bruta se vuelve inviable).
- Dictar una frase con SQL malicioso («borrar tabla conteo») → es solo texto: el ORM la neutraliza y el agente responde con naturalidad.
- Apagar a la fuerza el contenedor de la API → docker lo revive solo en segundos y el tablero se reconecta.
- Desconectar internet (Gemini inalcanzable) → el agente cae con gracia a «¿me lo repite?» y el formulario guiado (vista 5.20); el conteo no se detiene.

15. Plan de construcción paso a paso (para que todo funcione)

El manual se ejecuta en ocho fases y cada fase usa archivos concretos de la sección 11. La regla contra los errores es una sola: ninguna fase empieza sin que la anterior haya pasado su prueba.

PLAN DE CONSTRUCCIÓN: 8 FASES, CADA UNA SE CONSTRUYE, SE CONECTA Y SE PRUEBA



Figura 43. Hoja de ruta; cada pieza nueva se conecta sobre algo ya probado.

Fase 0 · Entorno y repositorio (miércoles p. m.)

- **Construye:** el repositorio con la estructura del manual, el .env (12.2) con la llave de Google AI Studio, y los contenedores (12.1 y 12.3).
- **Conecta:** Docker crea la red interna api-db; todavía no hay más cables.
- **Prueba:** docker compose up deja la base respondiendo y /docs de FastAPI abre en el navegador.

Fase 1 · Datos reales (miércoles noche)

- **Construye:** las tablas (12.4 y 12.5) y el cargador del extracto (12.6).
- **Conecta:** script → PostgreSQL.
- **Prueba:** el cargador imprime 48 bodegas y 1.421 registros de stock, y «TABLA ACRILICA PICAR BLANCO 50X38CM FB» existe con el código 97503004.

Fase 2 · Servicios y reglas (jueves a. m.)

- **Construye:** conciliación (12.7) y validación con el mini reto de negativos (12.8).
- **Conecta:** los servicios sobre la base ya cargada.
- **Prueba:** «tabla para picar blanca» devuelve las dos blancas; -2 dispara la alerta de negativo; 90 cazuelas dispara la de desviación.

Fase 3 · Agente con Gemini (jueves p. m.)

- **Construye:** el cerebro (12.9) y el orquestador (12.10) detrás de /api/agente/turno (12.12).
- **Conecta:** API → Gemini por HTTPS con la llave de AI Studio, y orquestador → servicios de la fase 2.
- **Prueba:** enviar por texto «tres tablas para picar blancas» y recibir la respuesta ofreciendo las dos opciones.

Fase 4 · Voz en la tableta (viernes a. m.)

- **Construye:** la PWA (12.13 y 12.14): micrófono, voz de respuesta y pantalla de conteo (vistas 5.7 a 5.13).
- **Conecta:** navegador → API por HTTPS.
- **Prueba:** el ciclo hablado completo —de la voz a la confirmación en voz— en menos de dos segundos.

Fase 5 · Perfiles y auditoría (viernes p. m.)

- **Construye:** login por perfil (vista 5.1), candado de sesión, recuento ciego, comparativo y cierre con doble firma (vistas 5.17 y 5.10), y el ingreso seguro con contraseña cifrada y token firmado (14.2 y 14.3).
- **Conecta:** los estados de la sección 10 sobre los conteos ya funcionando.
- **Prueba:** dos usuarios en la misma bodega → el segundo queda bloqueado; al cerrar solo aparecen las referencias con diferencia.

Fase 6 · Multibodega, consultas y creación (sábado a. m.)

- **Construye:** el tablero en vivo (vista 5.13, WebSocket de 12.12), consultar datos e imprimir, explicar reportes, y las altas «pendientes de aprobación».
- **Conecta:** el WebSocket a todas las tabletas y el generador de archivos (12.11) a la carpeta de reportes.
- **Prueba:** dos bodegas contadas en paralelo; «¿cuánto arroz hay y dónde?» responde en voz y produce el archivo.

Fase 7 · Dashboard y pruebas finales (sábado p. m.)

- **Construye:** el consolidado con formato de carga (12.11), el tablero de Power BI (sección 13) y el manejo de errores (sin red → cola local; Gemini caído → formulario guiado, vista 5.20).
- **Conecta:** PostgreSQL → Power BI y el consolidado → carpeta de carga para My Inventory.
- **Prueba:** recorrer completa la matriz de la sección 16 sin fallos y congelar el código; el domingo solo se graba el pitch.

16. Matriz de cumplimiento: ninguna funcionalidad se queda en el papel

La tabla amarra propuesta, vistas, código y pruebas. Es la lista de chequeo del sábado en la noche:

Funcionalidad de la propuesta	Vista / Código	Prueba que lo demuestra
Abrir bodega y contar por voz	5.7 · 12.12–12.14	Bodega abierta y primer ítem guardado solo hablando.
Conciliación con nombres oficiales	5.8 · 12.7	«tabla para picar blanca» queda como TABLA ACRILICA PICAR BLANCO 50X38CM FB (97503004).
Desambiguación por voz o toque	5.8 · 12.10	Con dos candidatas, el agente lee opciones y registra la elegida.

Funcionalidad de la propuesta	Vista / Código	Prueba que lo demuestra
Anomalía de magnitud (9 vs. 90)	5.13 · 12.8	Dictar 90 cazuelas dispara «el sistema espera 10, ¿confirma?».
Validación de negativos (mini reto)	5.13 · 12.8	«Menos dos kilos» se rechaza con explicación hablada.
Corrección con rastro	11.1 · 12.10	«Uy, no: son nueve» corrige y conserva el valor anterior.
Doble conteo ciego del auditor	5.17 · 5.10 · 12.12	El auditor no ve el conteo 1; el cierre muestra solo diferencias.
Conteos simultáneos multibodega	5.13 · 12.12	Dos bodegas avanzan a la vez; el candado impide duplicados.
Crear producto/bodega pendiente	5.9 · 12.10	La creación queda «pendiente» hasta la aprobación del auditor.
Consultar datos e imprimir	5.20 · 11.3 · 12.11	«¿Cuánto arroz hay?» responde en voz y genera el archivo.
Explicar tablas y reportes	11.2 · 12.10	«No entiendo esta tabla» produce la explicación simple.
Archivo de carga para My Inventory	5.28 · 12.11	Consolidado con nombres y códigos idénticos a la base.
Dashboard conectado a Power BI	5.10 · 5.20 · 12	El informe publicado en Power BI Service se ve insertado dentro de la aplicación con los mismos datos.
Ayuda y escalamiento al auditor	12.9 (regla 7)	«Llame al administrador» registra el caso y notifica.
Ciberseguridad: token, perfiles y límites	14.2–14.4	Sin token → 401; seis claves seguidas → bloqueo; la base no expone puertos.
Contingencia sin voz ni internet	5.10 · 14.6	Con la red o Gemini caídos, el formulario guiado sigue contando y sincroniza al volver.
Pedido al almacén por voz (momento 1)	5.3 · 5.4 · 12.16	Decir «cincuenta ajiacos» produce el pedido con solo los insumos que faltan.
Cálculo por receta estándar	5.5 · 12.16	La cantidad por porción por las porciones, menos el stock de esa bodega.
Legalización del servicio (momento 3)	5.11 · 5.12 · 12.16	Lo pedido contra lo usado, con sobrante devuelto y merma registrada.

Funcionalidad de la propuesta	Vista / Código	Prueba que lo demuestra
Histórico de consumo real	5.12 · 12.16	Cada legalización alimenta el histórico que compara servicio contra servicio.
Reportes de subutilización y sobrepedido	5.21 · 12.16	Muestra qué insumos se piden y sobran, y en cuánto se sobrepide frente a la receta.
Navegación coherente entre las 29 vistas	5.0a · 12.20	El menú resalta siempre donde vive la vista; las transiciones del agente están en el enrutador.
Detalle y línea de tiempo por bodega	5.14 · 12.17	Cada bodega muestra minuto a minuto qué pasó, de la apertura a la doble firma.
Consulta de inventario por artículo	5.15 · 12.17	«¿Cuánto arroz hay?» responde total, ubicaciones y cobertura en días.
Bandeja de alertas del administrador	5.19 · 12.17	Las alertas abiertas se resuelven desde una sola cola de trabajo.
Gestión de usuarios y permisos	5.25 · 12.17	Alta, baja y perfil de cada persona sin tocar la base de datos.
Registro de trazabilidad inmutable	5.26 · 12.17	Toda acción sensible queda con persona, hora y dato; nada se borra.
Ayuda y estado del sistema en la app	5.27 · 12.17	El botón «Pedir ayuda» lleva a FAQ, soporte y estado de los servicios.
Diseño adaptable a cualquier dispositivo	5.21 · 5b	La misma pantalla en 375, 768 y 1440 px: el micrófono y los botones se reorganizan solos.

Si el sábado en la noche las veintinueve filas pasan, el agente cumple todo lo prometido en la propuesta: se construyó por fases probadas, cada conexión tiene su código y cada funcionalidad tiene su pantalla. Lo que queda para el domingo es solo contarlos en dos minutos.